

ARM PrimeCell

Advanced Audio CODEC Interface (PL041)

Design Manual



ARM PrimeCell Advanced Audio CODEC Interface (PL041) Design Manual

© Copyright ARM Limited 1999. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell Advanced Audio CODEC Interface

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	AACI DESIGN OVERVIEW	3
4.1	Signal Description	3
4.2	Programmer's Model	5
4.3	Block Partitioning	7
4.3.1	APB Interface and Register Block	9
4.3.2	Timing Control	10
4.3.3	Frame Decoder	10
4.3.4	Slot 0 Generator	12
4.3.5	Frame Generator	13
4.3.6	Channel Block	13
4.3.7	Transmit Control Logic	16
4.3.8	Receive Control Logic	17
4.3.9	Interrupt Generation Logic	17
4.3.10	Synchronisers	17
5	AACI IMPLEMENTATION DETAILS	20
5.1	Design Hierarchy	20
5.2	AACI Top Level Module (Aaci)	21
5.3	APB Interface and Register Block (AaciApbifReg)	21
5.3.1	Input Module	21
5.3.2	Output module	23
5.3.3	AACI Registers	23
5.4	Timing Controller (AaciTmgCntl)	25
5.4.1	AACIBITCLK Count Generation	26
5.4.2	AACISYNC Signal Generation	26
5.5	Frame Decoder (AaciFrmDec)	26
5.5.1	Frame State Machine	27
5.5.2	Frame Decoder	27

5.6	Slot 0 Generator (AaciSlot0Gen)	29
5.7	Frame Generator (AaciFrmGen)	31
5.7.1	Slot 2 Valid bit generation	31
5.7.2	Slot 0 Register	31
5.7.3	FIFO Data Mux	32
5.7.4	Transmit shift register	32
5.7.5	Integration Test Mux	32
5.7.6	Low Power Mode Detection	32
5.8	Transmit Channel Logic (AaciTxChannel)	33
5.9	Transmit FIFO Control Logic (AaciTxFCntl)	34
5.10	Transmit FIFO Register File (AaciTxRegFile)	36
5.11	Receive Channel Logic (AaciRxChannel)	36
5.12	Receive FIFO Control Logic (AaciRxFCntl)	38
5.13	Receive FIFO Register File (AaciRxRegFile)	40
5.14	DMA Transmit Channel Logic (AaciDMATChannel)	41
5.14.1	DMA signal generation	41
5.15	DMA Transmit FIFO Control Logic (AaciDMATxFCntl)	42
5.16	DMA Receive Channel Logic (AaciDMARChannel)	42
5.16.1	DMA signal generation	43
5.17	DMA Receive FIFO Control Logic (AaciDMARxFCntl)	44
5.18	Transmit Control Logic (AaciTxCntl)	45
5.19	Receive Control Logic (AaciRxCntl)	46
5.20	Interrupt Generator (AaciIntrGen)	47
5.20.1	Interrupt Generation Logic	47
5.20.2	Integration Test Muxes	49
5.21	PCLK to AACIBITCLK domain Synchronisers (AaciPtoBSync)	50
5.22	AACIBITCLK to PCLK domain Synchronisers (AaciBtoPSync)	50
5.23	Revision Designator block (AaciRevAnd)	50
5.24	AACI Integration Test Strategy	51
5.24.1	Input Integration Test Harness	51
5.24.2	Output Integration Test Harness	51
5.25	Frequency Considerations	52
5.26	Software Restrictions	53

6	AACI FUNCTIONAL VERIFICATION TEST DETAILS	55
6.1	AACI VHDL Verification Testbench	55
6.1.1	Unit Under Test	57
6.1.2	APB Bridge Model	57
6.1.3	Trickbox	57
6.1.4	Protocol Checker	57
6.1.5	APB Multiplexer	57
6.1.6	Test Vectors	58
6.1.7	Generics	58
6.1.8	Running Simulations	59
6.2	AACI Verilog Verification Testbench	59
6.2.1	Unit Under Test	62
6.2.2	APB Bridge Model	62
6.2.3	Trickbox	62
6.2.4	Protocol Checker	62
6.2.5	APB Multiplexer	62
6.2.6	Vector Sets	63
6.2.7	Parameters	63
6.2.8	Running Simulations	64
6.3	Trickbox Description	64
6.3.1	Trickbox Signal Description	65
6.3.2	Trickbox Register Description	67
6.3.3	Trickbox Functional Description	72
6.4	Functional Tests	73
6.4.1	Register Tests	73
6.4.2	AACISYNC protocol test	74
6.4.3	AACIRESET protocol test	74
6.4.4	Warm Reset protocol Check	74
6.4.5	Cold Reset protocol check in power down mode	75
6.4.6	Interrupt Tests	75
6.4.7	DMA Tests	79
6.4.8	Data Setup/Hold Test	85
6.4.9	FIFO Status Flag Test	85
6.4.10	FIFO pointer cross-over tests	86
6.4.11	Data Tests	87
6.4.12	Loop back Test	106
6.5	Verification Test Vector Generation	107
7	AACI INTEGRATION TEST DETAILS	108
7.1	AACI VHDL Integration testbench	108
7.1.1	Unit Under Test	110
7.1.2	Test Vectors	110
7.1.3	Running Simulations	111
7.2	AACI Verilog Integration testbench	111
7.2.1	Unit Under Test	113
7.2.2	Test Vectors	114
7.2.3	Running Simulations	114

7.3	Integration Trickbox	115
7.3.1	Integration Trickbox Signal Description	115
7.3.2	Integration Trickbox Functional Description	115
7.4	Integration Tests	116
7.4.1	Integration Testing of primary inputs and outputs	116
7.4.2	Integration Testing of intra-chip outputs	116
7.4.3	Integration Testing of intra-chip inputs	117
7.5	Integration Test Vector Generation	117

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A01	1st June, 2000	First draft.
A02	7 th June, 2000	Incorporated review changes.
A03	7 th August, 2000	Incorporated review changes.
A	5 th September, 2000	First Release

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM DDI 0173	ARM PrimeCell Advanced Audio CODEC Interface (PL041) Technical Reference Manual
2	PL041 INTM 0000	ARM PrimeCell Advanced Audio CODEC Interface (PL041) Integration Manual
3	ARM DUI 0006	AMBA Compliance Test Suite User Guide
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide
6	SCB00-QPRO-0008-A03	ARM PrimeCell Integration Vector Strategy

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASIC	Application Specific Integrated Circuit
FIFO	First-In-First-Out buffer
HDL	Hardware Description Language
TIC	Test Interface Controller
VHDL	VHSIC Hardware Description Language
VHSIC	Very High Speed Integrated Circuit
PCM	Pulse Code Modulation

TDM	Time Division Multiplexed
AACI	Advanced Audio CODEC Interface

2 SCOPE

This document describes the design and implementation of the AACI (Advanced Audio CODEC Interface). The design description is split between two sections – Section 4 and Section 5. The testbenches and the test programs for functional verification and integration testing are described in Section 6 and Section 7.

Section 4 presents an overview of the AACI design, describing the functional partitioning of the AACI and the signal interconnections. Section 5 presents detailed descriptions on the AACI design. Section 6 describes the testbench and the test programs used to generate test vectors for functional verification of the AACI. Section 7 describes the testbenches and the test programs used to generate test vectors for Integration Testing of the AACI.

3 INTRODUCTION

The AACI is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM PrimeCell Advanced Audio CODEC Interface (PL041) Technical Reference Manual [Ref.1]. The ARM PrimeCell Advanced Audio CODEC Interface (PL041) Integration Manual [Ref.2] describes how the block may be integrated into a typical industry standard ASIC design flow.

The AMBA Compliance Test Suite User Guide [Ref.3] is a source for further information about AMBA bus compliance testbenches.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of TICTalk test vectors for production test.

4 AACI DESIGN OVERVIEW

The Advanced Audio CODEC Interface (AACI) is an Advanced Microcontroller Bus Architecture (AMBA) slave block that connects to the Advanced Peripheral Bus (APB). The AACI offers a means to communicate to off-chip CODECs that support the AC-Link protocol – examples include the Wolfson Microelectronics WM9704 device, the CS4297 CODEC from Cirrus Logic and the LM4549 CODEC from National Semiconductors. This digital link (AC-Link) is a bi-directional, five-wire – AACISYNC, AACIBITCLK, AACISDATAOUT, AACISDATAIN and AACIRESET – serial Time Division Multiplexed (TDM) format interface, designed for dedicated point-to-point interconnects. It is a fixed rate, serial PCM digital stream and handles multiple input, and output audio streams, as well as control register accesses employing a TDM scheme. The AC-Link architecture divides each audio frame into 12 outgoing and 12 incoming data streams, each with 20-bit sample resolution. The AACI has four channels which can operate simultaneously therefore allowing support for four different sample rates. All channels can be serviced by the microprocessor and channel 1 can also be serviced by a Direct Memory Access Controller.

4.1 Signal Description

The following tables summarise the AACI signal list. **Table 4.1-1** lists the APB interface signals, while **Table 4.1-2** lists the module-specific signals.

Signal Name	Type	Source / Destination	Description
PCLK	IN	Clock Generator	APB Clock.
PRESETn	IN	Reset Controller	APB Reset (active low).
PADDR[11:2]	IN	APB Bridge	This is part of the peripheral address bus, which is used for decoding the internal address space.
PWDATA[31:0]	IN	APB Bridge	Data is written into the peripheral on this bus.
PRDATA[31:0]	OUT	APB Data Multiplexer.	Data is read out of the peripheral via this bus.
PENABLE	IN	APB Bridge	This signal is used to time all accesses on the peripheral bus.
PWRITE	IN	APB Bridge	When HIGH, this signal indicates a write into the peripheral and when LOW, a read from the peripheral.
PSEL	IN	APB Bridge	When HIGH, this signal indicates that the peripheral has been selected by the APB Bridge.

Table 4.1-1 APB Signal descriptions

Signal Name	Type	Source / Destination	Description
AACIBITCLK	IN	Audio CODEC	Clock supplied by the Audio CODEC, fixed at 12.288 MHz.
nAACIBITCLK	IN	Clock Generator	Inverted AACIBITCLK, fixed at 12.288 MHz.
nAACIBITCLKRST	IN	Reset Controller	Reset signal to AACIBITCLK clock domain. Active LOW.
nFAACIBITCLKRST	IN	Reset Controller	Reset signal to nAACIBITCLK clock domain. Active LOW.
AACISYNC	OUT	Audio CODEC	AACIBITCLK divided by 256. (48 kHz). Also output asynchronously when Audio CODEC is in Warm reset state.
AACIRESET	OUT	Audio CODEC	Reset signal from the AACI to Audio CODEC.

AACISDATAIN	IN	Audio CODEC	Serial Data In from Audio CODEC.
AACISDATAOUT	OUT	Audio CODEC	Serial Data Out to Audio CODEC.
SCANENABLE	IN	Test Controller	Scan Enable signal for all clock domains.
SCANINPCLK	IN	Test Controller	Scan Input signal for PCLK domain.
SCANINBITCLK	IN	Test Controller	Scan Input signal for AACIBITCLK domain.
SCANINnBITCLK	IN	Test Controller	Scan Input signal for nAACIBITCLK domain.
SCANOUTPCLK	OUT	Test Controller	Scan Output signal for PCLK domain.
SCANOUTBITCLK	OUT	Test Controller	Scan Output signal for AACIBITCLK domain.
SCANOUTnBITCLK	OUT	Test Controller	Scan Output signal for nAACIBITCLK domain.
AACIINTR	OUT	Interrupt Controller	Combined AACI interrupt. Asserted when any of the individual interrupts is asserted.
AACIRXINTR1-4	OUT	Interrupt Controller	Receive FIFOs Interrupt.s. Asserted when they are at least half full.
AACITXINTR1-4	OUT	Interrupt Controller	Transmit FIFOs Interrupts. Asserted when they are at least half-empty.
AACIRXTOINTR1-4	OUT	Interrupt Controller	Receive Timeout Interrupts. Asserted when the receive FIFOs are not empty and no further data is received for TOC number of frames programmed in AACIRXCR registers.
AACIRXTOFEINTR1-4	OUT	Interrupt Controller	Receive Timeout Empty Interrupts. Asserted when the receive FIFOs are empty and no further data is received for TOC number of frames programmed in AACIRXCR registers.
AACITXCINTR1-4	OUT	Interrupt Controller	Transmit Complete Interrupts. Asserted when transmit FIFOs are empty and transmit shift registers are empty.
AACIURINTR1-4	OUT	Interrupt Controller	Transmit Underrun Interrupts. Asserted when data for all slots programmed in the AACITXCR registers are not present in transmit FIFOs at the time of transmission.
AACIORINTR1-4	OUT	Interrupt Controller	Receive Overrun Interrupts. Asserted if data is received and the Receive FIFOs are already full.
AACIWINTR	OUT	Interrupt Controller	Wake up Interrupt. Asserted by a high on AACISDATAIN in the absence of AACIBITCLK.
AACIGPIOINTR	OUT	Interrupt Controller	GPIO Interrupt. Asserted when bit 0 of slot 12 of the incoming AACISDATAIN is 1.
AACIS1RXINTR	OUT	Interrupt Controller	Slot 1 Rx Interrupt. Asserted when AACISL1RX register has new data that has not been read.
AACIS1TXINTR	OUT	Interrupt Controller	Slot 1 Tx Interrupt. Asserted when there is no data present in AACISL1TX register.
AACIS2RXINTR	OUT	Interrupt Controller	Slot 2 Rx Interrupt. Asserted when AACISL2RX register has new data that has not been read.
AACIS2TXINTR	OUT	Interrupt Controller	Slot 2 Tx Interrupt. Asserted when there is no data present in AACISL2TX register.
AACIS12RXINTR	OUT	Interrupt Controller	Slot 12 Rx Interrupt. Asserted when AACISL12RX

			register has new data that has not been read.
AACIS12TXINTR	OUT	Interrupt Controller	Slot 12 Tx Interrupt. Asserted when there is no data present in AACISL12TX register.
AACIDMASREQRX	OUT	DMA Controller	Single Word DMA transfer request. Asserted when Receive FIFO 1 fill level is 1, 2 or 3 if Receive Timeout is not asserted. Asserted when Receive FIFO 1 fill level is 2 or 3 if Receive Timeout is asserted.
AACIDMALSREQRX	OUT	DMA Controller	Last Single Word DMA transfer request. Asserted when Receive FIFO 1 fill level is 1 and Receive Timeout Interrupt is present.
AACIDMABREQRX	OUT	DMA Controller	Burst DMA transfer request. Asserted when Receive FIFO 1 fill level is 4 or more if Receive Timeout is not asserted. Asserted when Receive FIFO 1 fill level is more than 4 if Receive Timeout is asserted.
AACIDMALBREQRX	OUT	DMA Controller	Burst DMA transfer request. Asserted when Receive FIFO 1 fill level is 4 and Receive Timeout Interrupt is present for Receive FIFO 1.
AACIDMACLRRX	IN	DMA Controller	DMA Receive Request Clear. Clears the DMA request signals for Receive when asserted.
AACIDMABREQTX	OUT	DMA Controller	Burst DMA transfer request. Asserted when Transmit FIFO 1 fill level is 4 or less.
AACIDMACLRTX	IN	DMA Controller	DMA Transmit Request Clear. Clears the DMA request signal for Transmit when asserted.

Table 4.1-2 Block-specific signal descriptions

4.2 Programmer's Model

All the register addresses in the AACI are offset from the AACI base address. Locations at offsets 0x110-0x180 are reserved for possible future extensions. The AACI Functional Mode registers are shown in the table below:

Address	Type	Width	Reset Value	Name	Description
AACI base + 0x00	R/W	29/29	0x00000000	AACIRXCR1	Control register for receive FIFO1.
AACI base + 0x04	R/W	17/17	0x00000	AACITXCR1	Control register for transmit FIFO1.
AACI base + 0x08	R	12	0x00B	AACISR1	Status register channel 1.
AACI base + 0x0C	R	7	0x00	AACIISR1	Interrupt Status channel 1.
AACI base + 0x10	R/W	7/7	0x00	AACIIE1	Interrupt Enable channel 1.
AACI base + 0x14	R/W	29/29	0x00000000	AACIRXCR2	Control register for receive FIFO2.
AACI base + 0x18	R/W	17/17	0x00000	AACITXCR2	Control register for transmit FIFO2.
AACI base + 0x1C	R	12	0x00B	AACISR2	Status register channel 2.
AACI base + 0x20	R	7	0x00	AACIISR2	Interrupt Status channel 2.
AACI base + 0x24	R/W	7/7	0x00	AACIIE2	Interrupt Enable channel 2.
AACI base + 0x28	R/W	29/29	0x00000000	AACIRXCR3	Control register for receive FIFO3.
AACI base + 0x2C	R/W	17/17	0x00000	AACITXCR3	Control register for transmit FIFO3.
AACI base + 0x30	R	12	0x00B	AACISR3	Status register channel 3.

AACI base + 0x34	R	7	0x00	AACIISR3	Interrupt Status channel 3.
AACI base + 0x38	R/W	7/7	0x00	AACIIE3	Interrupt Enable channel 3.
AACI base + 0x3C	R/W	29/29	0x00	AACIRXCR4	Control register for receive FIFO4.
AACI base + 0x40	R/W	17/17	0x00000000	AACITXCR4	Control register for transmit FIFO4.
AACI base + 0x44	R	12	0x00B	AACISR4	Status register channel 4.
AACI base + 0x48	R	7	0x00	AACIISR4	Interrupt Status channel 4.
AACI base + 0x4C	R/W	7/7	0x00	AACIIE4	Interrupt Enable channel 4.
AACI base + 0x50	R	20	0x00000	AACISL1RX	Data received on SLOT1
AACI base + 0x54	R/W	20/20	0x00000	AACISL1TX	Data to be transmitted on SLOT1
AACI base + 0x58	R	20	0x00000	AACISL2RX	Data received on SLOT2
AACI base + 0x5C	R/W	20/20	0x00000	AACISL2TX	Data to be transmitted on SLOT2
AACI base + 0x60	R	20	0x00000	AACISL12RX	Data received on SLOT12
AACI base + 0x64	R/W	20/20	0x00000	AACISL12TX	Data to be transmitted on SLOT12
AACI base + 0x68	R	14	0x0A80	AACISLFR	Slot Flag register
AACI base + 0x6C	R	8	0x00	AACISLISTAT	Slot Interrupt status register
AACI base + 0x70	R/W	9/9	0x000	AACISLIEN	Slot Interrupt enable register
AACI base + 0x74	W	13	-	AACIINTCLR	Interrupt Clear register
AACI base + 0x78	R/W	12/12	0x000	AACIMAINCR	Main Control register
AACI base + 0x7C	R/W	1/1	0x1	AACIRESET	RESET control register.
AACI base + 0x80	R/W	1/1	0x0	AACISYNC	SYNC control register.
AACI base + 0x84	R	28	0x00000000	AACIALLINTS	All FIFO interrupt status register.
AACI base + 0x88	R	2	0x0	AACIMAINFR	Main Flag register.
AACI base + 0x90 – 0xAC	R/W	32/32	0x00000000	AACIDR1	Data read or written from/to FIFO1.
AACI base + 0xB0 – 0xCC	R/W	32/32	0x00000000	AACIDR2	Data read or written from/to FIFO2.
AACI base + 0xD0 – 0xEC	R/W	32/32	0x00000000	AACIDR3	Data read or written from/to FIFO3.
AACI base + 0xF0 – 0x10C	R/W	32/32	0x00000000	AACIDR4	Data read or written from/to FIFO4.

Table 4.2-1 Memory map for Functional registers in the AACI

Data from/to the FIFOs can be read/written to a range of 8 sequential addresses, this allows the microprocessor to use its load and store multiple operands to read/write from/to the FIFO.

The Test registers for the AACI are listed in **Table 4.2-2**.

Address	Type	Width	Reset Value	Name	Description
AACI base + 0x180	R/W	4/4	0x0	AACITCR	Test Control Register
AACI base + 0x184	R/W	3/2	0x0	AACIITIP	Integration Test Input Read/Set Register
AACI base + 0x188	R/W	30/30	0x00000000	AACIITOP0	Integration Test Output Set/Read Register 0

AACI base + 0x18C	R/W	13/13	0x0000	AACIITOP1	Integration Test Output Set/Read Register 1
-------------------	-----	-------	--------	-----------	---

Table 4.2-2 Memory map for Test Registers in the AACI

The AACI Identification registers are listed in **Table 4.2-3** as shown below.

Address offset	Type	Width	Reset Value	Name	Description
AACI base + 0xFE0	R	8	0x41	AACIPERIPHID0	Peripheral Identification register bits 7:0.
AACI base + 0xFE4	R	8	0x10	AACIPERIPHID1	Peripheral Identification register bits 15:8.
AACI base + 0xFE8	R	8	0x04	AACIPERIPHID2	Peripheral Identification register bits 23:16.
AACI base + 0xFEC	R	8	0x00	AACIPERIPHID3	Peripheral Identification register bits 31:24.
AACI base + 0xFF0	R	8	0x0D	AACIPCELLID0	PrimeCell Identification register bits 7:0.
AACI base + 0xFF4	R	8	0xF0	AACIPCELLID1	PrimeCell Identification register bits 15:8.
AACI base + 0xFF8	R	8	0x05	AACIPCELLID2	PrimeCell Identification register bits 23:16.
AACI base + 0xFFC	R	8	0xB1	AACIPCELLID3	PrimeCell Identification register bits 31:24.

Table 4.2-3 Memory map for Identification registers in the AACI

4.3 Block Partitioning

The AACI contains logic that controls the AC-Link to the Audio CODEC and an interface to the APB.

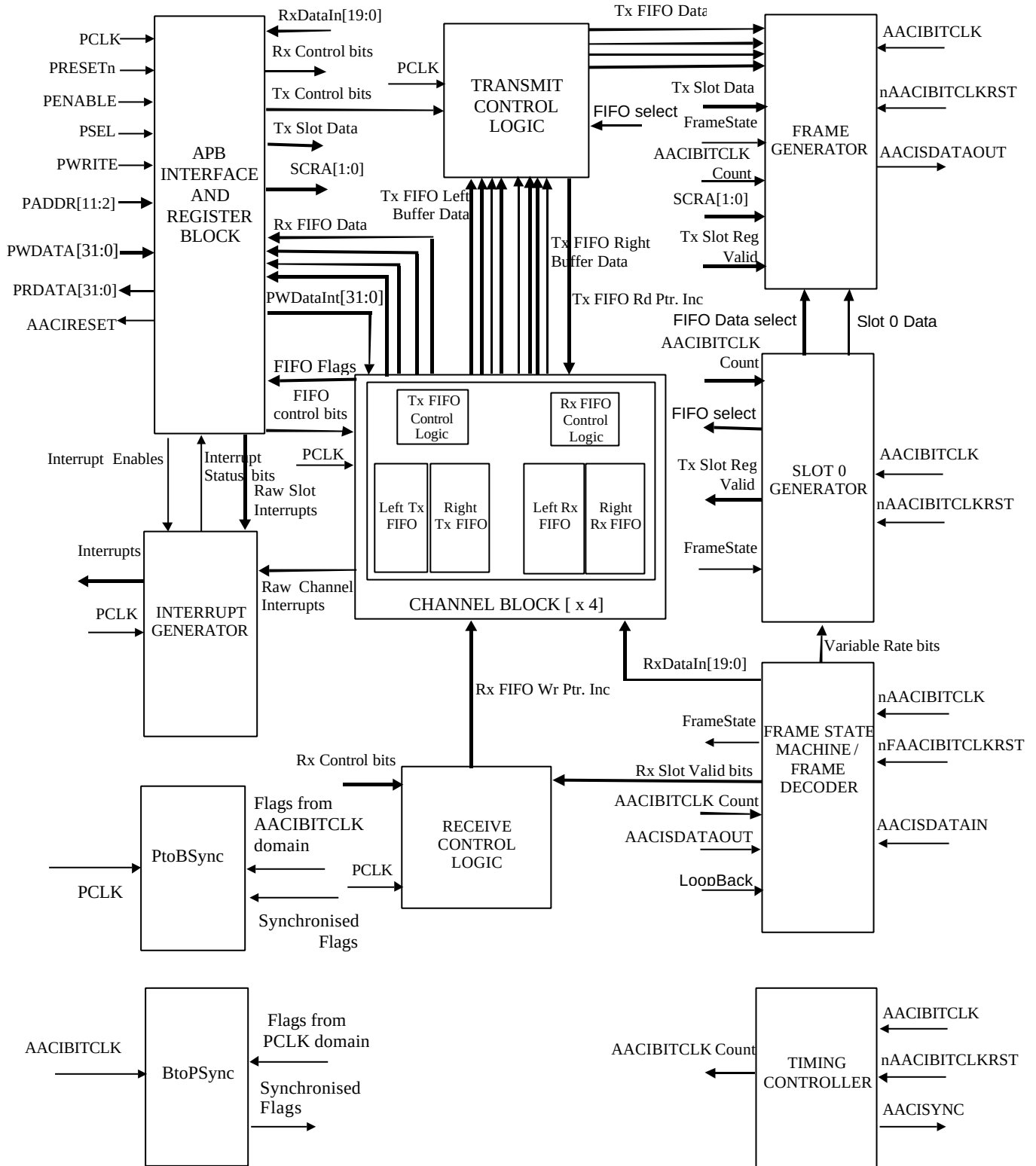


Figure 4.3-1 AACI Block Diagram

The sub-modules in the design are:

- APB Interface and Register Block
- Timing Controller
- Frame Decoder
- Slot 0 Generator
- Frame Generator
- Channel Block
- Transmit Control Logic
- Receive Control Logic
- Interrupt Generation Logic
- Synchronisers

The design of these sub-modules is detailed below.

4.3.1 APB Interface and Register Block

The APB interface and Register block consists of the APB interface and the AACI registers. The APB interface deals with reads and writes to registers from the APB. A simplified block diagram of the APB interface and Register block is shown in **Figure 4.3-2**

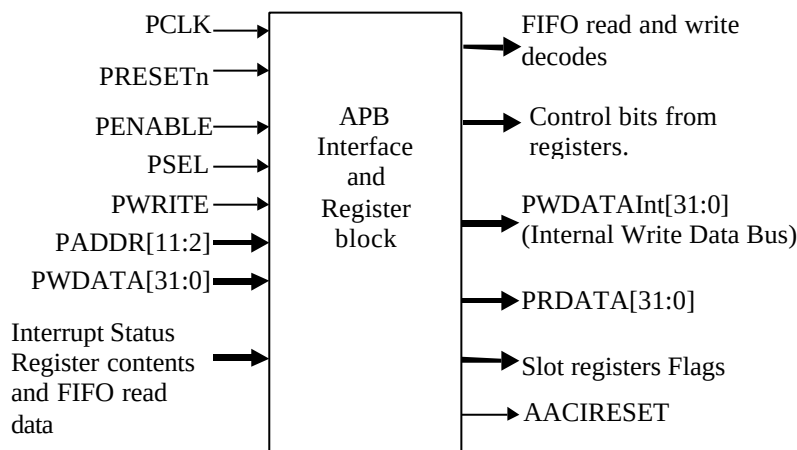


Figure 4.3-2 APB Interface and Register Block

This block decodes the accesses to the various registers of the AACI and generates write enables for all the writable registers. The writable registers of the AACI are AACITXCR1-4, AACIRXCR1-4, AACIIE1-4, AACISL1TX, AACISL2TX, AACISL12TX, AACISLIEN, AACIINTCLR, AACIMAINCR, AACIRESET, AACISYNC and AACIDR1-4. Of these registers, AACIINTCLR is a virtual register with no storage element. Writes to this register are used to clear the RxOverrun, RxTOFE, TxUnderrun and Wakeup interrupts in the AACI. AACIDR1-4 are the FIFO registers. Reads and Writes to the AACIDR registers result in reads and writes to the Channel FIFOs. Data from/to the FIFOs can be read/written to a range of 8 sequential addresses. This allows the microprocessor to use its load and store multiple operands to read/write to the FIFOs. The internal databus, for the AACI, PWDATAInt[31:0], is generated by gating the input data bus, PWDATA[31:0] with PSEL and PWRITE. The APB interface and register block also implements the output read data multiplexer and the output register that form the read interface. A 2-buffer mechanism is employed to synchronise the slot valid bits and the transmit enable bits of the AACITXCR1-4 registers. Hardware masking is employed to ignore writes to the AACITXCR1-4 registers during the 2-buffer synchronisation in order to prevent metastability. The SlxTxEmpty and SlxRxValid [x = 1, 2, 12]

flags which indicate the empty/full status of the slot Tx/Rx registers are also generated in this block. The AACIRESET output of the AACI is driven by the ForcedRESET bit in the AACIRESET register.

4.3.2 Timing Control

The Timing Controller Block consists of an AACIBITCLK counter, which is used for the generation of SyncBITCLK signal and AACISYNC signal. The AACIBITCLK counter is also used by the Frame Generator and Frame Decoder sections of the AACI to transmit / receive data corresponding to the slots based on the counts. A block diagram of the Timing Controller is shown in **Figure 4.3-3**.

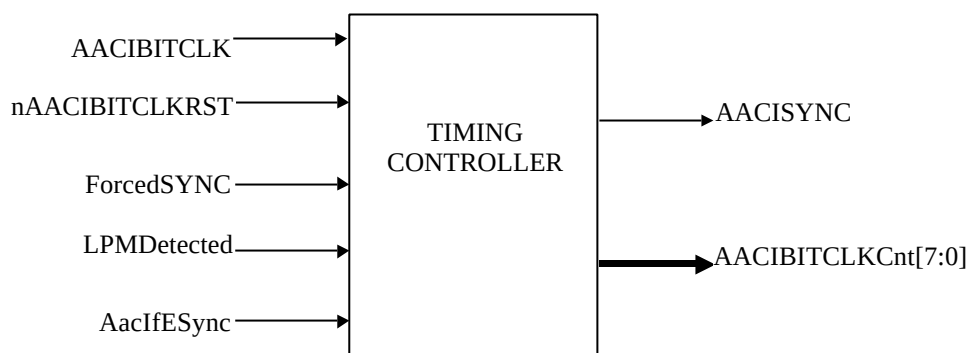


Figure 4.3-3 Timing Controller

The AACIBITCLK counter increments by one on every rising edge of AACIBITCLK when the AACI is enabled. The reset value of the counter is 255. On the next rising edge of AACIBITCLK, the counter wraps around to 0. The AACISYNC output of the AACI is generated as a logical OR of the ForcedSYNC bit of the AACISYNC register and SyncBITCLK, which is generated from the value of the AACIBITCLK counter. SyncBITCLK is set high on the rising edge of AACIBITCLK when the AACIBITCLK count is 255, and it is cleared at the rising edge of the AACIBITCLK when the count is 15. SyncBITCLK is then held low till the count reaches 255 again. Thus the AACISYNC signal has a high for 16 AACIBITCLKs and low of 240 AACIBITCLKs. When the AC-Link enters low power mode, the counter is loaded with 243 so that the AACISYNC pulse can be generated as soon as possible when the AC-Link comes out of low power mode. ForcedSYNC is to be asserted for a minimum of 1.3us to restart the AC-Link by a warm reset.

4.3.3 Frame Decoder

This module implements the decoder for the incoming data AACISDATAIN. It also contains the Frame State Machine, which is used for Frame Generation/Decode.

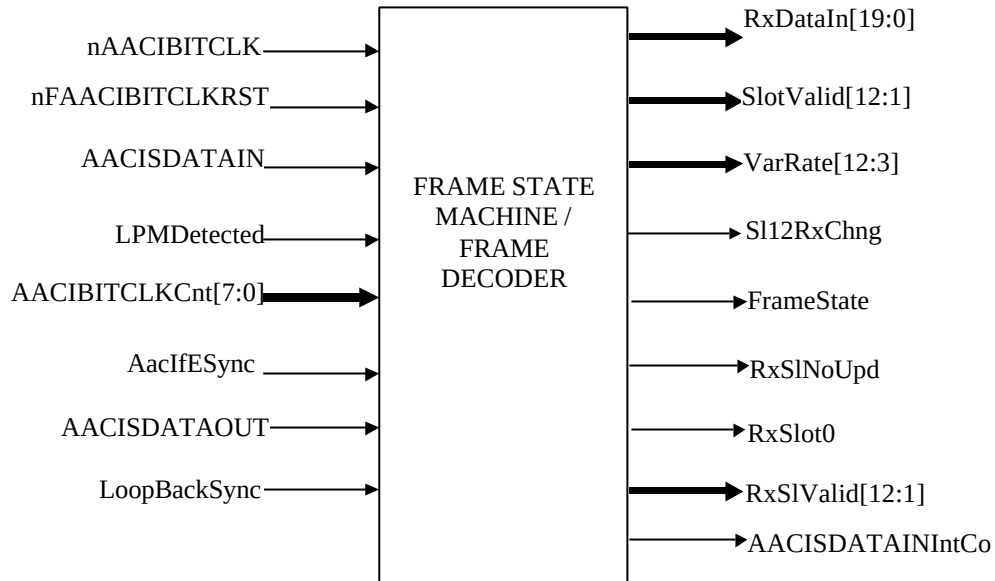


Figure 4.3-4 Frame Decoder

Frame State Machine

The Frame State Machine has two states – ST_CODECIDLE and ST_CODECREADY. When nFAACIBITCLKRST is asserted, the Frame State Machine is initialised in the ST_CODECIDLE state. If the CODEC Ready bit (bit 15 in slot 0 of AACISDATAIN) is 1, or if the LoopBack bit is set in the AACIMAINCR register, the Frame State Machine goes to ST_CODECREADY. If the AC-Link is in Low Power Mode or when the CODEC ready bit is 0, then the Frame State Machine goes to ST_CODECIDLE. Frame Generation and Decoding happens only when the state machine is in ST_CODECREADY state.

Frame Decoder

The Frame Decoder decodes the data for each slot in the incoming frame and drives the slot valid information in slot 0 on SlotValid[12:1] and the data in slots 1 to 12 on RxDataIn[19:0].

When the Frame State Machine is in ST_CODECREADY state, on every rising edge of nAACIBITCLK, the data in the receive shift register (RxShift[19:0]) is shifted left by one bit and data received on AACISDATAIN is stored in the LSB (bit 0) of the receive shift register. When the Frame State Machine is in ST_CODECREADY state, a signal DecSlot0 is asserted for one AACIBITCLK duration at the end of slot 0 reception. This signal is used as a qualifier to load the slot valid information from the receive shift register into the SlotValid[12:1] register. Bits 12 to 1 of this register indicate the validity of the slots 1 to 12 in the incoming frame. In ST_CODECREADY state, a signal DS1to12 is asserted for one AACIBITCLK duration at the end of reception of data for slots 1 to 12. This signal is used to load the data corresponding to each slot from the receive shift register into the RxDataIn[19:0] bus when the data corresponding to that slot has been completely shifted in. Data on the RxDataIn[19:0] bus is written into the Rx FIFOs and the slot Rx registers. When data corresponding to a valid slot has been loaded into RxDataIn[19:0], one bit in RxSIVValid[12:1] corresponding to that slot is asserted for one slot duration indicating the slot number of the data in RxDataIn[19:0]. The RxSIVValid[12:1] vector is used to route the received data on the RxDataIn[19:0] bus into the correct channels receive FIFO or into the slot receive registers. At the end of slot 1 reception, the Variable Sample Rate Conversion VSRC bits from the receive shift register are stored in the VarRate[12:3] register.

RxShift[19:0], DecSlot0, DS1to12, SlotValid[12:1], RxSIVValid[12:1], VarRate[12:3] and RxDataIn[19:0] are initialised to zero on assertion of nFAACIBITCLKRST signal or when the Frame State machine is in ST_CODECIDLE state.

A multiplexer is used to select the AACISDATAIN input of the Frame Decoder module. When the LoopBack bit of the AACIMAINCR is set to 1, the AACISDATAOUT output from the Frame Generator is connected to the input

AACISDATAIN of the Frame Decoder. Otherwise the external AACISDATAIN from the AC-Link is connected to the AACISDATAIN of the Frame Decoder.

4.3.4 Slot 0 Generator

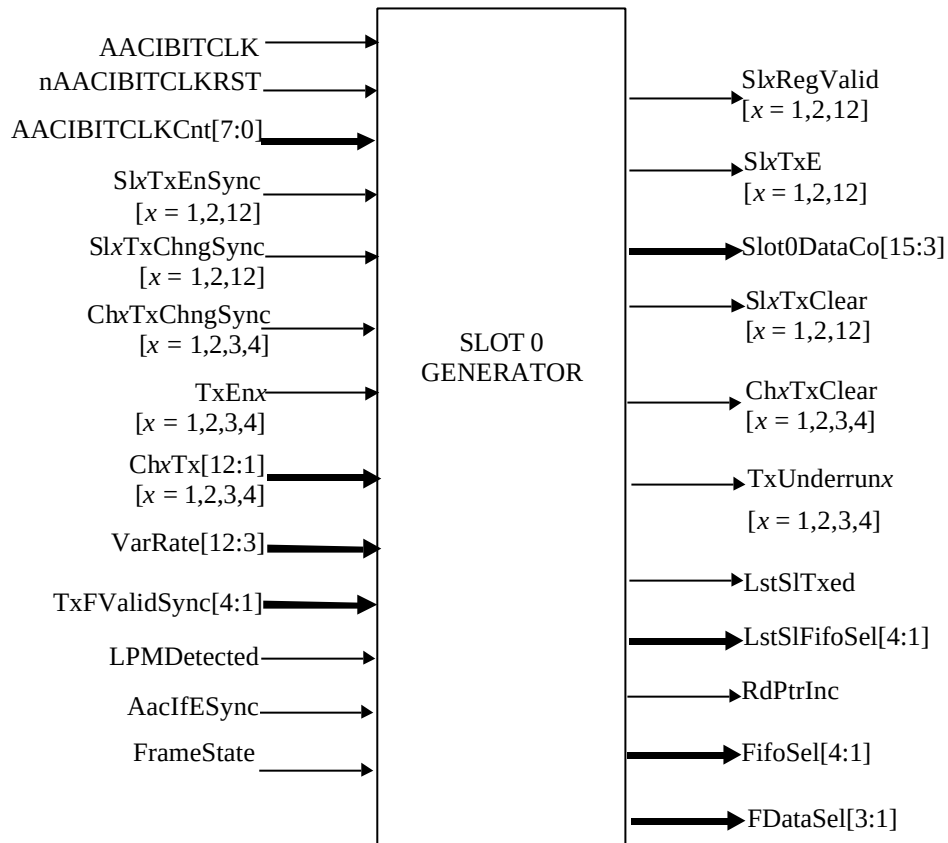


Figure 4.3-5 Slot0 Generator

This module generates the slot 0 information depending on whether data for each of the slots is present in the channels or the slot transmit registers. It also generates SI12RegValid, SI2RegValid and SI1RegValid signals when slot Tx register data is available and data for the corresponding slot is not available in the channel. SI1TxEx, SI2TxEx and SI12TxEx are toggled when the data from the corresponding slot transmit register has been loaded into the transmit shift register. These signals are used to generate the SlxTxEmpty [x = 1, 2, 12] flags. The TxUnderrun flags for any of the four transmit channels are toggled when the Tx FIFO of that channel does not contain enough data to be transmitted in the next frame. The LstSlTxed signal is toggled when the last slot to be transmitted from a channel in the current frame has been shifted out of the transmit shift register. One bit of a vector LstSlFifoSel[4:1] is asserted when LstSlTxed toggles to indicate the channel that caused it to toggle. These 2 signals are used to deassert the TxBusy signal of that channel. This module also generates a toggle signal called RdPtrInc that indicates that the read pointer of one of the channel Tx FIFOs has to be incremented. The exact channel is selected using the FifoSel[4:1] vector. One bit in this vector is asserted when the data from one of the channels has been loaded into the transmit shift register. The FDataSel[3:1] vector is used in the Frame Generator block to select data from one of the 4 Tx FIFOs for transmission. The slot 0 generator block asserts one bit in this vector corresponding to the channel from which the data in the next slot has to be transmitted. **Figure 4.3-5** shows the block diagram of the Slot 0 Generator.

4.3.5 Frame Generator

This module drives AACISDATAOUT to the Audio CODEC, selecting the data from the slot transmit registers and the transmit FIFOs. Block diagram of the Frame Generator is given in **Figure 4.3-6**.

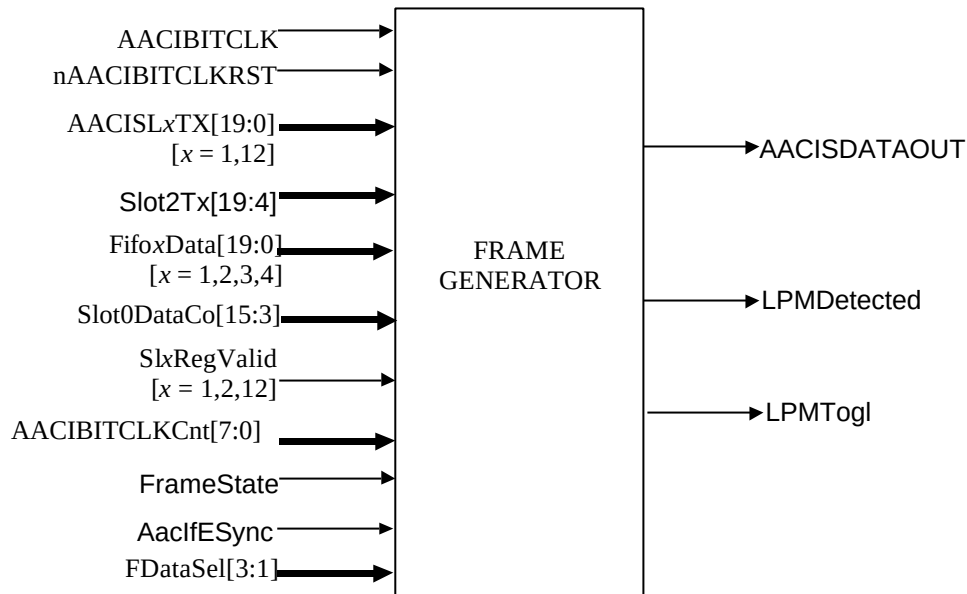


Figure 4.3-6 Frame Generator

Slot0Data[14:3] is clocked out from the combinational Slot0Data (Slot0DataCo[14:3]) at the start of slot 0 transmission when the frame state machine is in ST_CODECREADY state and the AACI is enabled. At the start of slot 0 transmission, data from the combinational Slot0Data is loaded into the transmit shift register if it is a Primary CODEC register access or if the slot valid bit of slot 1 is not asserted. If the slot valid bit of slot 1 is set, and if it is a Secondary CODEC register access, the slot valid bits of slot 1 and 2 are zeroed and the SCRA[1:0] bits are transmitted on bits 1 and 0 of slot 0. If slot 1 data indicates a CODEC register read access, even if data for slot 2 is available, the slot valid bit of slot 2 is zeroed. At all other slot boundaries, data from one of the channels or the slot registers is loaded into the transmit shift register if the valid bit of that slot in slot 0 was 1. Data for slots 1, 2 and 12 are sourced from either the channels or the slot registers depending on the Sl1RegValid, Sl2RegValid and Sl12RegValid signals respectively from the Slot 0 Generator block. The channel that is to be the source of data for a slot is selected based on the vector FDataSel[3:1] from the Slot 0 Generator. The value on bit 19 (MSB) of the transmit shift register is transmitted on the AACISDATAOUT line and the remaining bits are shifted left by one bit on every rising edge of AACIBITCLK. When the frame state machine is in ST_CODECREADY state, the transmit shift register is zeroed and zeros are transmitted on the AACISDATAOUT line.

The CODEC is programmed into low power mode only through the slot transmit registers. If the data from the slot 1 transmit register indicates an access to the Powerdown register of the CODEC (address 26h) and bit 16 of the slot 2 transmit register is set, the LPMDetected signal is asserted. The LPMTogl signal is toggled at the end of slot 2 transmission to indicate the entry of the AC-Link into Low Power Mode.

4.3.6 Channel Block

Each Channel consists of a receive FIFO, a transmit FIFO and the control logic for the transmit and receive FIFOs. This block is instantiated four times to generate 4 Channels for the device. Each FIFO has two 20-bit wide and 4 deep buffers organised as a left buffer and a right buffer. Each FIFO acts as a 20x8 FIFO in non-Compact Mode, and a 40x4 (32x4 effectively) FIFO in Compact Mode. The advantage of such an arrangement over a simple 20x8 FIFO is that all the unpacking and resizing logic will be transferred to the non APB side of the FIFOs to facilitate meeting of timing requirements on the APB side. **Figure 4.3-7** shows the block diagram of a Channel.

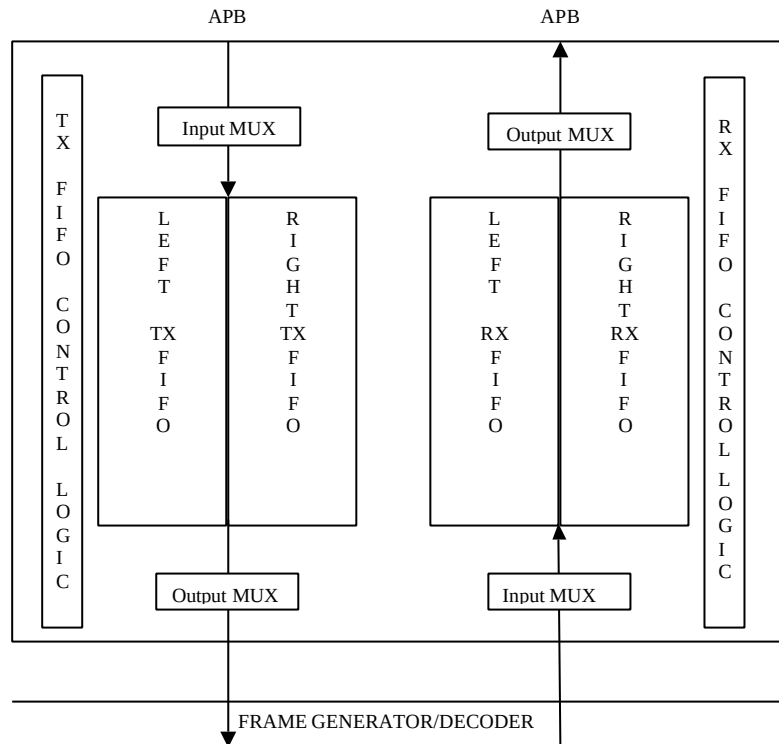


Figure 4.3-7 Channel Description

Transmit FIFO

The Transmit FIFO is two 20 bit wide, 4 deep buffers organised as a left buffer and a right buffer. Reads and writes to the FIFO result in accesses to both buffer locations alternatively, addressed by a single read pointer and a write pointer respectively. The pointer values are compared to deduce the FIFO fill level. The Transmit FIFO drives the signals TxFF, TxHE and TxFE when the FIFO is full, half empty and empty respectively.

Writes to the Transmit FIFO occur through the APB interface in the PCLK domain when the AACIDR registers are written into. In Compact mode a write results in the lower 20 bits getting written into the right buffer, and the upper 12 bits in the left buffer and the write pointer is incremented by 2. In Non-Compact mode, writes are made alternatively into right and left buffers and the write pointer is incremented by one. When the read pointer increment signal to the Tx FIFO from the Tx control logic is asserted, the read pointer is incremented by one.

The Transmit FIFO always drives the read data buses of the right and left buffers with the data from the location in the corresponding buffer pointed to by the read pointer with the LSB masked. For example, when the read pointer points to "000" or "001" data in Right buffer location "00" and Left buffer location "00" are driven. The Tx FIFO also generates the TxBusy and TxFValid signals. The TxBusy signal is set if the Tx FIFO contains data or if the data from this channel is being transmitted. The TxFValid signal indicates whether the FIFO contains enough data to be transmitted during the next frame.

When the AACI is disabled through deassertion of the AacIfE bit in the AACIMAINCR register, the transmit FIFOs are flushed. When TxFEn is deasserted, the transmit FIFO becomes a 1-deep holding register and the TxFF, TxFE and TxHE flags indicate the full/empty status of the holding register.

The Transmit FIFOs 2, 3 and 4 are similar to FIFO 1, but without the DMA signals AACIDMACLRTX and AACIDMABREQTX since the DMA requests are generated only in Channel 1.

When the transmit FIFO 1 has 4 words or less, DMA signal AACIDMABREQTX is asserted. This signal is cleared when AACIDMACLRTX is asserted.

Receive FIFO

The receive FIFO is similar to the Transmit FIFO in most respects. The receive FIFO consists of two 20-bit wide, 4-deep buffers organised as a right buffer and a left buffer. Writes to the receive FIFO occur from the Frame Decoder section. Reads from the receive FIFO occur through the APB interface when the AACIDR registers are read.

On receiving a slot of data (Slots 1–12), the Frame decoder section of the AACI drives the received data on the RxDataIn[19:0] bus. When the RdPtrIncSync (Receive FIFO read pointer increment) signal is asserted, RxDataIn[19:0] is clocked into the location of the FIFO as pointed to by the current value of the write pointer. After every write, the write pointer is incremented by one.

The write data for the right and left buffer of the FIFOs, are generated from RxDataIn[19:0] depending on the value of RSize and bit 0 of the write pointer. Bit 0 of the WritePtr signal determines the buffer in which data from the RxDataIn[19:0] bus is stored. Data is stored in the right buffer when WritePtr[0] is 0 and in the left buffer when WritePtr[0] is 1. Depending on the value of RSize, appropriate number of MSB bits of RxDataIn[19:0] are LSB justified and the remaining bits of the write databuses to the right and left buffers of the FIFO are zeroed. For example, if the RSize is “11” indicating a 12-bit data, RxDataIn[19:8] is written into the least significant 12 bits of the write databus to the right / left buffer. The remaining bits of the write databus are zeroed.

The Receive FIFO always drives the right and left buffers' read databuses with the contents of the FIFO location pointed to by the current value of the read pointer with the lower bit masked. For example, when the read pointer points to “000” or “001”, data in Right buffer location “00” and Left buffer location “00” are driven. These two data are multiplexed to form the Rx FIFO's read databus to the APB interface. The control logic of this multiplexer comes from Compact Mode as programmed in the AACIRXCRx ($x = 1$ to 4) register and bit 0 of the read pointer of the particular FIFO. In Non-compact mode, when bit 0 of the read pointer is 0, the data from the right buffer is driven onto the FIFO's read databus and when bit 0 of the read pointer is 1, the data from the left buffer is driven onto it. In compact mode, the LSB 16 bits of the left buffer read databus are driven on the MSB 16 bits of the FIFO's read databus and the LSB 16 bits of the right buffer read databus are driven on the LSB 16 bits of the read databus of the FIFO, thereby returning 32 bits of data.

The receive FIFO controller asserts the RxOverrun signal when the FIFO is already full and an additional data is received by the same FIFO, thereby resulting in an overrun of the FIFO. In this case, the data that caused the overrun is discarded. The Overrun condition is cleared when the RxOECIrcCo signal is asserted. The RxFE (Receive FIFO Empty) signal is asserted when there are no valid entries in the Receive FIFO. RxFF and RxHF signals are asserted when the Receive FIFO is full and half full respectively.

When the AACI is disabled through deassertion of the AacIfE bit in the AACIMAINCR register, the receive FIFOs are flushed. When RxFEN is deasserted, the receive FIFO becomes a 1-deep holding register and the RxFF, RxFE and RxHF flags indicate the full/empty status of the holding register.

Receive FIFOs 2, 3 and 4 are similar to Receive FIFO 1, but without the DMA signals AACIDMACLRX, AACIDMASREQRX, AACIDMABREQRX, AACIDMALSREQRX and AACIDMALBREQRX since DMA request signals are generated only in Channel 1. Apart from these signals, Receive FIFOs 2, 3 and 4 differ from the Receive FIFO 1 in the RxTimeout and RxTOFE generation logic.

Each receive FIFO has a timeout counter which is loaded with the value in the TOC field of the AACIRXCRx ($x = 1$ to 4) register. This counter decrements by one at the end of a frame if no data was received by the receive FIFO during that frame. When the counter decrements to zero, the RxTimeout or RxTOFE signal are asserted depending on whether the FIFO was empty or non-empty respectively. Once asserted, RxTimeout is cleared when the data in the FIFO is read out through the APB interface. The RxTOFE signal is cleared when a '1' is written into the corresponding RxTOFEC bit in the AACIINTCLR register.

In channels 2, 3 and 4 only one of the 2, RxTimeout and RxTOFE are asserted at a time. Both, RxTimeout and RxTOFE are not reasserted once cleared until the counter counts down to 0 again. Here, it is assumed that the channel re-allocation / changing of Link-list is done in both, the RxTimeout ISR and the RxTOFE ISR. If channel 1 is not DMA enabled, it behaves exactly like the other channels. However, when the DMAEnable bit of the AACIMAINCR register is asserted and DMALSREQRX or DMALBREQRX is not asserted, RxTOFE is asserted

after RxTimeout is de-asserted due to the FIFO becoming empty. This is under the assumption that in the DMA channel case, the channel reallocation or Link-list change is done when the DMA Controller reads the last word from the FIFO. However, if the DMALSREQRX or DMALBREQRX are not asserted, the RxTOFE interrupt will have to be raised for channel reallocation.

The DMASREQRX signal is asserted if the receive FIFO contains 1, 2 or 3 words when timeout has not occurred. If timeout has occurred, DMASREQRX is asserted when the FIFO contains 2 or 3 words. The DMALSREQRX signal is asserted if timeout has occurred and the FIFO contains only 1 word. DMABREQRX is asserted if the receive FIFO contains 4 words or more when timeout has not occurred. If timeout has occurred, DMABREQRX is asserted if the FIFO contains more than 4 words. The DMALBREQRX signal is asserted if timeout has occurred and the FIFO contains only 4 words.

DMALSREQRX is not asserted if any other DMA receive request has been asserted. Similarly, if DMALBREQRX is also mutually exclusive with all the other DMA receive requests. However, DMASREQRX and DMABREQRX are not mutually exclusive.

All the DMA receive requests are cleared when AACIDMACLRX is asserted. If a high on AACIDMACLRX is not detected for a given DMA request, it is possible that multiple DMA requests are asserted on a given clock. When the AACIDMACLRX signal remains asserted, all DMA requests remain cleared. When the AACIDMACLRX signal is de-asserted, the FIFO fill level conditions are re-evaluated and a DMA request is raised, if required.

4.3.7 Transmit Control Logic

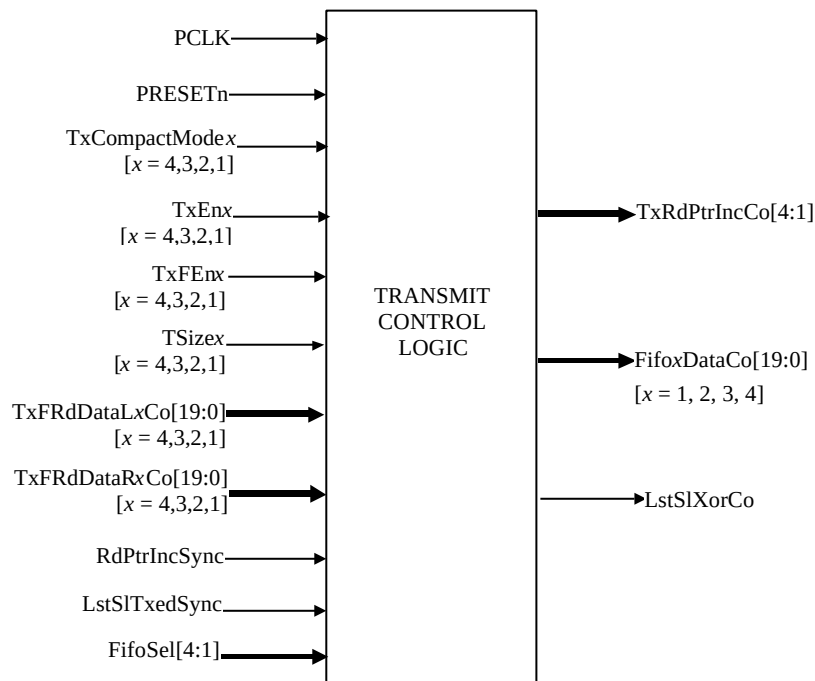


Figure 4.3-8 Transmit Control Logic

Transmit Control Logic multiplexes data from the left buffer and the right buffer of the transmit FIFO of each channel to generate the read data bus of the Tx FIFO, the multiplexer being controlled by TxCompactMode, TSize and bit 0 of the read pointer. In Compact Mode, when the LSB of the read pointer is 0, valid bits in the right buffer, depending on TSize, are shifted left to make the data MSB justified, and the remaining bits are stuffed with zeros to form the 20-bit FIFO read data. When the LSB of the read pointer is 1, valid bits in the left buffer, remaining valid bits in the right buffer and the appropriate number of zeros are concatenated to form the 20-bit FIFO read data. In non-Compact mode, bits from only the right buffer or the left buffer are selected depending on the LSB of

the read pointer, MSB justified, and stuffed with zeros to form the 20-bit read data. In the Frame Generator, the data from one of the four transmit FIFOs is selected to be the data source for the next slot. The Transmit control logic also generates the read pointer increment signals to the Tx FIFOs depending on the FifoSel[4:1] signal from the Slot 0 generator. **Figure 4.3-8** gives the block diagram for Transmit control logic.

4.3.8 Receive Control Logic

Receive Control logic selects the FIFO to write the incoming data, RxDataIn, based on the slot valid bits and the receive enable bits of the AACIRXCRx (x = 1 to 4) registers and the receive slot valid bits from the frame decoder, and appropriately sets the corresponding FIFO's write pointer increment to 1. **Figure 4.3-9** gives the block diagram for Receive Control Logic.

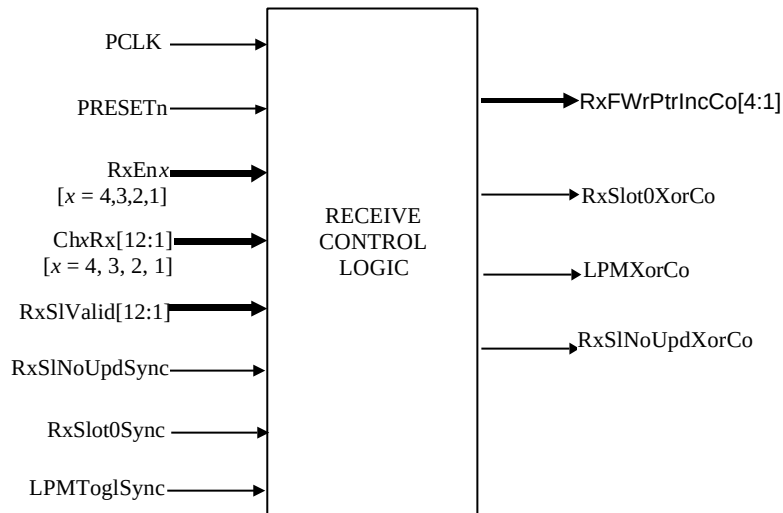


Figure 4.3-9 Receive Control Logic

4.3.9 Interrupt Generation Logic

All the maskable active high interrupts and the combined single interrupt AACIINTR are generated in this module. The channel interrupts AACIRXINTR1-4, AACITXINTR1-4, AACIORINTR1-4, AACIURINTR1-4, AACIRXTOINTR1-4, AACIRXTOFEINTR1-4 and the AACITXCINTR1-4 are generated by qualifying FIFO flags with the interrupt enable bits from AACIIE1-4 registers. The AACIS1TXINTR, AACIS2TXINTR and AACIS12TXINTR are generated by qualifying the SI1TxEmpty, SI2TxEmpty and SI12TxEmpty flags with their interrupt enable bits. The AACIS1RXINTR, AACIS2RXINTR and AACIS12RXINTR are generated by qualifying the SI1RxValid, SI2RxValid and SI12RxValid flags with their interrupt enable bits from the AACISLIEN register. The Wakeup interrupt is generated combinationally when the LowPowerMode bit of the AACIMAINCR is set and a high is detected on the AACISDATAIN line when the AACI is enabled. The GPIO interrupt is asserted when bit 0 of slot 12 in the incoming frame is set. These interrupts are also qualified by their enable bits from the AACISLIEN register. The combined AACI interrupt is asserted when any of the individual interrupts mentioned above are asserted and the corresponding mask is enabled.

4.3.10 Synchronisers

All synchronisers for signals crossing clock domains have been grouped into two sub-blocks – one for signals crossing over from the PCLK domain into the AACIBITCLK domain (**Figure 4.3-10**) and the other for signals crossing over from the AACIBITCLK domain into the PCLK domain (**Figure 4.3-11**).

The signals crossing clock domains are double synchronised with flip-flops operating on the rising edge of the clock into which the signals get synchronised.

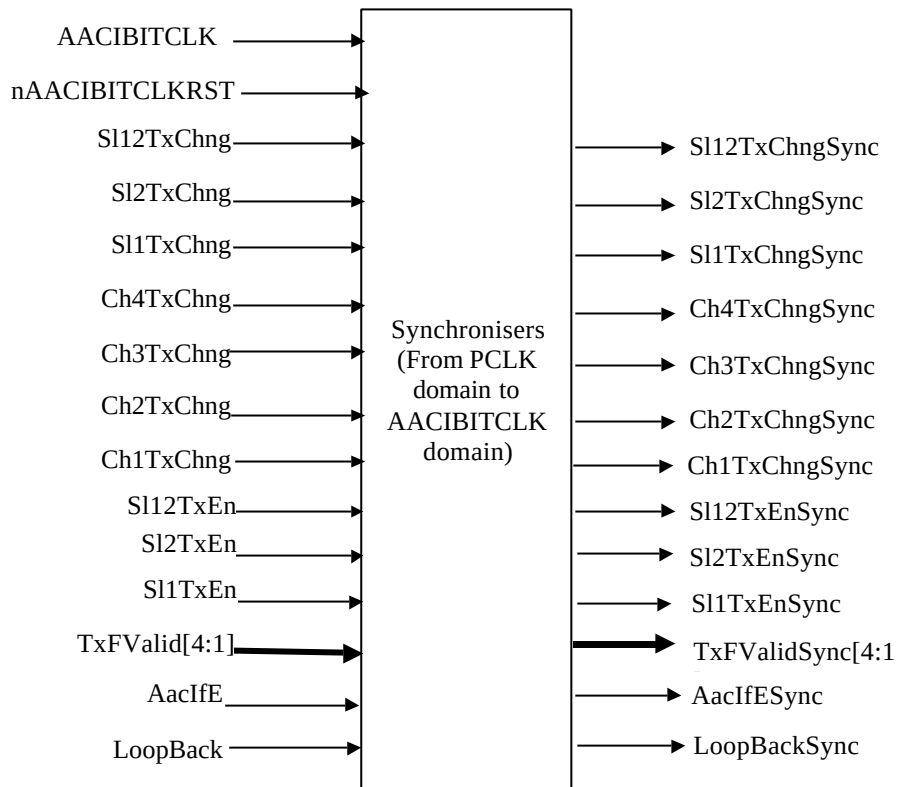


Figure 4.3-10 Synchronisers for signals crossing into AACIBITCLK domain

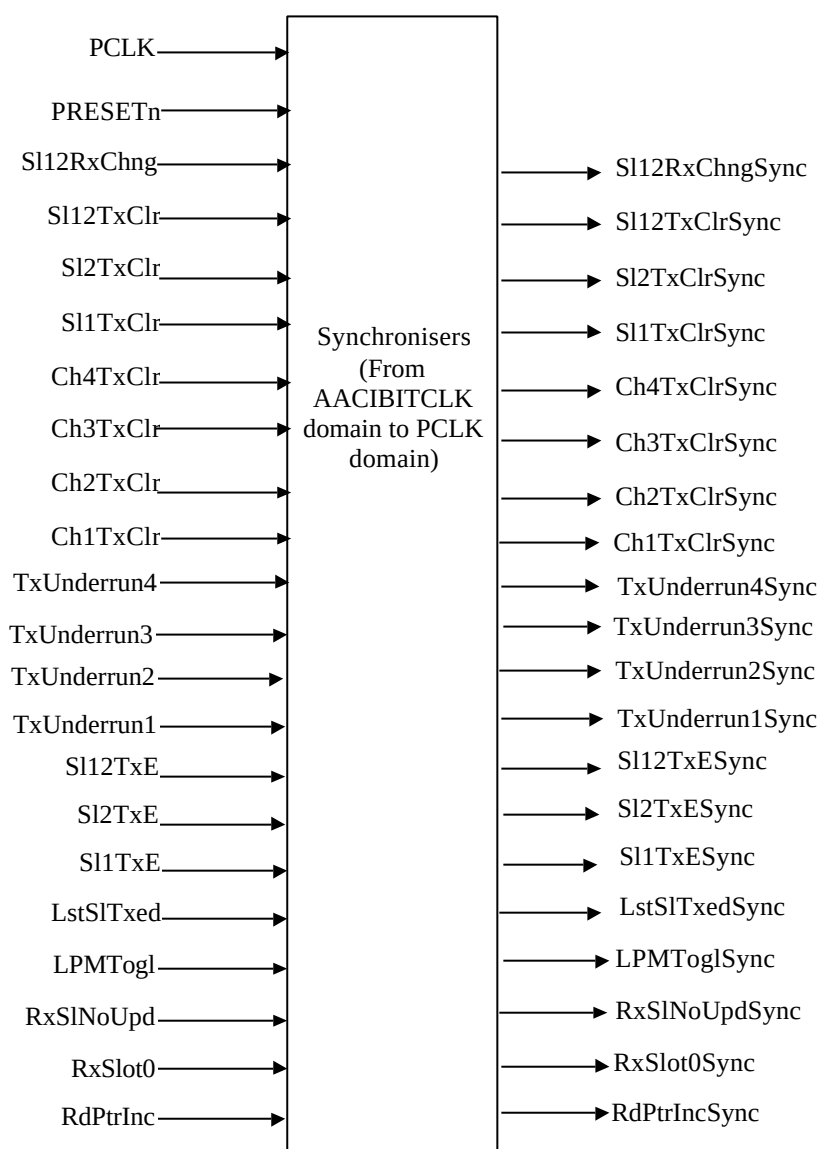


Figure 4.3-11 Synchronisers for signals crossing into PCLK domain

5 AACI IMPLEMENTATION DETAILS

5.1 Design Hierarchy

The hierarchy in the AACI is shown in **Figure 5.1-1**. A short description about the functionality of each of the sub-modules is also given within brackets.

Aaci (Top level)

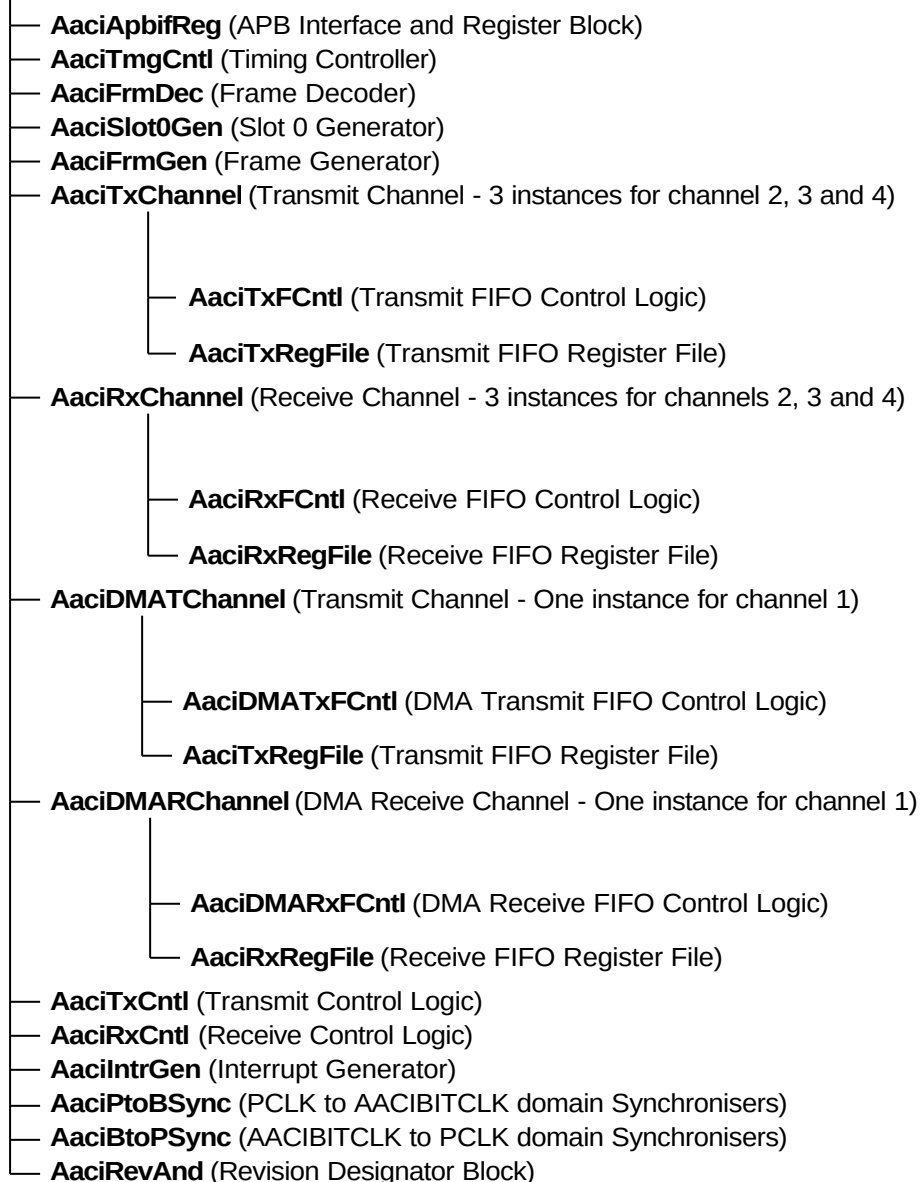


Figure 5.1-1 AACI Design Hierarchy

5.2 AACI Top Level Module (Aaci)

The top-level module is a structural block which instantiates and interconnects the main functional blocks in the AACI.

5.3 APB Interface and Register Block (AaciApbifReg)

The APB interface and Register block consists of the input module, the output module and the AACI registers. The APB interface deals with reads and writes to the AACI registers from the APB. Power saving is done by gating the address bus, PADDR[11:2] by PSEL, and the write databus, PWDATA[31:0] by PSEL and PWRITE to generate PAddrGated[11:2] and PWDatInt[31:0] respectively.

5.3.1 Input Module

This section of the APB write interface generates decodes for writes to all the writable registers in the device. A common write enable signal is generated sequentially on the rising edge of PCLK when PSEL is high and PENABLE is low and PWRITE is high. The write enables for the AACI registers are generated from a combinatorial address decode and the common write enable signal. Each register is clocked by PCLK and is written to when the appropriate write enable is high. The clocked versions of the write enables of the AACIRXCR1-4 registers are used in the Rx FIFOs to reload the timeout counter. The write enables of the AACIDR1-4 registers are clocked from the D-input of the common write enable. This helps to ease timings in the FIFO write paths. In the normal mode of operation of the FIFOs, data is written into the Tx FIFOs on writes to the AACIDR1-4 registers. When FIFOTEST[1:0] bits from the AACITCR register indicate a test mode for the FIFOs, data is written into the Rx FIFO or the Tx FIFO depending on the value on the FIFOTEST[1:0] bits. The block diagram in **Figure 5.3-1** illustrates the structure of the input module. In the normal mode of operation of the FIFOs, writes and reads to the AACIDR register actually result in writes to the transmit FIFO and reads from the receive FIFO respectively.

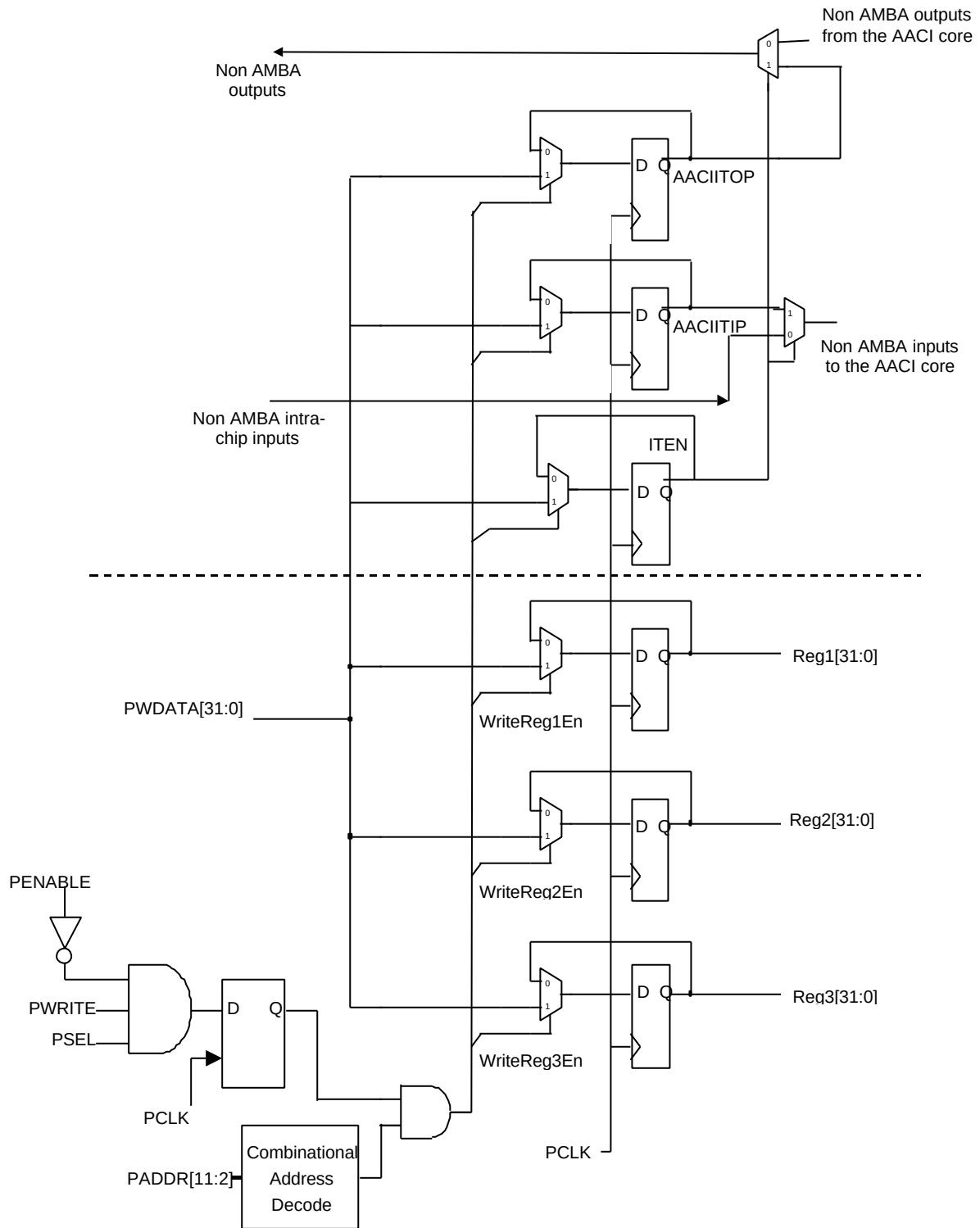


Figure 5.3-1 Input Module

5.3.2 Output module

The output module in the APB interface consists of a multiplexer for selecting which of the various sources for read data is driven onto the read databus. The inputs of the multiplexer are fed from a bank of flip-flops clocked by the system PCLK. The registered output of the multiplexer is driven on the read databus. A combinatorial address decode is used to select which of the output sources are driven onto the bus. The read enables of the AACIDR registers are registered to ease timings in the FIFO read paths. If the FIFOs are in normal mode, data from the Rx FIFOs are returned on reads to the AACIDR registers. However, if the FIFOTEST[1:0] bits of the AACITCR register indicate a test mode for the FIFOs and if FIFOTEST[0] is set, data from the Tx FIFOs is returned on reads to the AACIDR registers. All the non-AMBA inputs and outputs are also made observable through APB read accesses to Integration test registers. The output multiplexer is left such that, by default, zeros are driven on the data bus. This ensures that there are no restrictions placed on the type of external bus connection method used for the data bus on the APB. The external data buses of the peripherals on the APB may then be connected to the APB bridge using the Muxed or the ORed bus connection method. A block diagram of the output module is shown in **Figure 5.3-2**.

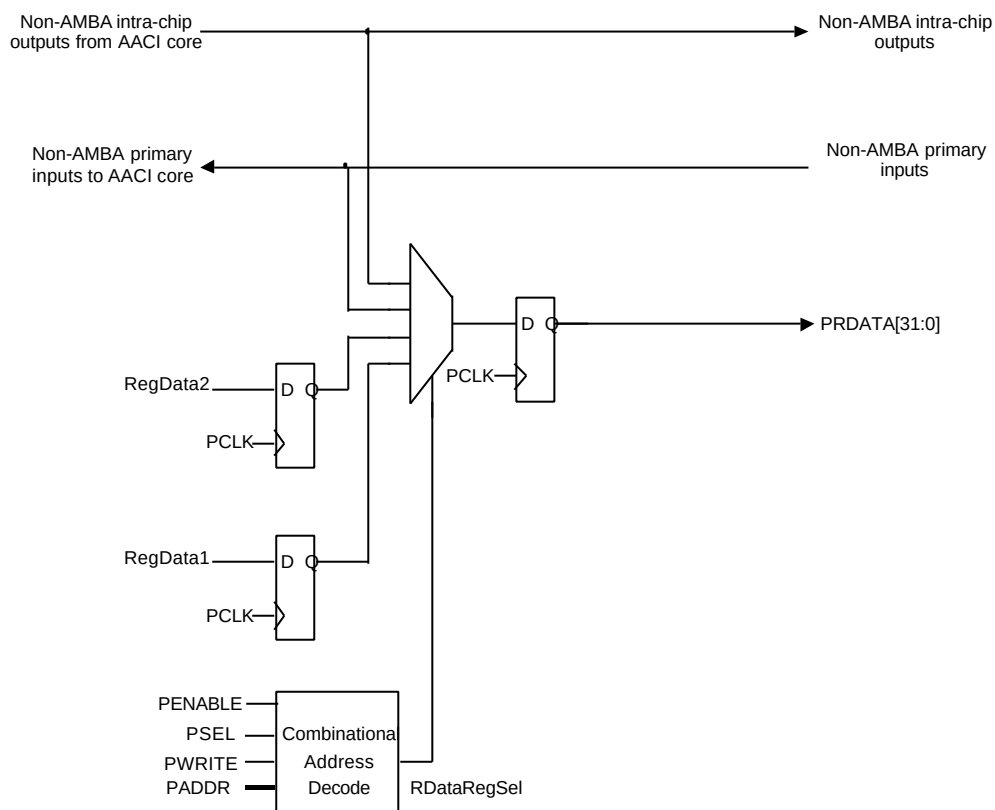


Figure 5.3-2 Output module

5.3.3 AACI Registers

The Interrupt status registers are located in the Interrupt generation block. All other registers of the AACI are located in the APB interface and Register block. In the integration test mode, the non-AMBA outputs of the AACI can be controlled by writing to the AACIITOP0 and AACIITOP1 registers when the Integration test enable (ITEN) bit of the AACITCR register is set. The AACIRESET output of the AACI is driven by the ForcedRESET bit of the AACIRESET register.

This module also generates the clear signals for the Tx Underrun errors, the Rx Overrun errors, the RxTOFEs (Rx Timeout FIFO Empty) error and the Wakeup interrupt from writes of '1' to the bits of the AACIINTCLR register. The clear signals for the Tx Underrun errors are asserted for one PCLK duration when a '1' is written into the corresponding bits of the AACIINTCLR register, or if the a '0' is written into the AacIfE bit of the AACIMAINCR register. These signals are used in the Interrupt generator block to clear the Tx Underrun errors. All the other interrupt clear signals are also generated as one PCLK wide pulses when there is a write of '1' to the corresponding bits in the AACIINTCLR register. These pulses are used to clear the Rx Overrun errors, RxTOFE errors and the Wakeup interrupt which are generated in the PCLK domain.

Writes to the Slot Rx registers are done from the AACIBITCLK domain. Data received on AACISDATAIN in slots 1, 2 and 12 in the AACIBITCLK domain are written into the Slot 1, 2 and 12 Rx registers respectively in the PCLK domain. The RxSINoUpd signal from the Frame Decoder block in the AACIBITCLK domain is toggled at the end of every valid slot reception and one bit corresponding to the slot number is set in the RxSIVValid[12:1] vector.

RxSINoUpd is double synchronised to the PCLK domain. If bit 1 of RxSIVValid[12:1] is set when the synchronised RxSINoUpd signal toggles, data is written into the Slot 1 Rx register if it is enabled. Similarly data is written into the Slots 2 and 12 Rx registers when the bits 2 and 12 respectively are set when the RxSINoUpd signal toggles, and the corresponding register enable is set. The SI1RxValid, SI2RxValid and SI12RxValid flags are asserted when data is written into the Slot 1, 2 and 12 Rx registers respectively. These flags are cleared when the data in these registers are read through the APB. The SI1RxBusy, SI2RxBusy and SI12RxBusy signals are asserted if the slot valid bits of slots 1, 2 and 12 respectively are set in the received slot 0. They are cleared when the data from the slot receive registers are read through the APB.

SI1TxEmpty is cleared when data is written into the AACISL1TX register. It is set when the double synchronised version of the SI1TxEmpty signal from the frame generator toggles indicating that data from the AACISL1TX register has been loaded into the transmit shift register. The SI2TxEmpty and SI12TxEmpty flags are also generated similarly to indicate the empty/full status of the AACISL2TX and AACISL12TX registers. When new data is written into the AACISL1TX, AACISL2TX and AACISL12TX registers signals SI1TxChng, SI2TxChng and SI12TxChng are toggled. These signals are double synchronised to the AACIBITCLK domain to indicate availability of valid data in the slot transmit registers. When new data is written into the AACISL1TX, AACISL2TX and AACISL12TX registers, the SI1TxBusy, SI2TxBusy and SI12TxBusy flags are asserted. These flags are cleared when the double synchronised versions of SI1TxClr, SI2TxClr and SI12TxClr signals are toggled indicating that the data in the slot transmit registers have been transmitted. In the normal mode of operation of the slot transmit registers, there is a restriction on software that new data should not be written into them till the previous data has been loaded into the transmit shift register. To enable testing the slot registers, in register test mode (REGTEST bit of AACITCR is set), software is allowed to write into the slot transmit registers before the previous data gets loaded into the transmit shift register.

The AACI uses two different techniques for the synchronisation of registers which cross between PCLK and AACIBITCLK domains, 2-buffer synchronisation and double synchronisation.

Classical double-synchronisation has not been used to synchronise the AACITXCR1-4 register bits to the AACIBITCLK domain. This is because for the registers which contain multiple-bit fields like AACITXCR1-4, if double synchronisation is used, then some bits in these fields could get synchronised one AACIBITCLK later than other bits. Thus, there could be a one-AACIBITCLK window during which the bit fields are incorrect.

In this 2-buffer technique, writes from the APB to the AACITXCR1-4 registers, go to the first stage buffers. When there is a write to AACITXCR1, a control signal, Ch1TxChng toggles. This signal is double synchronised to the AACIBITCLK domain. In the AACIBITCLK domain, an edge on the synchronised signal is detected by generating a delayed version and XOR-ing the synchronised signal with its delayed version. Thus, a one-AACIBITCLK-wide load pulse is generated with every write to the AACITXCR1 register. When this load signal is asserted, the contents of the first stage Ch1Tx[12:1] (slot valid bits of channel 1) and TxEn1 buffer are copied into the second stage buffers in the AACIBITCLK domain. The next write into the AACITXCR1 register in the PCLK side is allowed only after the first data in this register is safely copied into the second stage buffer. Metastability on the second stage is hence ruled out. This hardware protection implementation requires a feed back signal from the AACIBITCLK domain to again allow writes to the AACITXCR1 register after the synchronisation is complete. The slot valid and transmit enable bits of the AACITXCR2, AACITXCR3 and AACITXCR4 registers are also

synchronised into the AACIBITCLK domain by the 2-buffer technique. **Figure 5.3-3** illustrates the 2-buffer mechanism.

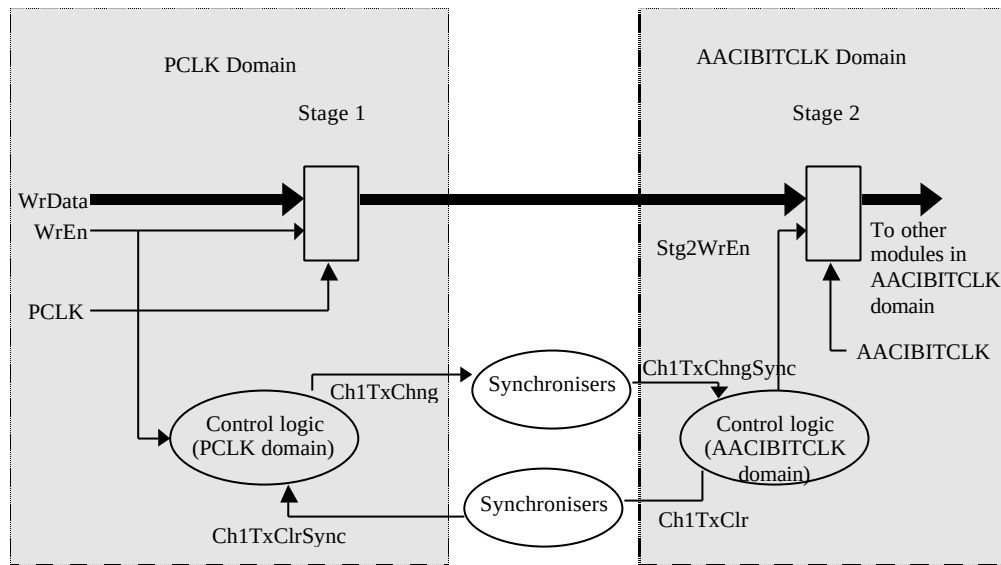


Figure 5.3-3 Two-buffer synchronisation mechanism

The SCRA[1:0] bits of the AACIMAINCR register are not synchronised to the AACIBITCLK domain even though they are used in the Frame Generator section, which operates on AACIBITCLK. This is because of a software restriction that the SCRA[1:0] bits should be changed only when there is no data for slots 1 and 2 in the channels and the slot registers. The SCRA[1:0] bits are used for generation of the transmit slot 0 in the AACIBITCLK domain only when there is a valid CODEC register read/write access (slot 1 is valid). Since the software restriction implies that slot 1 will not be valid when the SCRA[1:0] bits are changed, synchronisation of the SCRA[1:0] bits is not required.

All other bits of the AACIMAINCR register that are used in the AACIBITCLK domain are synchronised to the AACIBITCLK domain by the double-synchronisation technique. 2-buffer synchronisation is not required in this case since the functionality of the AACI is not affected even if some of them get synchronised one AACIBITCLK before the others.

The hardware masking of writes to the AACITXCR1-4 register is disabled during the register test mode (REGTEST bit of AACITCR register is set) to enable effective testing of the registers.

5.4 Timing Controller (AaciTmgCntl)

The Timing Controller Block consists of an AACIBITCLK counter, which is used for the generation of SyncBITCLK signal and AACISYNC signal generation logic. The AACIBITCLK counter is also used by the Frame Generator and Frame Decoder sections of the AACI to transmit / receive data corresponding to the slots based on the counts. A block diagram of the Timing Controller is shown in **Figure 5.4-1**.

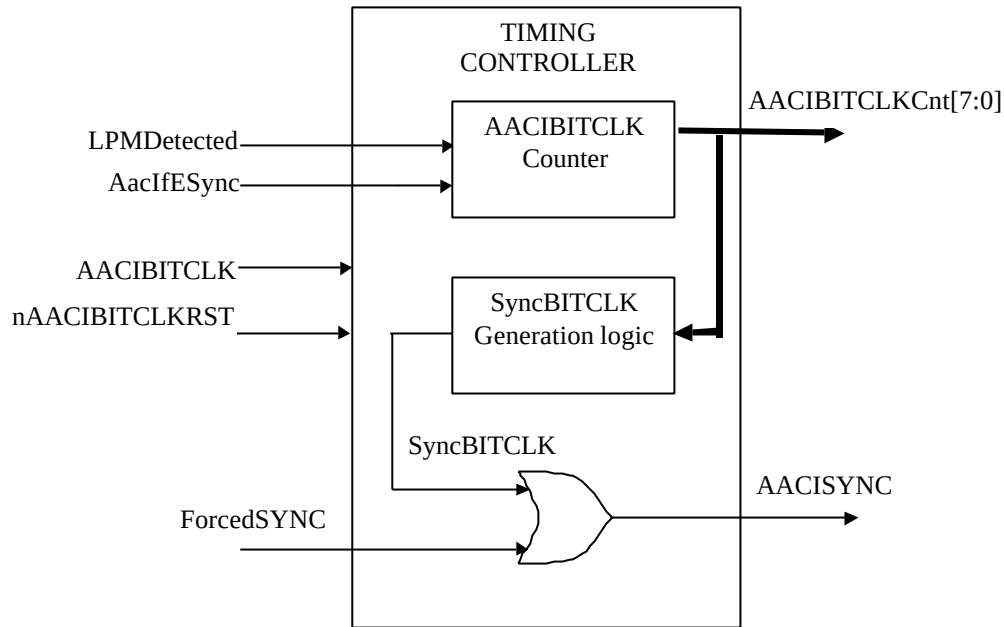


Figure 5.4-1 Timing Controller

5.4.1 AACIBITCLK Count Generation

AACIBITCLKCnt[7:0] is a count of AACIBITCLK when the AACI is enabled and AC-Link is active. AACIBITCLKCnt[7:0] is incremented by 1 on the rising edge of AACIBITCLK when AacIfE (AACI enable) is set to 1 and the LPMDetected (the hardware detected low power mode signal generated by the frame generator block) signal is set to 0. AACIBITCLKCnt[7:0] is initialised to 255 on reset. The counter wraps over to 0 from 255 at the next rising edge of AACIBITCLK. When the LPMDetected signal is set to 1, the AACIBITCLKCnt[7:0] is assigned a value of 243. The AACIBITCLK dies down within a maximum of 1 us after the end of slot 2 when the AC-Link goes into Low Power mode. Since the AACIBITCLK frequency is 12.288MHz, there can be a maximum of 12 rising edges of AACIBITCLK after the end of slot 2 transmission. Loading the AACIBITCLK counter with a value 243 ensures that the AACIBITCLK counter reaches a maximum count of 255 before the clock dies down and the AACISYNC pulse is generated as soon as possible once the AC-Link comes out of Low Power Mode.

5.4.2 AACISYNC Signal Generation

The AACISYNC signal generated from the AACIBITCLK counter, SyncBITCLK signal is set high for a duration of 16 AACIBITCLKs and held low for a duration of 240-AACIBITCLKs. SyncBITCLK is set high on the rising edge of AACIBITCLK when AACIBITCLKCnt[7:0] is 255. Since 255 is the reset value of the counter, SyncBITCLK is set high at the first rising edge of the incoming AACIBITCLK. It is made zero at the rising edge of AACIBITCLK when the AACIBITCLKCnt[7:0] is 15 and held low till AACIBITCLKCnt[7:0] again reaches 255.

If the SyncBITCLK signal is set or the ForcedSYNC bit of the AACISYNC register is set, the AACISYNC output of the AACI is set. The ForcedSYNC bit is used by the software to restart the AC-Link by a warm reset when it is in Low Power mode. Software is expected to set the ForcedSYNC bit for a minimum of 1.3 us time when a warm reset needs to be issued to restart the AC-Link.

5.5 Frame Decoder (AaciFrmDec)

This module decodes the data for slots 0 to 12 from the incoming data on the AACISDATAIN pin. It also implements the Frame State Machine, which is used for Frame Generation / Decode.

5.5.1 Frame State Machine

The Frame State Machine has two states – ST_CODECIDLE and ST_CODECREADY. When nFAACIBITCLKRST is asserted, the Frame State Machine is initialised as ST_CODECIDLE. When AACIBITCLK count is equal to 1, if AACISDATAIN is set high or if the LoopBack bit of the AACIMAINCR register is set, the Frame State Machine goes to ST_CODECREADY. If the AC-Link is in Low Power Mode (LPMDetected is 1) or when AACISDATAIN is zero on AACIBITCLK count of 1, then the Frame State Machine goes to ST_CODECIDLE. Otherwise the Frame State Machine remains in the ST_CODECREADY state. Frame Generation and Decoding happen only when the state machine is in the ST_CODECREADY state. **Figure 5.5-1** shows the Frame State Machine.

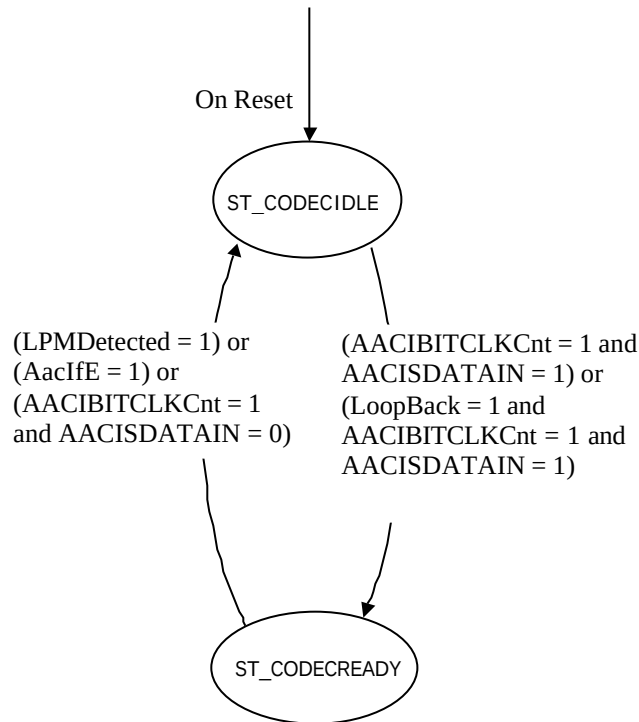


Figure 5.5-1 Frame State Machine

5.5.2 Frame Decoder

The Frame decoder does serial to parallel conversion of AACISDATAIN to form data of various slots to be stored in the AACI receive FIFOs or slot receive registers. **Figure 5.5-2** shows a block diagram of the Frame Decoder.

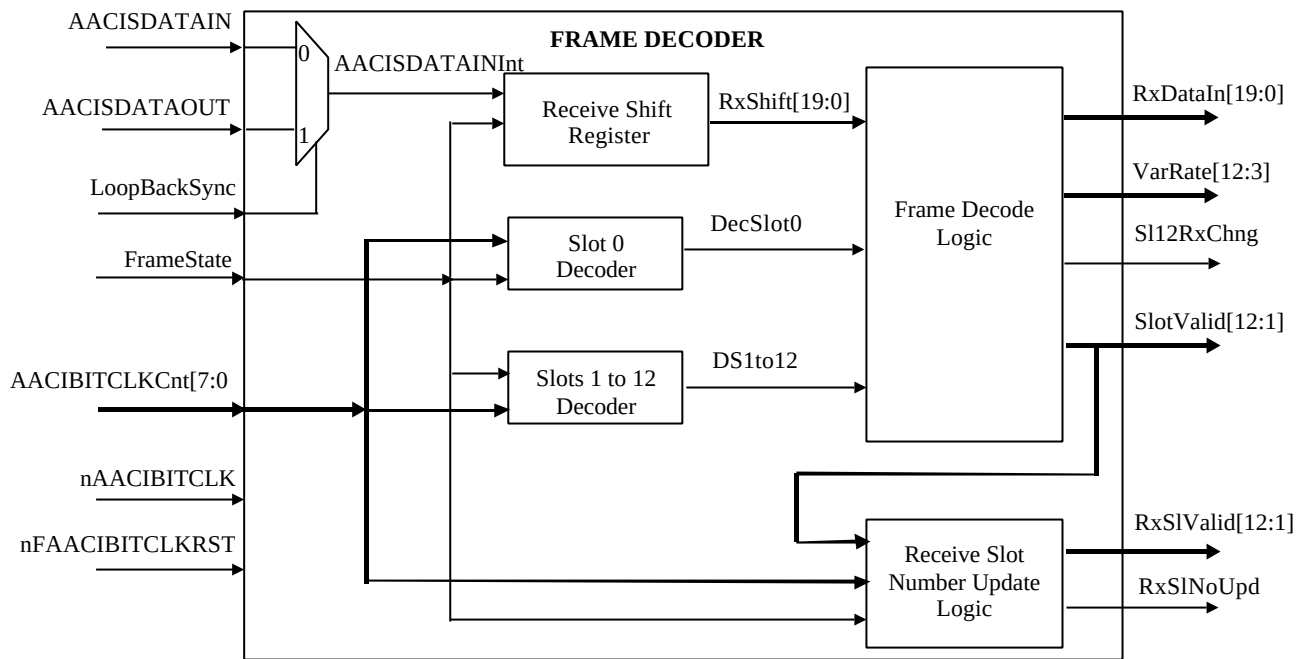


Figure 5.5-2 Frame Decoder

LoopBack Mode Multiplexer

A multiplexer is used to select the AACISDATAIN input of the Frame Decoder module. When the LoopBack bit of the AACIMAINCR is set to 1, the AACISDATAOUT output from the Frame Generator module is connected to the AACISDATAINInt signal, which is used by the Frame Decoder. Otherwise the AACISDATAIN input from the AC-Link is connected to the AACISDATAINInt signal used by the Frame Decoder. For Integration Test purposes, the AACISDATAIN input from the AC-Link is also connected to the APB Interface and Register block to make it readable through bit 2 of the AACIITIP register. Refer to **Section 5.24** for more details on the Integration Test Strategy.

Receive Shift Register

When the Frame State Machine is in ST_CODECREADY state, on every rising edge of nAACIBITCLK, RxShift[19:0] is shifted left by one bit and data received on AACISDATAINInt is stored in the LSB (bit 0) of RxShift[19:0]. When the FrameState is ST_CODECIDLE or when nFAACIBITCLKRST is asserted, all bits of RxShift[19:0] are zeroed.

Slot 0 Decoder

When FrameState is ST_CODECREADY and the AACIBITCLK Counter indicates that slot 0 data has been completely shifted in i.e., when the counter is equal to 16, DecSlot0 signal is set to 1. Otherwise DecSlot0 signal is zeroed. Thus DecSlot0 is asserted for one clock in ST_CODECREADY state, when the slot 0 data has been completely shifted into the receive shift register. When the FrameState is ST_CODECIDLE, or when nFAACIBITCLKRST is asserted, DecSlot0 is zeroed. DecSlot0 determines when the slot 0 data needs to be clocked in from the receive shift register. When DecSlot0 signal is asserted, bits 14 to 3 in the receive shift register are assigned to SlotValid[12:1]. Bits 12 to 1 of this register indicate validity of slots from 1 to 12 in the current frame being received. The SlotValid[12:1] signal is zeroed when nFAACIBITCLKRST is asserted.

Slots 1 to 12 Decoder

When the Frame State Machine is in ST_CODECREADY state and the AACIBITCLK counter indicates values for end of slots 1 to 12 then DS1to12 signal is set to 1. The values of AACIBITCLKCnt for Slots 1 to 12 respectively are 36, 56, 76, 96, 116, 136, 156, 176, 196, 216, 236 and 0. At all other counts of AACIBITCLKCnt, DS1to12

signal is zeroed. It is also zeroed when the Frame State Machine is in ST_CODECIDLE State, or when nFAACIBITCLKRST is asserted. When DS1to12 is 1 indicating that data for one of slots 1 to 12 has been completely shifted in, the receive shift register data is loaded into the RxDataIn[19:0] register. On assertion of nFAACIBITCLKRST, RxDataIn[19:0] is zeroed. RxDataIn[19:0] is used to load data into the relevant receive FIFOs and slot receive registers.

At the count of 37 where slot 1 data is stored in RxDataIn[19:0], bits 11 to 2 of the receive shift register are assigned to the VarRate[12:3] register. Bits 12 to 3 of this register indicate whether data needs to be sent in the next frame for slots 3 to 12. For example, if VarRate[12] is asserted, data for slot 3 should not be sent in the next frame. On assertion of nFAACIBITCLKRST, VarRate[12:3] is made zero.

Receive Slot Number Update Logic

When FrameState is ST_CODECREADY, when AACIBITCLK counter indicates a count of 1 more than the count at which DS1to12 was asserted for a particular slot, and the SlotValid[12:1] register data indicates that the corresponding slot is valid, one bit in the RxSIVValid[12:1] vector corresponding to that slot is set to 1. At the count for slot 0 i.e. 17, RxSIVValid[12] is set to zero. For slots 2 to 12, RxSIVValid[slot number -1] is set to zero and RxSIVValid[slot number] is asserted if that slot is valid in the current frame. Thus when a valid slot data is stored in RxDataIn, RxSIVValid[slot number] signal is asserted for one slot width. RxSIVValid[12:1] is used by the receive FIFO, and the slot registers in the PCLK domain to detect the availability of data for a slot. Another signal, RxSINoUpd is toggled at the same counts at which RxSIVValid[12:1] is generated indicating that the slot number has to be updated. RxSINoUpd is toggled even if the SlotValid[12:1] vector indicates that the slot is not valid. This signal is double synchronised to the PCLK domain to be used by the Rx FIFOs and the slot receive registers to detect slot boundaries. When nFAACIBITCLKRST is asserted or when the FrameState is ST_CODECIDLE both RxSIVValid[12:1] and RxSINoUpd are zeroed.

When a rising edge is detected on RxSIVValid[12] and bit 0 of RxDataIn is 0 indicating that bit 0 of slot 12 in the current frame is zero, SI12RxChng is toggled. The double synchronised version of this signal is used in the PCLK domain to generate the AACIGPIOINTR interrupt in the Interrupt generation block.

5.6 Slot 0 Generator (AaciSlot0Gen)

This module implements the slot 0 generation logic, the slot transmit register valid information and the transmit underrun generation logic for all the channels.

The AACIBITCLK domain buffers for storing the 2-buffer synchronised versions of the slot valid and transmit enable bits of the AACITXCR1-4 registers are also implemented in this module. For further details on the 2-buffer synchronisation technique, refer to **Figure 5.3-3** in **Section 5.3.3**.

The 2-buffer synchronised versions of the slot valid bits from the AACITXCR1-4 registers, ChxTxSync[12:1] ($x = 1, 2, 3, 4$), are loaded into the corresponding ChxTxSIVal[12:1] ($x = 1, 2, 3, 4$) registers when TxFValidSync[x] ($x = 1, 2, 3, 4$) and TxEnxSync ($x = 1, 2, 3, 4$) are asserted. TxFValidSync[4:1] is the double-synchronised version of the signals generated by the Tx FIFO control logic of the 4 channels to indicate that the Tx FIFO has enough data to transmit in the next frame. TxEnxSync ($x = 1, 2, 3, 4$) is the 2-buffer synchronised version of the transmit enable bit from the AACITXCRx ($x = 1, 2, 3, 4$) register. When TxFValidSync[x] ($x = 1, 2, 3, 4$) or TxEnxSync ($x = 1, 2, 3, 4$) are not asserted, ChxTxSIVal[12:1] ($x = 1, 2, 3, 4$) is zeroed. The bits of the D-inputs of the ChxTxSIVal[12:1] ($x = 1, 2, 3, 4$) registers corresponding to the 4 channels are OR-ed together to form the ChTxSIVal[12:1] vector. The bits of this vector contain information about the slots that have been programmed to be transmitted from the channels.

Toggle signals called SI1TxChng, SI2TxChng and SI12TxChng are used to indicate valid writes into the AACISL1TX, AACISL2TX and AACISL12TX registers respectively. On detection of an edge on the double synchronised version, SIxTxChngSync ($x = 1, 2, 12$), in the AACIBITCLK domain, the corresponding SIxDataValid signal is asserted. The slot transmit register enable bits (SI1TxEn, SI2TxEn and SI12TxEn) from the AACIMAINCR register are double synchronised to the AACIBITCLK domain. At the start of a frame transmission, if SI1DataValid is asserted, SI1TxEnSync is asserted, and the slot valid bit for slot 1 is not asserted in the ChTxSIVal[12:1] vector, a signal SI1RegValid is asserted to indicate that the source of data for slot 1 in the current frame is the

AACISL1TX register. Similarly, the SI2RegValid and SI12RegValid signals are asserted at the start of transmission of a frame to indicate the data source for slots 2 and 12 in the current frame. At the AACIBITCLK count at which the data for slots 1, 2 or 12 get loaded from the corresponding slot transmit register into the transmit shift register, the SI1TxEmpty, SI2TxEmpty or SI12TxEmpty signals are toggled to indicate that the corresponding slot transmit register is empty. These signals are double synchronised to the PCLK domain and used to generate the SIxTxEmpty flags. Once set, the SIxRegValid ($x = 1, 2, 12$) signal is deasserted when the data for the corresponding slot gets completely shifted out of the transmit shift register.

Data to be transmitted on slot 0 in a frame is combinationally generated by setting the slot valid bits of the slots which are available for transmission at the start of the frame and setting the frame valid bit if at least one slot is marked valid. The slot valid bit of slot 1 (Slot0DataCo[14]) is set when the D-input of the SI1RegValid register is asserted or ChTxSIVal[1] is asserted. Similarly, the slot valid bit of slot 12 (Slot0DataCo[3]) is set when the D-input of SI12RegValid is asserted or ChTxSIVal[12] is asserted, provided VarRate[3] is not asserted. The validity of slot 2 in a frame depends on the validity of slot 1. Hence the valid bit of slot 2 (Slot0DataCo[13]) is asserted when the D-input of the SI2RegValid register or ChTxSIVal[2] is asserted, provided Slot0DataCo[14] is asserted. If Slot0DataCo[14] is not asserted, indicating that slot 1 is not valid, the valid bit of slot 2 is not asserted even if data is available for slot 2. For each of the other slots 3-11, the slot valid bit is asserted if the corresponding bit in the ChTxSIVal[12:1] vector is asserted, and the VarRate bit corresponding to that slot is not asserted. The frame valid bit, Slot0DataCo[15] is formed by OR-ing all the other bits (bits 14 to 3) of Slot0DataCo[15:3] together.

At the start of transmission of each frame, the transmit underrun flag of a channel is toggled when the channel is transmit enabled if the slot valid bit corresponding to a slot in the ChxTxSync[12:1] ($x = 1, 2, 3, 4$) of that channel is set and the VarRate bit corresponding to that slot is not set, but the slot valid bit of that slot in Slot0DataCo[15:3] is not set indicating that the channel does not contain enough data to be transmitted in the frame. However, if the AacIfE bit is cleared at the start of transmission of a frame, the transmit underrun flags are not toggled.

The Slot 0 Generator module also implements the logic for 4 12-bit slot shift registers corresponding to the 4 channels. The slot shift registers are loaded with the slot valid information at the start of transmission of each frame in the ST_CODECREADY state, and their contents are shifted right (from bit 12 to bit 1) at every slot boundary. The load values for the 4 shift registers are obtained by AND-ing the bits of the ChxTxSIVal[12:1] ($x = 1, 2, 3, 4$) register with Slot0DataCo[3:14]. At any slot boundary, if the LSB (bit 1) of the slot shift register of a channel is set, it indicates that the data source for the next slot is that channel. At all slot boundaries where the LSB of any of the 4 slot shift registers is set, a signal called RdPtrInc is toggled. The LSBs of the 4 slot shift registers are stored in the FifoSel[4:1] register at all slot boundaries. The double synchronised version of RdPtrInc and FifoSel[4:1] are used in the PCLK domain to generate the transmit FIFO read pointer increment signals to the 4 channels. The LSBs of the slot shift registers of channels 1, 2 and 3 are brought out of the slot 0 generator module as a vector output FDataSel[3:1] to be used by the Frame Generator module to select data from one of the 4 channels for transmission in a slot. A signal SShiftEnd is asserted for one slot duration when the contents of any of the slot shift registers changes from 1 to 0. At a slot boundary, if this SShiftEnd signal is asserted, it means that the last slot to be transmitted from one of the channels has been transmitted completely. The LstSITxed signal is toggled at this point and the value of FifoSel[4:1] at this slot boundary is assigned to LstSIFifoSel[4:1] to indicate the channel whose last data has been transmitted. These 2 signals are used to deassert the TxBusy signals of the 4 channels.

If the data to be transmitted in slot 1 indicates a CODEC register read access, the valid bit of slot 2 has to be cleared and zeros have to be transmitted on slot 2. However, the data in slot 2 has to be removed from the slot transmit registers or channels. In order to decide whether the slot valid bit of slot 2 has to be cleared before the start of transmission of a frame, the data for slot 1 has to be made available to the Frame Generator module before the start of slot 0 transmission. The slot 0 generator module generates a vector called SI1DataSel[3:0]. This vector is formed by concatenating the LSBs of the Ch3TxSIVal[12:1], Ch2TxSIVal[12:1], Ch1TxSIVal[12:1] registers and the D-input of the SI1RegValid register. The SI1DataSel[3:1] vector is used to select the data for slot 1 from the 4 channels and the slot 1 transmit register.

5.7 Frame Generator (AaciFrmGen)

This module drives AACISDATAOUT to the Audio CODEC, selecting the data from the slot transmit registers (AACISL1TX[19:0], AACISL2TX[19:0] and AACISL12TX[19:0]) and channel Tx FIFO read data inputs (FifoDataCo[19:0], $x = 1, 2, 3, 4$). **Figure 5.7-1** shows the block diagram of the Frame Generator.

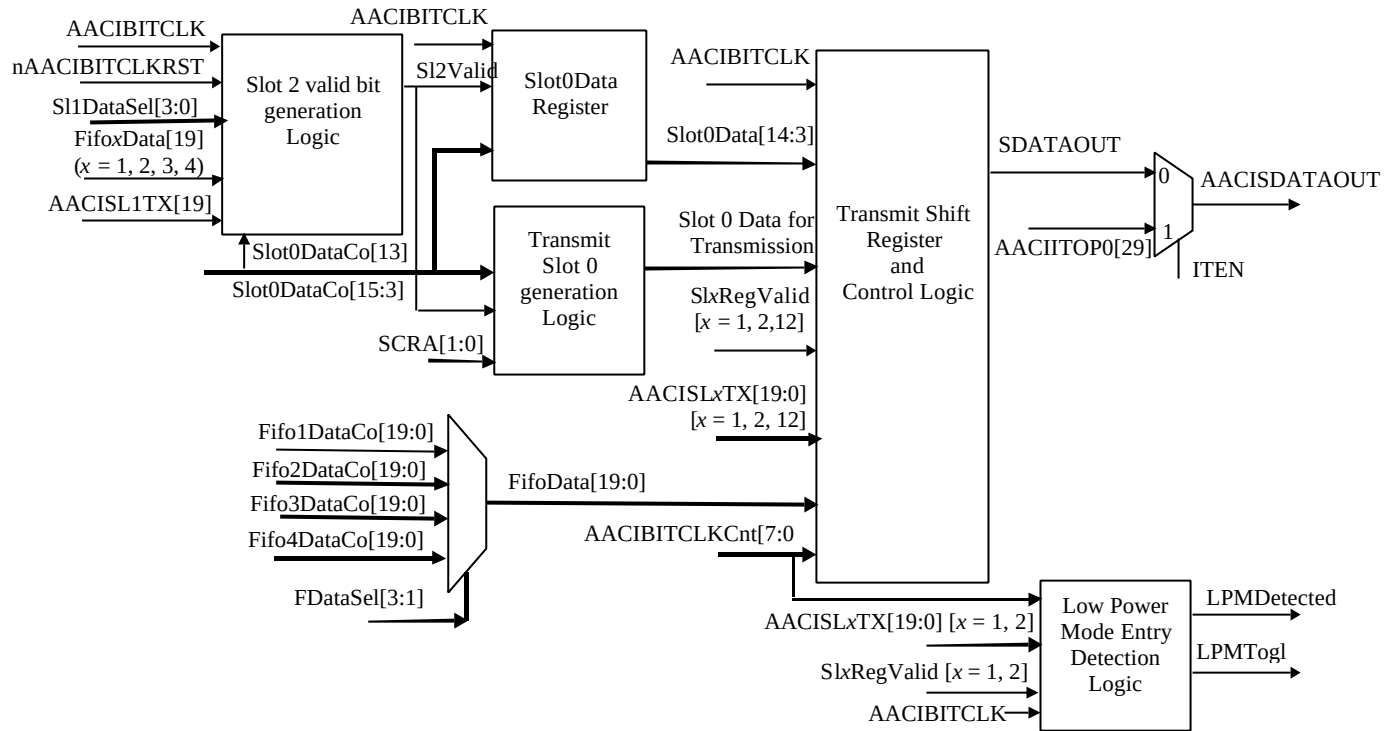


Figure 5.7-1 Frame Generator

5.7.1 Slot 2 Valid bit generation

Bit 19 of the Tx FIFO read data from one of the channels or AACISL1TX register data is selected based on the value of SI1DataSel[3:0] to generate the RegRead signal. If SI1DataSel[0] is asserted, AACISL1TX[19] is assigned to RegRead. If SI1DataSel[1], SI1DataSel[2] or SI1DataSel[3] is asserted, Fifo1DataCo[19], Fifo2DataCo[19] or Fifo3DataCo[19] is assigned to RegRead. If none of the bits of SI1DataSel[3:0] are asserted, Fifo4DataCo[19] is assigned to RegRead. Thus the RegRead signal is asserted only if the slot 1 data indicates a CODEC register read access. When Slot0DataCo[14] indicates a valid slot 1 and the RegRead signal is cleared indicating that it is a CODEC register block write access, the SI2Valid signal follows the slot valid bit of slot 2 as generated by the slot 0 generator block (Slot0DataCo[13]).

5.7.2 Slot 0 Register

The validity of data in slots 1 to 12 is stored in the Slot0Data[14:3] register at the start of every frame transmission when the AACI is enabled and the FrameState is ST_CODECREADY. Bits 14 and 12-3 are generated from the corresponding bits of the combinational Slot0Data (Slot0DataCo[15:3]). The SI2Valid signal is driven on bit 13. If the AACI is disabled at the start of a frame or if the frame state machine is in ST_CODECIDLE state, the Slot0Data[14:3] register is cleared. The bits of the Slot0Data[14:3] register are used to qualify the loading of data from the relevant transmit FIFO or slot transmit register into the transmit shift register at the start of transmission of each slot.

5.7.3 FIFO Data Mux

FDataSel[3:1] signal from the Slot 0 generator is used to select data from one of the channels for transmission and drive the selected data on FifoData[19:0]. If FDataSel[3] is asserted, data from channel 3 (Fifo3DataCo[19:0]) is selected for transmission. Similarly if FDataSel[2] or FDataSel[1] is asserted, data from the corresponding channel is selected for transmission. If none of the bits of FDataSel[3:1] are asserted, data from channel 4 (Fifo4DataCo[19:0]) is driven on FifoData[19:0].

5.7.4 Transmit shift register

When Frame State Machine is in ST_CODECREADY state, data to be transmitted is selected as indicated by the slot count deduced from the AACIBITCLKCnt[7:0]. The AACIBITCLK counts for transmission of slot 0 to slot 12 respectively are 0, 16, 36, 56, 76, 96, 116, 136, 156, 176, 196, 216 and 236. At AACIBITCLK counts corresponding to the start of each slot, data to be transmitted on AACISDATAOUT is loaded into the transmit shift register if the frame state machine is in ST_CODECREADY state.

Data to be transmitted on slot 0 is generated depending on the value of the SCRA[1:0] bits in the AACIMAINCR register. If SCRA[1:0] is 00, it indicates a Primary CODEC register access. If SCRA[1:0] is 01, 10 or 11, it indicates a Secondary CODEC register access. For Primary CODEC register accesses, or if the slot valid bit of slot 1 is not set, Slot0DataCo[15:14] are to be transmitted on bits 15 and 14 of slot 0 and Slot0DataCo[12:3] are to be transmitted on bits 12 to 3 of slot 0. SI2Valid is transmitted on bit 13 of slot 0. Bits 3 to 0 are zeroed. For Secondary CODEC accesses where slot 1 is valid, the slot valid bits of slots 1 and 2 (bits 14 and 13 of slot 0) are cleared, Slot0DataCo[15] is transmitted on bit 15 of slot 0, Slot0DataCo[12:3] are transmitted on bits 12 to 3 of slot 0, and the SCRA[1:0] bits are transmitted on the least significant 2 bits of slot 0. The 16 bits to be transmitted on slot 0 are loaded into bits 19 to 4 of the transmit shift register, while the lower 4 bits of the transmit shift register are zeroed.

For all other slots, the Slot0Data[14:3] register bits are used to determine whether data has to be sent on AACISDATAOUT for a particular slot in a frame. If any of the channels have been programmed to transmit data on a particular slot, data for that slot is transmitted from that channel. For slots 1, 2 and 12, data is sent from the respective slot transmit registers only if any channel has not been programmed to transmit data for that slot, and the slot transmit register has been written into and enabled. Signals SI1RegValid, SI2RegValid and SI12RegValid from the Slot 0 Generator block are used to determine whether the data for the slots 1, 2 and 12 in the current frame have to be transmitted from the slot transmit registers or the channels. For all other counts where there is no slot transition, the MSB (bit 19) is driven on the SDATAOUT signal and the transmit shift register is shifted left (from bit 0 to 19) by 1. When the Frame State Machine is in ST_CODECIDLE state, the transmit shift register is zeroed.

5.7.5 Integration Test Mux

In normal mode, the value on the SDATAOUT signal is driven on the AACI output port AACISDATAOUT. When the Integration test enable (ITEN) bit of the AACITCR register from the APB Interface and Register block is set, the ITOPSDATAOUT signal value is driven on the AACISDATAOUT output port of the AACI. ITOPSDATAOUT is driven by bit 29 of the AACIITOP0 register. Refer to **Section 5.24** for more details on the Integration Test Strategy.

5.7.6 Low Power Mode Detection

The CODEC is programmed into low power mode only through the slot transmit registers. If the slot 1 and slot 2 data for the current frame are programmed to be transmitted from the slot transmit registers and if the data from the slot 1 transmit register indicates an access to the Powerdown register of the CODEC (address 26h) and bit 16 of the slot 2 transmit register is set, the LPMDetected signal is asserted and the LPMTogl signal is toggled at the end of slot 2 transmission to indicate the entry of the AC-Link into Low Power Mode. The LPMDetected signal is used in the AACIBITCLK domain to detect the entry of the AC-Link into Low Power Mode and the LPMTogl signal is double synchronised and used in the PCLK domain to detect Low Power Mode entry.

5.8 Transmit Channel Logic (AaciTxChannel)

This module is instantiated three times in the Aaci Top Level Module (Aaci) to implement the Transmit Channel logic for the channels 2, 3 and 4 that are not DMA capable. **Figure 5.8-1** shows a block diagram of a Transmit Channel.

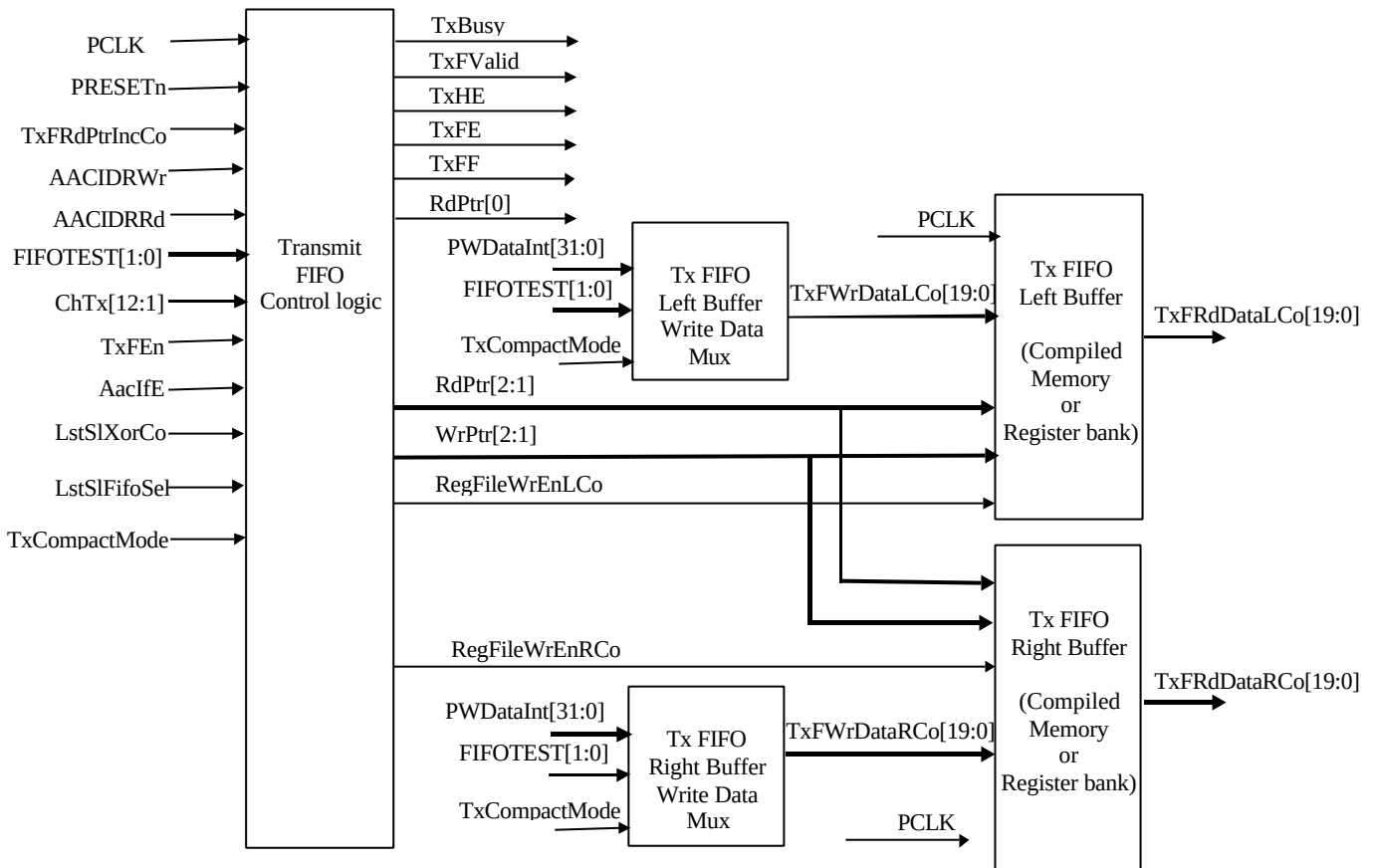


Figure 5.8-1 Transmit Channel Logic

The Transmit Channel Logic block instantiates the Transmit FIFO control logic (AaciTxFCntl) and the data storage elements (AaciTxRegFile). The AaciTxRegFile is instantiated twice to form the right and left buffers of the Tx FIFO. In the normal mode of operation of the FIFOs (FIFOTEST = 00), writes to the transmit FIFO occur from the APB through the APB interface, which operates on PCLK. Reads from the Transmit FIFO occur from the Frame Generator section, which operates on AACIBITCLK.

The Transmit FIFO has two 20-bit wide and 4 deep buffers organised as a left buffer and a right buffer. Each FIFO acts as a 20x8 FIFO in non-Compact Mode, and a 40x4 (32x4 effectively) FIFO in Compact Mode. The advantage of such an arrangement over a simple 20x8 FIFO is that all the unpacking and resizing logic will be transferred to the non APB side of the FIFOs to facilitate meeting of timing requirements on the APB side. Reads and writes to the FIFO are tracked with a Read pointer and a Write pointer respectively. The APB interface in the AACI drives the AACIDRWrr input to the transmit FIFO with the decode of writes to the AACIDR register. The AACIDRWrr input pulse is one PCLK wide. When the AACIDRWrr input is asserted in normal mode (FIFOTEST[1:0] = 00), the write enable of the left or right buffer (RegFileWrEnLCo or RegFileWrEnRCO) is asserted by the Transmit FIFO Control logic, and the write data from the APB (PWDDataInt[31:0]) is clocked into the location in the FIFO pointed to by the current value of the write pointer as described below. In Compact mode a write results in lower 20 bits getting

written into the right buffer, and the upper 12 bits in the left buffer and the write pointer is incremented by 2. In Non-Compact mode, writes are made alternatively into right and left buffers and the write pointer is incremented by one. For example, a write to location 0 results in a write to the right buffer and a write to location 1 results in a write to the left buffer and location 2 results in a write to the right buffer and so on. In the FIFO test mode where FIFOTEST[1:0] = 11, a data 0x55555 is written into the Tx FIFO on reads to the AACIDR register. In all other test modes of the Tx FIFO, the write data buses of the right and left buffers (TxFWrDataRCo[19:0] and TxFWrDataLCo[19:0]) are driven by the PWDatInt[31:0] bus as in the normal mode. The TxFRdDataLCo[19:0] and TxFRdDataRCo[19:0] outputs are always driven with the contents of the FIFO location pointed to by the current value of the MSB two bits of the read pointer (RdPtr[2:1]). The least significant bit of the read pointer (RdPtr[0]) is used by the Transmit Control Logic section to select data from the appropriate buffer and drive the FIFO read databus. The write pointer and read pointer are controlled by the AciTxFCntI block.

The TxFValid output signal, which is double synchronised to the AACIBITCLK domain, to indicate to the Slot 0 Generator section that valid data is available in the FIFO for transmission, is also generated by the AciTxFCntI block. The Transmit Control logic section drives the TxFRdPtrIncCo input signal. This signal is asserted in the PCLK domain when the Frame generator section has transmitted a slot from this channel. The TxFRdPtrIncCo input is used to increment the read pointer of the Tx FIFO by 1 both in Compact mode as well as non-Compact mode.

The TxFF signal indicates the FIFO contains data for 8 slots. The assertion of the TxHE signal indicates that there are 4 or fewer entries in the transmit FIFO. The TxFE (Transmit FIFO Empty) output signal is asserted when there are no valid data elements in the transmit FIFO. The TxBusy signal is asserted if the Tx FIFO contains data for transmission or if data from the Transmit FIFO is being transmitted. All these signals are generated by the AciTxFCntI block and are readable through the AACISR register corresponding to each channel.

The Tx FIFO is flushed when the AACI is disabled by clearing the AacIfE bit in the AACIMAINCR register. When the Tx FIFO enable (TxFEen) bit in the AACITXCR register is deasserted, the transmit FIFO becomes a 1-deep holding register and the TxFF, TxHE and TxFE flags indicate the full/empty status of the holding register.

5.9 Transmit FIFO Control Logic (AciTxFCntI)

The Transmit FIFO control logic controls the read and write pointers for the FIFO, besides controlling accesses to the FIFO register file. It also generates the FIFO fill level status signals and the Transmit FIFO Busy signal, TxBusy. The TxFValid signal generated by this module is an indication of the availability of data from the Tx FIFO for transmission in the next frame.

The WrPtrIncValid signal is asserted when a valid write access to the FIFO occurs. The following considerations apply when generating this signal:

- When there is a write access to the Transmit FIFO, the write data is clocked into the FIFO only if the FIFO is not already full. If the FIFO is full when the write access occurs, a simultaneous read should also occur for the write data to be clocked in. In normal operation, the Interrupt service routine that writes data into the FIFO is not expected to write more data than is required to fill the FIFO. This check is done to protect from such writes in case they occur.
- This signal is asserted when the AACIDRWrr signal is asserted if FIFOTEST[1:0] is 10 or 00. When FIFOTEST[1:0] is 11, WrPtrIncValid is asserted when the AACIDRRd signal is asserted to allow a write of 0x55555 into the transmit FIFO.

The WrPtr[2:0] signal is used as the write pointer signal. The Write pointer is incremented every time the WrPtrIncValid signal is asserted. The write pointer is incremented by one when the Transmit FIFO is in non-Compact mode and in Compact mode, it is incremented by 2 on the rising edge of PCLK if the WrPtrIncValid signal is asserted.

The write enable signals to the right and left buffers (RegFileWrEnRCo and RegFileWrEnLCo) are generated combinationaly from the WrPtrIncValid signals. In Non-Compact mode, writes to the left buffer are enabled when WrPtr(0) is 1 and writes to right buffer are enabled when WrPtr(0) is 0. This mechanism allows writes to the right and left buffers of the Tx FIFO alternately. In Compact mode, writes to both the buffers are enabled when WrPtr(0)

is 0. In Compact mode, the write pointer is incremented by 2 per assertion of WrPtrIncValid and data is written into both the buffers at the same time. Hence both the write enables to both the buffers are asserted when WrPtr(0) is 0.

The RdPtrIncValid signal is asserted on a valid read to the FIFO when it is not already empty. In normal mode (FIFOTEST = 00), the RdPtrIncValid signal is asserted when the TxFRdPtrIncCo input from the Transmit Control Logic section is asserted after a data from the Tx FIFO has been loaded into the transmit shift register in the Frame Generator. In test mode, the RdPtrIncValid signal is asserted when the AACIDRRd signal from the APB interface is asserted when FIFOTEST[1:0] is 01 or 11. The read pointer (RdPtr[2:0]) is incremented by 1 per assertion of the RdPtrIncValid signal.

The write pointer and read pointer are cleared when the AACI is disabled by clearing the AacIfE bit of the AACIMAINCR register.

There exists a possibility that the write pointer increments through “111” to “000”. The direct difference between the pointers will not give the correct fill level. To take this into account, the ‘Wrap’ signal has been introduced.

The Wrap signal is set when the Write pointer has wrapped around but the read pointer hasn’t. The signal is reset when the read pointer also subsequently wraps around. The Wrap signal is prepended to the write pointer and from the resultant value, the read pointer is subtracted to determine the fill level of the FIFO. The TxFFillLevel[3:0] signal indicates the number of locations in the FIFO that contain valid data. This signal takes a range of values starting with zero, when the FIFO is empty, one when there has been one write to the transmit FIFO and upto eight when the FIFO has all its eight locations filled. A count of the number of slots that have been programmed to be transmitted from the Tx FIFO (number of 1s in ChTx[12:1]) is compared with the TxFFillLevel[3:0] and the TxFValid output signal is asserted if the count is less than or equal to the Tx FIFO fill-level. The TxFValid signal is double synchronised to the AACIBITCLK domain and used in the Slot 0 Generator module to detect the availability of enough data from the Tx FIFO for transmission in the next frame.

The following considerations apply when generating the Wrap signal:

- When the write pointer is at “111” in non-Compact mode (“011” in Compact mode) and the WrPtrIncValid signal is asserted and the RdPtrIncValid signal is not asserted then set the Wrap.
- When the read pointer is at “111” and the RdPtrIncValid signal is asserted, and the WrPtrIncValid signal is not asserted, clear Wrap. To combine this condition with the previous condition, toggle Wrap when either of the two conditions occurs.

When both the pointers are at “111” with the FIFO being full (Wrap already set) and both the WrPtrIncValid signal and the RdPtrIncValid signal are asserted, then the write is allowed to complete and the FIFO is still full, but retain the value of the Wrap bit i.e. let it remain set, thus indicating that the FIFO is still full. When the AACI is disabled, initialise the Wrap signal to 0.

The FIFO status flags are generated from the value of TxFFillLevel[3:0].

The TxFF (Transmit FIFO Full) signal indicates the status of the Transmit FIFO. When the FIFO is enabled

- In non-compact mode, when the FIFO is one less than full and there is another write to the FIFO without a simultaneous read, the TxFF signal is set. When the FIFO is already full and there is a read from the FIFO, without a simultaneous write, the TxFF signal is cleared.
- In Compact mode, the FIFO has to be filled with even multiples of slot data (2, 4, 6 or 8 slots). Hence, the FIFO can be flagged as non-Full only after at least 2 entries [NOT 1 entry] are free.

When the FIFO is disabled, TxFF is set when there is a write to the FIFO without a simultaneous read, and it is cleared when there is a read without a simultaneous write. Compact mode is not supported when the FIFO is disabled.

When the Tx FIFO contains one word and there is a read without a simultaneous write, the Transmit FIFO is said to be empty and the TxFE flag is asserted. When the Tx FIFO contains no data and there is a write to it, the FIFO is said to be non-empty and the TxFE flag is cleared.

When the Tx FIFO is enabled, it is said to become half-empty when it contains 5 words and there is a valid read without a simultaneous valid write. The TxHE flag is set at this point. The TxHE flag is cleared when the Tx FIFO contains 4 words and there is a valid write without a simultaneous valid read. When the Tx FIFO is disabled, it is said to become half empty when there is a valid read without a simultaneous valid write. The TxHE flag is cleared when there is a valid write without a simultaneous valid read.

The TxFF flag is cleared when AacIfE is deasserted. The TxFE and TxHE flags are set when AacIfE is deasserted.

The Tx FIFO is said to be busy when it contains at least one valid data or the data from the Tx FIFO is being transmitted by the Frame Generator section. The TxBusy signal is asserted when the Tx FIFO is non-empty and cleared when the LstSIXorCo signal from the Transmit control Logic and the LstSIFifoSel from the Slot 0 generator logic are asserted indicating that the last slot that is programmed to be transmitted from the Tx FIFO has been completely shifted out of the transmit shift register.

5.10 Transmit FIFO Register File (AaciTxRegFile)

The Register File is implemented as a bank of D-types. This block can be replaced with a compiled memory if required. The Register File also contains a 2-to-4 decoder to decode the Write pointer and a 4-to-1 bus multiplexer for the read data bus. Write data is clocked into the location pointed to by the WrPtr[2:1] input, on the rising edge of PCLK on which the Write Enable input (RegFileWrEn) is sampled high. The TxFRdDataCo[19:0] bus is driven with the contents of the FIFO location pointed to by the current value of the read pointer (RdPtr[2:1]).

5.11 Receive Channel Logic (AaciRxChannel)

This module is instantiated three times in the Aaci Top Level Module (Aaci) to implement the Receive Channel Logic for the channels 2, 3 and 4 that are DMA capable. **Figure 5.11-1** shows the block diagram of a Receive Channel.

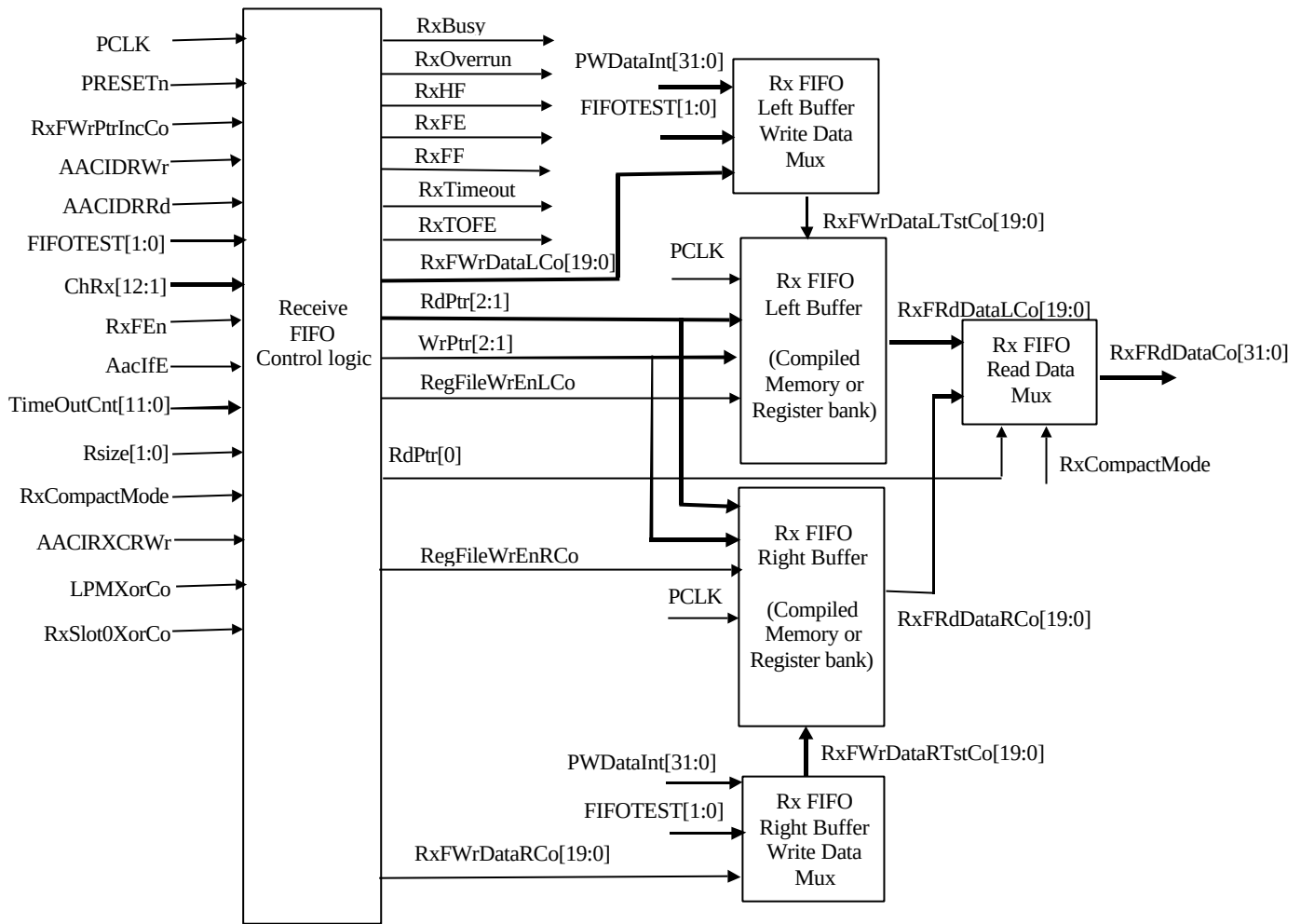


Figure 5.11-1 Receive Channel Logic

The Receive Channel Logic block instantiates the Receive FIFO control logic (AaciRxFcCntl) and the data storage elements (AaciRxRegFile). The AaciRxRegFile is instantiated twice to form the right and left buffers of the Rx FIFO. This module also implements the Rx FIFO write data test mode mux and the Rx FIFO read data mux. When the Rx FIFO is in normal mode (FIFOTEST = 00), writes to the Receive FIFO occur from the Frame Decoder section, which operates on AACIBITCLK and reads from the Receive FIFO occur through the APB interface operating on PCLK.

The Receive FIFO is similar to the transmit FIFO. It is implemented as a circular buffer with read and write pointers.

When a slot of data has been received, the Frame Decoder section drives the received data on the RxDataIn[19:0] bus. It then toggles the RxSInoUpd signal, which gets synchronised to PCLK and is seen as the RxSInoUpdSync input signal. When an edge is detected on the RxSInoUpdSync input, the Receive Control Logic section asserts a one-PCLK wide pulse on the RxFWrPtrIncCo signal input of the Receive Channel which has been programmed to receive that particular slot. When the RxFWrPtrIncCo signal is asserted, the receive FIFO control logic drives the data from the RxDataIn[19:0] input is clocked into the right or left buffer of the Rx FIFO depending on the value of the write pointer. If the LSB of the write pointer (WritePtr[0]) is 0, data from RxDataIn[19:0] is driven on the RxFWrDataRCo[19:0] bus input to the right buffer, and if WritePtr[0] is 1, the data from RxDataIn[19:0] is driven on the RxFWrDataLCo[19:0] bus input to the left buffer.

In normal mode, the data on RxFWrDataRCo[19:0] or RxFWrDataLCo[19:0] is written into the right and left buffer when their respective write enables are asserted. In test mode, a value 0x55555 is written into the Rx FIFO when the AACIDRRd input from the APB interface is asserted and FIFOTEST[1:0] is 10 and the data from the APB write databus (PWDDataInt[31:0]) is written into the Rx FIFO when the AACIDRWrr input is asserted and FIFOTEST[1:0] is 11 or 01.

The right and left buffers of the Rx FIFO always drive the RxFRdDataRCo[19:0] and RxFRdDataLCo[19:0] buses respectively with the data in the locations in the buffers to by the current value of the read pointer with the lower bit masked. For example when the read pointer points to "000" or "001" data in Right buffer "00" location and Left buffer "00" locations are driven. In non-Compact mode, the least significant 20 bits of the RxFRdDataCo[31:0] are driven by RxFRdDataRCo[19:0] when the LSB of the read pointer of the Rx FIFO is 0, and RxFRdDataLCo[19:0] when the LSB of the read pointer is 1. The MSB 12 bits of RxFRdDataCo are stuffed with zeros to make 32 bits. In Compact mode, the LSB 16 bits of the read data from the right buffer (RxFRdDataRCo[15:0]) are driven on RxFRdDataCo[15:0] and the LSB 16 bits of the read data from the left buffer (RxFRdDataLCo[15:0]) are driven on the RxFRdDataCo[31:16], thereby returning 32 bits of data.

When the FIFO is already full and an attempt to perform another write is seen, the RxOverrun output is asserted. This signal is cleared when the RxOEClr input signal is asserted. The RxTimeout output signal is asserted if the FIFO contains data and there are no further writes to the FIFO for TOC frames. The Timeout count is written into the TOC field of the AACIRXCR register. If Timeout occurs when the Rx FIFO is empty, the RxTOFE (Receive Timeout FIFO Empty) output is asserted. The RxBusy output is asserted when the slot 0 data on AACISDATAIN indicates that the incoming frame has valid data for a slot that has been programmed to be received by the Rx FIFO or if the Rx FIFO is non-empty. The RxHF (Receive FIFO half-full), RxFE (Receive FIFO Empty) and the RxFF (Receive FIFO full) output signals provide information on the fill status in the FIFO. All these signals are generated by the Receive FIFO Control Logic block and are readable through the AACISR register corresponding to each channel.

The Rx FIFO is flushed when the AACI is disabled by clearing the AacIfE bit in the AACIMAINCR register. When the Rx FIFO enable (RxFEn) bit in the AACIRXCR register is deasserted, the receive FIFO becomes a 1-deep holding register and the RxFF, RxHF and RxFE flags indicate the full/empty status of the holding register.

5.12 Receive FIFO Control Logic (AaciRxFCntl)

The Receive FIFO controller controls the read pointer and the write pointer. It also contains logic to detect and signal the FIFO Overrun condition, the Receive Timeout condition and the Receive Timeout FIFO Empty condition. This block also generates the RxBusy signal.

The WrPtrIncValid signal is asserted when a valid write access to the FIFO occurs. The following considerations apply when generating this signal:

- When there is a write access to the Receive FIFO, the write data is clocked into the FIFO only if the FIFO is not already full. If the FIFO is full when the write access occurs, a simultaneous read should also occur for the write data to be clocked in. In normal operation, the Interrupt Service routine is expected to read the data from the Receive FIFO at a faster rate than the AC-Link can write data into it. This check is done to protect from such writes in case they occur.
- This signal is asserted when the AACIDRRd signal is asserted if FIFOTEST[1:0] is 10 to allow a write of 0x55555 into the Receive FIFO. When FIFOTEST[1:0] is 11 or 01, WrPtrIncValid is asserted when the AACIDRWrr signal is asserted.

The WrPtr[2:0] signal is used as the write pointer signal. The Write pointer is incremented by one every time the WrPtrIncValid signal is asserted.

The write enable signals to the right and left buffers (RegFileWrEnRCo and RegFileWrEnLCo) are generated combinationally from the WrPtrIncValid signals. Writes to the left buffer are enabled when WrPtr(0) is 1 and writes to right buffer are enabled when WrPtr(0) is 0. This mechanism allows writes to the right and left buffers of the Tx FIFO alternately.

The RdPtrIncValid signal is asserted on a valid read to the FIFO when it is not already empty. In normal mode (FIFOTEST = 00), the RdPtrIncValid signal is asserted on assertion of the AACIDRRd signal from the APB interface when FIFOTEST[1:0] is 00. In test mode, the RdPtrIncValid signal is asserted when the AACIDRRd signal when FIFOTEST[1:0] is 10. The read pointer (RdPtr[2:0]) is incremented by 1 per assertion of the RdPtrIncValid signal in non-Compact mode. It is incremented by 2 per assertion of the RdPtrIncValid signal in Compact mode.

The write pointer and read pointer are cleared when the AACI is disabled by clearing the AacIfE bit of the AACIMAINCR register.

The Wrap signal is used to store the condition when the write pointer has wrapped around but the read pointer has not. The Wrap signal is used to calculate the FIFO fill level. The FIFOFillLevel signal is derived using the same mechanism as used for the transmit FIFO i.e. by subtracting the Read pointer from the value of the Write pointer with the Wrap bit prepended to the Write pointer. When the AACI is disabled, the Wrap signal is re-initialised to 0.

On receiving a slot of data (Slots 1–12), the Frame Decoder section drives the received data on the RxDataIn[19:0] bus. When the RxFWrPtrIncCo signal is asserted, RxDataIn[19:0] is clocked into the location of the FIFO as pointed to by the current value of the write pointer. After every write, the write pointer is incremented by one.

RxFWrDataLCo[19:0] and RxFWrDataRCo[19:0], the right and left buffer data for the FIFOs, are produced from RxDataIn[19:0] depending on the value of RSize and bit 0 of the write pointer. The WritePtr bit 0 determines which buffer stores data from RxDataIn. Data is directly stored in the right buffer when WrPtr[0] is 0 and in the left buffer when WrPtr[0] is 1. Depending on the value of RSize, appropriate number of MSB bits of RxDataIn[19:0] are LSB justified and the remaining bit of RxFWrDataLCo[19:0] and RxFWrDataRCo[19:0] are zeroed. **Table 5.12-1** shows generation of RxFWrDataLCo[19:0] and RxFWrDataRCo[19:0] with various values of RSize and WritePtr[0].

RSize	WritePtr[0]	Buffer	Data written to Buffer
12 bits	0	RxFWrDataRCo[19:0]	8{0}, RxDataIn[19:8]
	1	RxFWrDataLCo[19:0]	8{0}, RxDataIn[19:8]
16 bits	0	RxFWrDataRCo[19:0]	4{0}, RxDataIn[19:4]
	1	RxFWrDataLCo[19:0]	4{0}, RxDataIn[19:4]
18 bits	0	RxFWrDataRCo[19:0]	2{0}, RxDataIn[19:2]
	1	RxFWrDataLCo[19:0]	2{0}, RxDataIn[19:2]
20 bits	0	RxFWrDataRCo[19:0]	RxDataIn[19:0]
	1	RxFWrDataLCo[19:0]	RxDataIn[19:0]

Table 5.12-1 Receive FIFO Write Data Muxes

The Receive FIFO controller asserts the RxOverrun signal when the FIFO is already full and an additional data is received by the same FIFO, thereby resulting in an overrun of the FIFO. In this case, the data that caused the overrun is discarded. When the Receive FIFO is enabled (RxFEn bit of AACIRXCR register is 1), the RxOverrun is asserted if there is a write to the FIFO when the FIFO contains 8 valid entries. In Character mode (RxFEn bit is 0), the RxOverrun signal is asserted when there is a write to the Receive FIFO when it contains one valid data. The Overrun condition is cleared when the RxOEClrCo signal from the APB interface is asserted.

RxFFillLevel[3:0] is used to generate the Rx FIFO status flags.

The RxFE (Receive FIFO Empty) signal is asserted when there are no valid entries in the Receive FIFO. If the FIFO has one valid entry and there is a valid read detected, without a simultaneous write, the RxFE signal is asserted. If the FIFO is empty and there is a valid write detected, the RxFE signal is cleared. When the FIFO is in compact mode, it is said to be empty when it contains less than 2 entries, and it is said to be non-empty when it

contains 2 or more entries. When the Rx FIFO is disabled, the RxFE signal is asserted when there is a valid read to the holding register without a valid write. It is cleared when there is a valid write detected to the holding-register.

The RxHF (Receive FIFO Half-full) signal is asserted when the Receive FIFO contains 4 or more valid entries. It is asserted if the FIFO contains 3 valid entries and a valid write is detected without a simultaneous valid read. It is cleared when the FIFO contains 4 valid entries and a valid read is detected without a simultaneous valid write. If the FIFO is disabled, the RxHF signal is asserted if there is a valid write to the FIFO without a simultaneous valid read. It is cleared when there is a valid read without a simultaneous valid write.

The RxFF (Receive FIFO Full) signal is asserted when the FIFO contains 8 valid entries. When the FIFO has seven entries and there is another write without a simultaneous read, the RxFF signal is asserted. When the FIFO is already full and there is a read from the FIFO without a simultaneous write, the RxFF signal is cleared. When the FIFO is disabled, RxFF is asserted when there is a valid write without a simultaneous valid read. It is cleared when there is a valid read without a simultaneous valid write.

The RxHF and RxFF flags are cleared and RxFE flag is set when the AacIfE bit is deasserted.

A timeout counter is implemented to flag the RxTimeout and RxTOFE errors. The timeout counter is loaded with the contents of the TOC field of the AACIRXCR register when a new data is written into the AACIRXCR register, or when AacIfE is deasserted, or when the receive FIFO's write pointer increment is asserted. It is decremented by 1 at the end of every slot 0 reception, by which time, valid slot 12 data from the previous frame would have been written into the FIFO if the FIFO were programmed to receive slot 12 data. This prevents the counter from being decremented if the FIFO received only slot 12 in a frame. The counter is reloaded with the TOC value when RxTimeout or RxTOFE are asserted.

The RxTimeout flag is set when the timeout count is 0 at end of a frame and the FIFO is not empty. It is cleared when the Receive FIFO becomes empty. The RxTOFE flag is asserted when the timeout count is 0 at the end of a frame and the FIFO is empty. It is cleared on a write of 1 to the AACIINTCLR register's RxTOFEC bit. When the timeout counter decrements to 0, only one of the 2, RxTOFE or RxTimeout is asserted, and it is not reasserted once it is cleared till the counter counts down to 0 again. Here, it is assumed that the channel re-allocation / changing of Link-list is done in both, the RxTimeout ISR and the RxTOFE ISR.

The ChRxValid[12:1] vector is formed by ANDing the slot valid bits (SlotValid[12:1]) from the received Slot 0, generated by the Frame Decoder section with the slot valid bits in the AACIRXCR register at the end of slot 0 reception to determine which of the slots in the incoming frame will be stored in this Rx FIFO, provided the channel is receive enabled. The end of slot 0 reception is indicated by the assertion of the RxSlot0XorCo input signal. This signal is generated by the Receive Control logic when the double synchronised version of the RxSlot0 signal is toggled. The Frame Decoder generates the RxSlot0 signal in the AACIBITCLK domain at the end of slot 0 reception. The RxBusy signal is asserted when the ChRxValid[12:1] vector indicates that the incoming frame has valid data to be written into the Rx FIFO or when the Rx FIFO is non-empty. It is cleared when the FIFO is empty and the last data has been read out from this FIFO and there is no data for this channel in the current frame being received. It is also cleared when the AACI is disabled. On detection of Low Power mode entry, the RxBusy signal has to be cleared. Therefore the ChRxValid[12:1] is cleared at low power mode entry.

5.13 Receive FIFO Register File (AacIRxRegFile)

The Receive Register file is similar to the Transmit register file. Write data on the RxFWrDataCo[19:0] is clocked into the location pointed to by the current value of the write pointer (WrPtr[2:1]) on the rising edge of PCLK on which the write enable (RegFileWrEnCo) is sampled high. The RxFRdDataCo[19:0] bus is driven with the contents of the FIFO location pointed to by the current value of the read pointer (RdPtr[2:1]). The Register File is implemented as a bank of D-types. This block can be replaced with a compiled memory if required. The Register File also contains a 2-to-4 decoder to decode the write pointer and a 4-to-1 bus multiplexer for the read data bus.

5.14 DMA Transmit Channel Logic (AaciDMATChannel)

This module is instantiated once in the Aaci Top Level Module (Aaci) to implement the Transmit Channel Logic for the DMA capable channel 1. The DMA Transmit Channel logic for channel 1 is similar to the Transmit Channel Logic for the other channels, which has been described in **Section 5.8**, with some extra logic for the generation of the DMABREQTX signal. If the DMAEnable bit is cleared, the DMA Transmit Channel behaves exactly like the other Transmit Channels (Channels 2, 3 and 4). **Figure 5.14-1** shows the block Diagram of the DMA Transmit Channel.

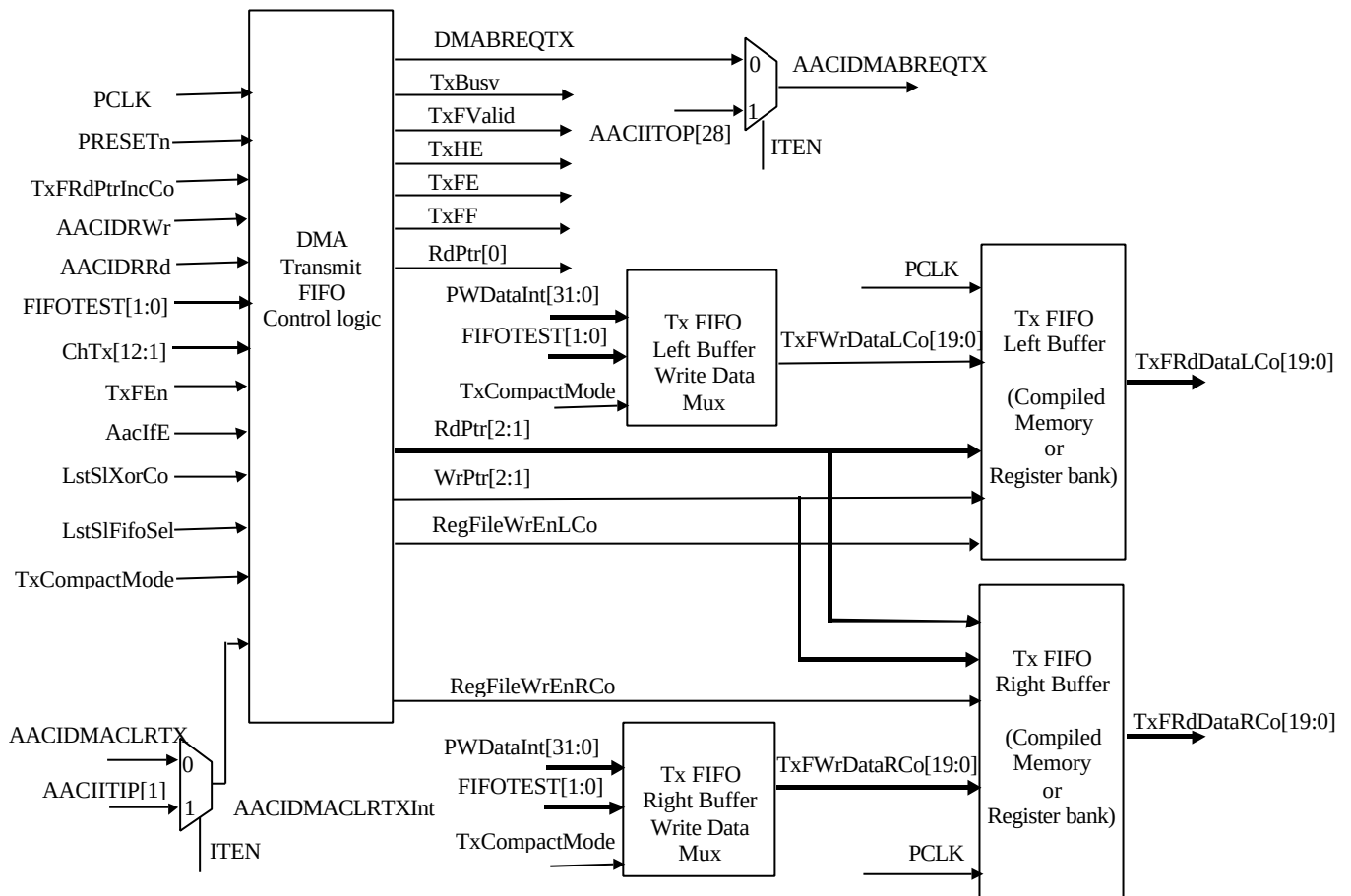


Figure 5.14-1 DMA Transmit Channel

The DMA Transmit Channel Logic block instantiates the DMA Transmit FIFO Control Logic (AaciDMATxFCntl) and the data storage elements (AaciTxRegFile). The AaciTxRegFile is instantiated twice to form the right and left buffers of the Tx FIFO. The AaciTxRegFile is described in **Section 5.10**.

5.14.1 DMA signal generation

The DMABREQTX signal output from the DMA Transmit FIFO Control Logic is asserted when the Tx FIFO contains 4 or less valid entries. It is cleared when the AACIDMACLRTXInt signal is asserted.

Integration Test Muxes

In normal mode, the value on the DMABREQTX signal output of the DMA Transmit FIFO Control Logic is driven on the AACI output port AACIDMABREQTX. When the Integration test enable (ITEN) bit of the AACITCR register

from the Register block is set, the ITOPDMAREQTX signal value is driven on the AACIDMABREQTX output port of the AACI. ITOPDMAREQTX is driven by bit 28 of the AACIITOP0 register.

In normal mode, the AACIDMACLRTXInt input to the DMA Transmit FIFO Control Logic is driven by the AACIDMACLRTX input of the AACI, which is driven by the DMA Controller. In Integration Test mode (ITEN = 1), ITIPDMACLRTX drives AACIDMACLRTXInt. ITIPDMACLRTX is driven by bit 1 of the AACIITIP register.

Refer to **Section 5.24** for more details on the Integration Test Strategy.

5.15 DMA Transmit FIFO Control Logic (AaciDMATxFCntI)

The DMA Transmit FIFO control logic controls the read and write pointers for the FIFO, besides controlling accesses to the FIFO register file. It also generates the FIFO fill level status signals and the Transmit FIFO Busy signal, TxBusy. The TxFValid signal generated by this module is an indication of the availability of data from the Tx FIFO for transmission in the next frame. The logic for the generation of these signals is similar to that in the Transmit FIFO Control Logic block that is described in **Section 5.9**. However, the DMA Transmit FIFO Control Logic includes additional logic for the generation of the DMA transmit request, DMABREQTX.

The DMABREQTX signal output of the DMA Transmit FIFO control logic indicates to the DMA Controller that the Tx FIFO contains 4 or less valid entries. DMABREQTX is asserted when the FIFO contains 5 valid entries and there is a valid read to the FIFO without a simultaneous valid write. Once set, it is cleared on the assertion of the AACIDMACLRTXInt input signal. It is also cleared when the AACI is disabled, or the channel is not DMA enabled (DMAEnable bit of the AACIMAINCR register is 0).

5.16 DMA Receive Channel Logic (AaciDMARChannel)

This module is instantiated once in the Aaci Top Level Module (Aaci) to implement the Receive Channel Logic for the DMA capable channel 1. The DMA Receive Channel logic for channel 1 is similar to the Receive Channel Logic for the other channels, which has been described in **Section 5.11**, with some extra logic for the generation of the DMA receive request signals. If the DMAEnable is cleared, the DMA Receive Channel behaves exactly like the other Receive Channels (Channels 2, 3 and 4). **Figure 5.16-1** shows the block diagram of the DMA Receive Channel.

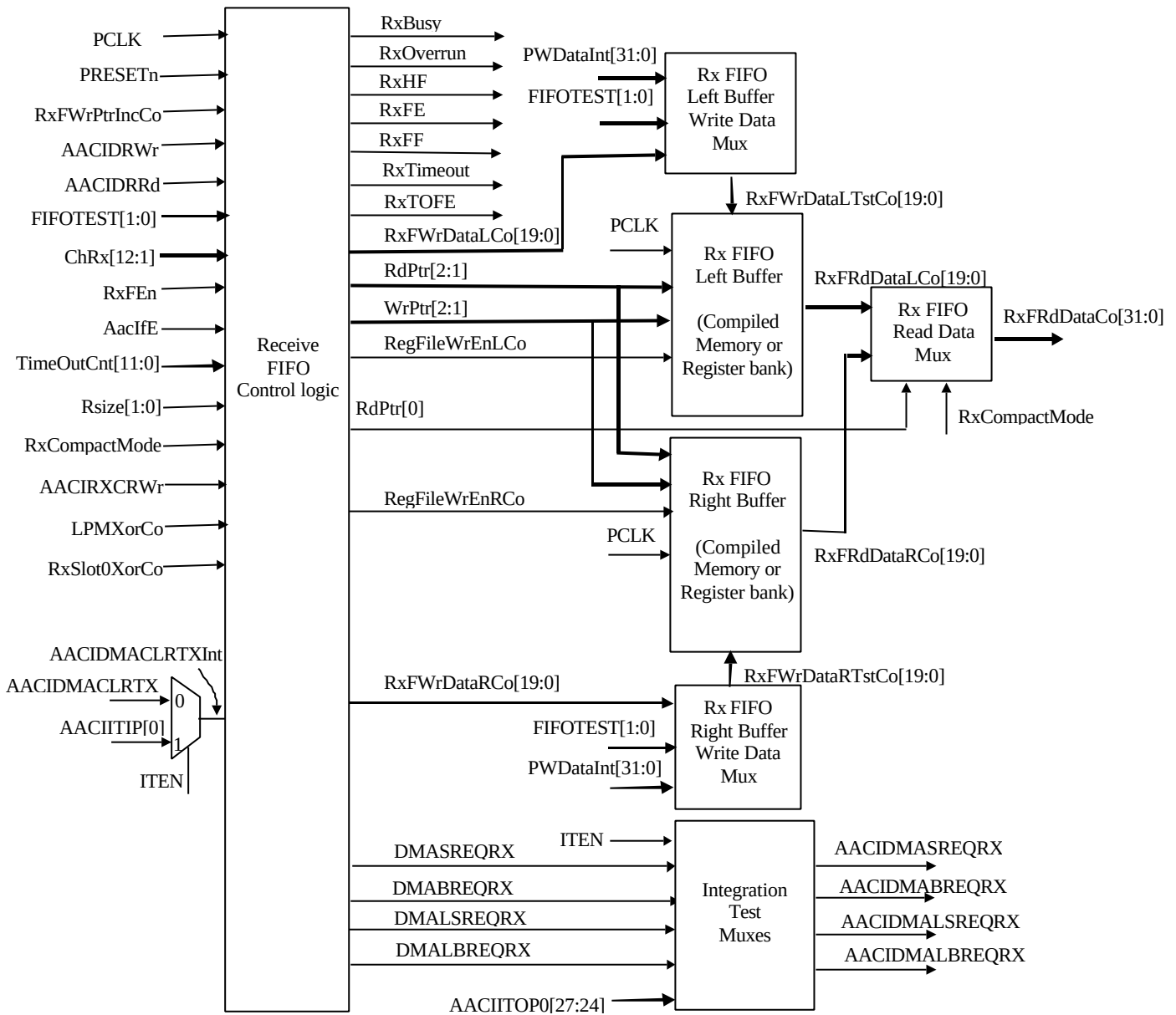


Figure 5.16-1 DMA Receive Channel Logic

The DMA Receive Channel Logic block instantiates the DMA Receive FIFO Control logic (AaciDMARxFCntI) and the data storage elements (AaciRxRegFile). The AaciRxRegFile is instantiated twice to form the right and left buffers of the Rx FIFO. The AaciRxRegFile is described in **Section 5.13**.

5.16.1 DMA signal generation

The DMA Receive FIFO Control Logic generates the DMASREQRX, DMABREQRX, DMALSREQRX and DMALBREQRX signals, which indicate the availability of single or burst data for the DMA. All the DMA receive requests are cleared when the AACIDMACLRRXInt signal is asserted.

Integration Test Muxes

In normal mode, the value on the DMASREQRX signal output of the DMA Receive FIFO Control Logic is driven on the AACI output port AACIDMASREQRX. When the Integration test enable (ITEN) bit of the AACITCR register from the Register block is set, the value of bit 3 of the ITOPDMAREQRX[3:0] is driven on the AACIDMARSEQRX output port of the AACI.

Similarly, the values on the DMABREQRX, DMALSREQRX and DMALBREQRX outputs of the DMA Receive FIFO Control Logic block are multiplexed with bits 2, 1 and 0 of the ITOPDMAREQRX[3:0] vector depending on the value of the ITEN bit. ITOPDMAREQRX[3:0] is driven by bits 27 to 24 of the AACIITOP0 register.

In normal mode, the AACIDMACLRXInt input to the DMA Receive FIFO Control Logic is driven by the AACIDMACLRX input of the AACI, which is driven by the DMA Controller. In Integration Test mode (ITEN = 1), ITIPDMACLRX drives AACIDMACLRXInt. ITIPDMACLRX is driven by bit 0 of the AACIITIP register.

Refer to **Section 5.24** for more details on the Integration Test Strategy.

5.17 DMA Receive FIFO Control Logic (AaciDMARxFCntI)

The DMA Receive FIFO controller controls the read pointer and the write pointer of the Rx FIFO in Channel 1. It also contains logic to detect and signal the FIFO Overrun condition and the RxBusy signal. The logic for the generation of these signals is similar to that in the Receive FIFO Control Logic block that is described in **Section 5.9**. However, the DMA Receive FIFO Control Logic includes additional logic for the generation of the DMA receive requests, DMASREQRX, DMABREQRX, DMALSREQRX and DMALBREQRX. There is also a difference in the mechanism for generation of the RxTimeout and RxTOFE signals when the channel 1 is DMA enabled. If channel 1 is not DMA enabled, it behaves exactly like the other channels.

A timeout counter is implemented to flag the RxTimeout and RxTOFE errors. The timeout counter is loaded with the contents of the TOC field of the AACIRXCR register when a new data is written into the AACIRXCR register, or when AacIfE is deasserted, or when the receive FIFO's write pointer increment is asserted. It is decremented by 1 at the end of every slot 0 reception, by which time, valid slot 12 data from the previous frame would have been written into the FIFO if the FIFO were programmed to receive slot 12 data. This prevents the counter from being decremented if the FIFO received only slot 12 in a frame. The counter is reloaded with the TOC value when RxTimeout or RxTOFE are asserted.

The RxTimeout flag is set when the timeout count is 0 at end of a frame and the FIFO is not empty. It is cleared when the receive FIFO becomes empty. The RxTOFE flag is asserted when the timeout count is 0 at the end of a frame and the FIFO is empty. It is cleared on a write of 1 to the AACIINTCLR register's RxTOFEC bit. When the timeout counter decrements to 0, if the DMAEnable bit is set, only one of the 2, RxTOFE or RxTimeout is asserted, and it is not reasserted once it is cleared till the counter counts down to 0 again. Here, it is assumed that the channel re-allocation / changing of Link-list is done in both, the RxTimeout ISR and the RxTOFE ISR. However, when the DMAEnable bit of the AACIMAINCR register is asserted, RxTOFE is asserted after RxTimeout is deasserted due to the FIFO becoming empty if the DMALSREQRX or DMALBREQRX is not asserted, and if timeout has not occurred. This is under the assumption that in the DMA channel case, the channel reallocation or Link-list change is done when the DMAC reads the last word from the FIFO. However, if the DMALSREQRX or DMALBREQRX are not asserted, the RxTOFE interrupt will have to be raised for channel reallocation.

The Timeout signal is asserted when the timeout counter decrements to 0. It is cleared under any of the following conditions:

- Data is written into the receive FIFO.
- The AACI is disabled.
- RxTOFE is cleared.
- DMALSREQRX or DMALBREQRX is asserted.

- RxTimeout is cleared when the channel is in non-DMA mode.

In non-Compact mode, the DMASREQRX signal output is asserted if the FIFO contains 1, 2 or 3 valid entries if the Timeout signal is asserted. If Timeout is not asserted, DMASREQRX is asserted only if the FIFO contains 2 or 3 valid entries. In compact mode, it is set when the receive FIFO contains 2 data irrespective of Timeout since only even number of slots can be read in compact mode. DMASREQRX is not asserted if DMALSREQRX is already asserted to avoid asserting it when the Timeout signal gets cleared due to DMALSREQRX assertion.

DMABREQRX is set when the Receive FIFO contains at least 4 data if Timeout is not asserted. It is set when the FIFO contains more than 4 data if Timeout is asserted. DMABREQRX is not asserted if DMALBREQRX is already asserted to avoid asserting it when the Timeout signal gets cleared due to DMALBREQRX assertion.

In non-compact mode, DMALSREQRX is set when the Receive FIFO contains 1 valid entry and Timeout is asserted. In compact mode, it is set when the receive FIFO contains 2 valid entries and Timeout is asserted since only even number of slots can be read in compact mode. However, if the DMASREQRX has already been asserted, DMALSREQRX is not asserted in order to keep DMASREQRX and DMALSREQRX mutually exclusive.

DMALBREQRX is set when the Receive FIFO contains 4 data and Timeout is asserted, provided DMABREQRX has not been asserted already. This ensures that DMABREQRX and DMALBREQRX are mutually exclusive.

All the DMA receive requests are cleared when the AACIDMACLRXInt signal is asserted. If a high on AACIDMACLRXInt is not detected for a given DMA request, it is possible that multiple DMA requests are asserted on a given clock. When the AACIDMACLRXInt signal remains asserted, all DMA requests remain cleared. When the AACIDMACLRXInt signal is deasserted, the FIFO fill level conditions are re-evaluated and a DMA request is raised, if required. All the DMA receive requests are also cleared when the DMAEnable is deasserted or AacIfE is deasserted.

5.18 Transmit Control Logic (AaciTxCntl)

Transmit Control Logic multiplexes data from the left buffer and the right buffer of the transmit FIFO of a channel. In the Frame Generator, the data from one of the four transmit FIFOs is selected to be the data source for the next slot. This module also generates the read pointer increment signals to the Tx FIFOs and the pulse on LstSIXorCo to indicate the toggling of the LstSITxed signal from the Slot 0 Generator in AACIBITCLK domain, which is used to clear the TxBusy signals from the 4 channels.

TxFRdDataLCo[19:0] and TxFRdDataRCo[19:0] from each FIFO is multiplexed to produce FifoxData[19:0], the multiplexer being controlled by TxCompactMode, TSize and bit 0 of the read pointer. In Compact Mode, when RdPtr0x ($x = 1, 2, 3, 4$) is 0, valid bits in the right buffer, depending on TSize, are moved left to make the data MSB justified, and the remaining bits are stuffed with zeros to form the 20-bit FIFO read data. When RdPtr0x is 1, valid bits in the left buffer, the remaining valid bits in the right buffer and the appropriate number of zeros are concatenated to form the 20-bit FIFO read data. In non-Compact mode, bits from only the right buffer or left buffer are selected depending on the value of RdPtr0x, and stuffed with zeros to form the 20-bit read data.

Table 5.18-1 shows the Transmit Control Mux for generation of FifoxData with various values of TxCompactMode, TSize and RdPtr0x.

Mode	TSize	RdPtr0x	FifoxData ($x = 1, 2, 3, 4$)
Compact Mode	12 bits	0	TxFRdDataRCo[11:0],8{0}
		1	TxFRdDataLCo[7:0],TxFRdDataRCo[19:16],8{0}
	16 bits	0	TxFRdDataRCo[15:0],4{0}
		1	TxFRdDataLCo[11:0],TxFRdDataRCo[19:16],4{0}
Non-Compact Mode	12 bits	0	TxFRdDataRCo[11:0],8{0}
		1	TxFRdDataLCo[11:0],8{0}
	16 bits	0	TxFRdDataRCo[15:0],4{0}

		1	TxFRdDataLCo[15:0],4{0}
	18 bits	0	TxFRdDataRCo[17:0],2{0}
		1	TxFRdDataLCo[17:0],2{0}
	20 bits	0	TxFRdDataRCo[19:0]
		1	TxFRdDataLCo[19:0]

Table 5.18-1 Transmit Control Mux

One bit of TxFRdPtrIncCo[4:1] corresponding to a FIFO is asserted for one PCLK duration when that particular FIFO has been selected for transmitting the current slot when RdPtrIncSync toggles. RdPtrInc and FifoSel[4:1] are generated in the Slot 0 generator section in the AACIBITCLK domain. RdPtrInc is double synchronised to the PCLK domain to obtain RdPtrIncSync. In the Transmit Control Logic module, the RdPtrIncSync signal is delayed by one PCLK to form D1RdPtrIncSync. The XOR of RdPtrIncSync and D1RdPtrIncSync will result in a one-PCLK wide pulse on every edge of RdPtrIncSync. This XOR pulse, RdPtrIncXor, is used for the detection of the toggling of the RdPtrInc signal. RdPtrIncXor is ANDed with each bit of the FifoSel[4:1] vector to form the TxFRdPtrIncCo[4:1] vector. The bits of the TxFRdPtrIncCo[4:1] vector are connected to the Tx FIFOs of the 4 channels to increment the read pointers of the FIFOs.

The Slot 0 Generator module generates toggles LstSITxed in the AACIBITCLK domain whenever the last slot that was programmed to be transmitted from a channel has been completely shifted out of the transmit shift register. LstSITxed is double synchronised to PCLK domain to form LstSITxedSync. The XOR of this signal and its PCLK delayed version, D1LstSITxed results in a one PCLK wide pulse on the LstSIXORCo output signal of the Transmit Control Logic module whenever LstSITxed is toggled. This signal is used along with one bit of LstSIFifoSel[4:1] from the Slot 0 Generator module in the Tx FIFOs of each channel to deassert TxBusy of the channel.

5.19 Receive Control Logic (AaciRxCntl)

Receive Control logic selects the FIFO to write the incoming data RxDataIn[19:0], and appropriately sets the corresponding bit in RxFWrPtrIncCo[4:1] to 1. This module also generates the XOR pulses to indicate toggling of LPMToglSync and RxSlot0Sync. The LPMXorCo and RxSlot0XorCo pulses are used in the Rx FIFO control logic of the 4 channels to detect low power mode entry and end of slot 0 reception respectively.

Bits 12 to 1 in the AACIRXCRx (ChxRx[12:1]) are ANDed with bits 12 to 1 of RxSIVValid[12:1] vector from the Frame Decoder in the AACIBITCLK domain to form SlotNRxFifoX[12:1]. Only one bit will be set in the SlotNRxFifoX[12:1] vector corresponding to the channel which has been programmed to store the data of the previous slot, if the previous slot was a valid slot. All bits of the other SlotNRxFifoX[12:1] vectors are zeroed. All the bits in the SlotNRxFifoX[12:1] are ORed together. The resultant of this ORing is qualified with the RxEnx and RxSINoUpdXorCo to produce the Rx FIFO write pointer increment signal (RxFWrPtrIncCo[x]). RxSINoUpdXorCo is the pulse generated by the XOR of RxSINoUpdSync and its delayed version, D1RxSINoUpdSync, where RxSINoUpdSync is generated by double synchronising the RxSINoUpd signal to PCLK domain. The toggling of the RxSINoUpd signal from the Frame decoder indicates that a valid slot has been received on the AACISDATAIn input port. The RxSINoUpdXorCo pulse is also used to detect the end of a slot reception in the APB interface and Register block to clock in data from the RxDataIn[19:0] bus into the slot receive registers.

The LPMTogl signal is toggled in the AACIBITCLK domain in the Frame Generator section to indicate the entry of the AC-Link into Low Power mode. The LPMTogl signal is double synchronised to PCLK domain to generate LPMToglSync. LPMToglSync is XOR-ed with its one PCLK delayed version, D1LPMToglSync to generate LPMXorCo. LPMXorCo is used to clear the RxBusy (x = 1, 2, 3, 4) signals of the 4 Receive Channels and SI2RxBusy on Low Power mode entry.

RxSlot0 is toggled in the AACIBITCLK domain by the Frame decoder module to indicate end of slot 0 data reception. The toggling of this signal is also detected by XORing the double synchronised RxSlot0Sync with its delayed version, D1RxSlot0Sync to form RxSlot0XorCo. RxSlot0XorCo is used in the 4 Receive Channels to

assert the RxBusyx (x = 1, 2, 3, 4) signals and in the APB interface and Register block to assert the SlxRxBusy (x = 1, 2, 12) signals

5.20 Interrupt Generator (AacIntrGen)

5.20.1 Interrupt Generation Logic

All the interrupts generated by the AACI are generated in this block. The Raw status of all the interrupts except the Transmit underrun interrupts, the Wakeup interrupt and the GPIO interrupt are inputs to the Interrupt Generator module from the channels or the APB Interface and Register block. The raw status of the channel interrupts are generated in the Receive Channel Logic, DMA Receive Channel Logic, Transmit Channel Logic and DMA Transmit Logic blocks of the AACI. The raw slot register interrupts are generated in the APB Interface and Register block. The Transmit underrun interrupts, the Wakeup interrupt and GPIO interrupt are generated in the Interrupt generator block. All the interrupts are cleared when the AACI is disabled.

Receive Interrupt Generation

The RxHF flag from the Rx FIFO is qualified with the RxIE bit in the AACIIE register to generate the Receive interrupt, RXINTR for a channel. The RXINTR signal is asserted if the RxIE bit is asserted and the Rx FIFO is at least half-full. It is cleared when the FIFO becomes less than half-full or when the mask bit, RxIE is deasserted. Four Receive interrupt signals, RXINTR1, RXINTR2, RXINTR3 and RXINTR4 are generated in this way, one for each Rx Channel.

Transmit Interrupt Generation

The TxHE flag from the Tx FIFO is qualified with the TxIE bit in the AACIIE register and the AacIfE bit in the AACIMAINCR register to generate the Transmit interrupt, TXINTR for a channel. The TXINTR signal is asserted if the TxIE bit is set and the Tx FIFO is at least half-empty. It is cleared when the FIFO becomes less than half-empty or when the mask bit TxIE or AacIfE is deasserted. The Tx FIFO is flushed when AacIfE is deasserted and hence the TxHE flag is asserted. However, the AACITXINTR interrupt has to be cleared when AacIfE is deasserted. Hence AACITXINTR generation is qualified with AacIfE. Four interrupts, TXINTR1, TXINTR2, TXINTR3 and TXINTR4 are generated in this way, one for each Tx Channel.

Receive Overrun Interrupt Generation

The RxOverrun flag from the Rx FIFO is qualified with the RxOIE bit in the AACIIE register to generate the Receive Overrun interrupt, ORINTR for a channel. ORINTR is asserted if the RxOIE bit of the AACIIE register is set and the RxOverrun signal from the corresponding Rx FIFO is asserted. It is cleared when either of the two signals is deasserted. The RxOverrun condition is cleared on a write of 1 to the RxOEC bit of the AACIINTCLR register. Four interrupts are generated in this way, one for each Rx Channel.

Transmit Underrun Interrupt Generation

If TxUnderrunSync, the double synchronised version of the TxUnderrun signal from the Slot 0 Generator block in the AACIBITCLK domain is toggled, and the mask bit TxUIE in the AACIIE register is set, the Transmit Underrun interrupt for a channel, URINTR is asserted high. It is cleared on a write of '1' to the TxUEC bit of the AACIINTCLR register, or if the AacIfE bit in the AACIMAINCR register or the TxUIE bit in the AACIIE register are cleared. Four Transmit Underrun interrupts are generated in this way, one for each channel.

Receive Timeout Interrupt Generation

The Receive Timeout Interrupt, RXTOINTR is asserted if the RxTIE bit is set in the AACIIE register and the RxTimeout signal from the Rx FIFO is asserted. It is cleared when either of the two signals is not asserted. Four Receive Timeout interrupts are generated in this way, one for each Rx Channel.

Transmit Complete Interrupt Generation

The Transmit complete interrupt, TXCINTR is asserted when TxBusy is deasserted indicating that the Tx FIFO and the transmit shift register are empty. It is deasserted when TxBusy is asserted indicating valid data in the Tx FIFO. It is qualified by the mask bit, TxClE from the AACIIE register and the AacIfE bit from the AACIMAINCR register. When AacIfE is deasserted, TxBusy is deasserted. But the TXCINTR needs to be cleared when AacIfE is deasserted. Hence the qualification with AacIfE is necessary. Four Transmit Complete interrupts, TXCINTR1, TXCINTR2, TXCINTR3 and TXCINTR4 are generated in this way, one for each channel.

Receive Timeout FIFO Empty Interrupt Generation

The receive timeout FIFO empty interrupt, RXTOFEINTR for a channel is asserted if the RxTOIE bit in the AACIIE register is set and the RxTOFE signal from the Rx FIFO is asserted. It is cleared when a '1' is written into the RxTOFEC bit of the AACIINTCLR register. Four such signals, RXTOFEINTR1, RXTOFEINTR2, RXTOFEINTR3 and RXTOFEINTR4 are generated, one for each channel.

Wakeup Interrupt Generation

The WINTRComb signal, which is used to generate combinational wakeup interrupt signal, is generated by a combinational AND of LowPowerMode and AacIfE bits in the AACIMAINCR and AACISDATAIN. The WINTRSyncComb signal which is used in generating sequential wakeup interrupt is generated by a combinational AND of LowPowerMode and AacIfE bits in the AACIMAINCR and AACISDATAINSync. AACISDATAIN is double synchronized in PCLK domain to get AACISDATAINSync. If the WISE bit in the AACISLIEN register is set to 1, the WINTRSyncComb is clocked and is used to drive the WINTR signal, qualified by the mask bit WakeupIE of the AACISLIEN register. Otherwise, WINTRComb is directly used to drive WINTR, qualified by the mask bit WakeupIE in the AACISLIEN register. **Figure 5.20-1** shows the generation of the Wakeup Interrupt.

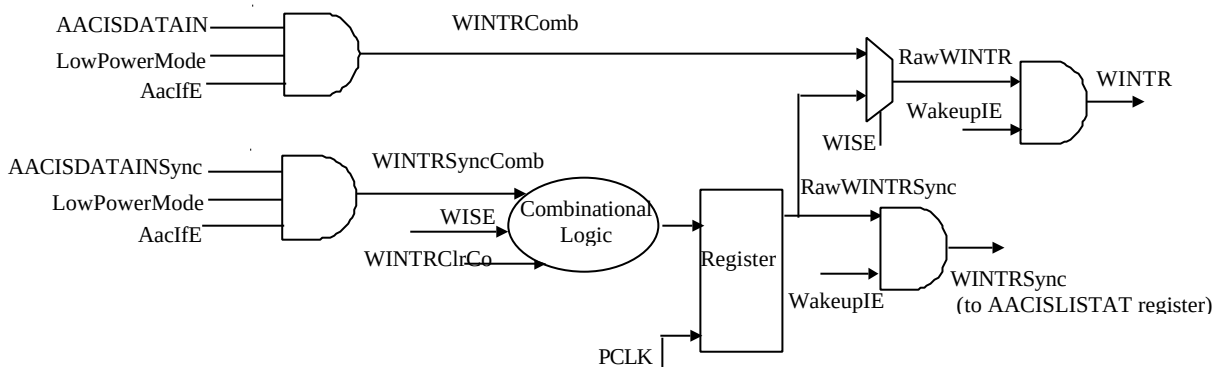


Figure 5.20-1 Wakeup Interrupt Generation Logic

GPIO Interrupt Generation

The GPIO interrupt, GPIOINTR is asserted when bit 0 in slot 12 of the incoming AACISDATAIN is 1. The Frame Decoder toggles a signal SI12RxChng to indicate this condition. SI12RxChng is double synchronised to the PCLK domain to form SI12RxChngSync. This signal and its delayed version D1SI12RxChngSync will be XORed to determine if the bit 0 in slot 12 of incoming AACISDATAIN is 1. If bit 0 of slot 12 is set, it indicates that one or more of the bits in slot 12 have changed since the last frame. The result of the XOR logic is qualified with its mask bit, GpioIE from the AACISLIEN register and the SI12RxEn bit from the AACIMAINCR register to assert the GPIO interrupt. When AACISL12RXRdCo signal from the APB Interface and Register block is asserted indicating that the data in the Slot 12 Rx Register is read, this interrupt is cleared.

Slot 12 Transmit Interrupt Generation

The Slot 12 Tx interrupt, S12TXINTR is generated by qualifying the S12TxEmpty flag from the APB Interface and Register block with the Slot12TxIE bit from the AACISLIEN register and the AacIfE bit from the AACIMAINCR register. Thus it is asserted when the data from the AACISL12TX register gets loaded into the transmit shift

register, and cleared on a write to the AACISL12TX register. Since the S12TxEmpty flag is asserted and S12TXINTR has to be deasserted on deassertion of AacIfE, the qualification with AacIfE is required.

Slot 12 Receive Interrupt Generation

The Slot 12 Rx, S12RXINTR interrupt is generated by qualifying the S12RxValid flag with the Slot12RxIE bit from the AACISLIEN register and the AacIfE bit from the AACIMAINCR register. Thus it is asserted when the AACISL12RX register has new data that has not been read and cleared on a read from the AACISL12RX register.

Slot 2 Transmit Interrupt Generation

The Slot 2 Tx interrupt, S2TXINTR is generated by qualifying the S2TxEmpty flag with the Slot2TxIE bit from the AACISLIEN register and the AacIfE bit from the AACIMAINCR register. Thus it is asserted when the data from the AACISL2TX register gets loaded into the transmit shift register, and cleared on a write to the AACISL2TX register. Since the S2TxEmpty flag is asserted and S2TXINTR has to be deasserted on deassertion of AacIfE, the qualification with AacIfE is required.

Slot 2 Receive Interrupt Generation

The Slot 2 Rx interrupt, S2RXINTR is generated by qualifying the S2RxValid flag with the Slot2RxIE bit from the AACISLIEN register and the AacIfE bit from the AACIMAINCR register. Thus it is asserted when the AACISL2RX register has new data that has not been read and cleared on a read from the AACISL2RX register.

Slot 1 Transmit Interrupt Generation

The Slot 1 Tx interrupt, S1TXINTR is generated by qualifying the S1TxEmpty flag with the Slot1TxIE bit from the AACISLIEN register and the AacIfE bit from the AACIMAINCR register. Thus it is asserted when the data from the AACISL1TX register gets loaded into the transmit shift register, and cleared on a write to the AACISL1TX register. Since the S1TxEmpty flag is asserted and S1TXINTR has to be deasserted on deassertion of AacIfE, the qualification with AacIfE is required.

Slot 1 Receive Interrupt Generation

The Slot 1 Rx interrupt, S1RXINTR is generated by qualifying the S1RxValid flag with the Slot1RxIE bit from the AACISLIEN register and the AacIfE bit from the AACIMAINCR register. Thus it is asserted when the AACISL1RX register has new data that has not been read and cleared on a read from the AACISL1RX register.

Combined Interrupt Generation

The individual masked interrupt outputs are also combined into a single output that is an OR function of the individual sources. This output can be connected to the system interrupt controller to provide another level of masking on an individual per-peripheral basis. The combined interrupt, INTR, is asserted when any of the individual interrupts above are asserted and the corresponding mask is enabled.

All the interrupts described above are readable through the AACIISR1, AACIISR2, AACIISR3, AACIISR4 and AACISLISTAT registers.

5.20.2 Integration Test Muxes

Based on the value of the ITEN bit, all the interrupts generated by the interrupt generation logic are multiplexed with the Integration Test output register bits and the outputs of these multiplexers are used to drive the interrupt output ports of the AACI.

When ITEN bit is cleared, the RXINTR1 interrupt signal generated by the Interrupt generation logic is driven on the AACIRXINTR1 interrupt output pin of the AACI. When ITEN is set, bit 23 of the AACIITOP0 register is driven on AACIRXINTR1. Similarly, the RXINTR2-4, TXINTR1-4, ORINTR1-4, URINTR1-4, RXTOINTR1-4 and TXCINTR1-4 signals are muxed with bits 22 to 0 of the AACIITOP0 register to drive the AACIRXINTR2-3, AACITXINTR1-4, AACIORINTR1-4, AACIURINTR1-4, AACIRXTOINTR1-4 and AACITXCINTR1-4 output pins of the AACI respectively. RXTOFEINTR1-4, WINTR, GPIOINTR, S12RXINTR, S12TXINTR, S2RXINTR, S2TXINTR, S1RXINTR, S1TXINTR and INTR are muxed with bits 12 to 0 of the AACIITOP1 register to drive the

AACIRXTOFEINTR1-4, AACIWINTR, AACIGPIOINTR, AACIS12RXINTR, AACIS12TXINTR, AACIS2RXINTR, AACOS2TXINTR, AACIS1RXINTR, AACIS1TXINTR and AACIINTR output pins respectively. Refer to **Section 5.24** for more details on the Integration Test Strategy.

5.21 PCLK to AACIBITCLK domain Synchronisers (AaciPtoBSync)

This block contains all synchronisers in the design that synchronise signals entering the AACIBITCLK domain.

All signals to be synchronised are double synchronised through D-type flip-flops clocked by AACIBITCLK.

The TxFValid[4:1]vector is generated by the transmit FIFOs operating on PCLK. It is required for the Slot 0 Generator section to detect if transmit data is available in a Tx FIFO. The SlxTxChng ($x = 1, 2, 12$) signals are generated in the APB interface to indicate valid writes to the slot transmit registers. Similarly, the ChxTxChng ($x = 1, 2, 3, 4$) signals are also generated in the APB interface to indicate valid writes to the AACITXCR registers. The SlxTxEn ($x = 1, 2, 12$) signals are driven by the slot transmit enable bits from the AACIMAINCR register. These signals are used in the Slot 0 Generator to detect availability of data from the slot transmit registers for transmission on the AACISDATAOUT output of the AACI. The AacIfE bit of the AACIMAINCR is a global enable of the AACI. It is double synchronised to the AACIBITCLK domain to be used in all the modules of the AACI which operate on AACIBITCLK or nAACIBITCLK. The synchronised LoopBack bit of the AACIMAINCR is used in the Frame Decoder to select the AACISDATAOUT input instead of the AACISDATAIN input for decoding.

5.22 AACIBITCLK to PCLK domain Synchronisers (AaciBtoPSync)

This block contains all synchronisers in the design that synchronise signals entering the PCLK domain.

All signals to be synchronised are double synchronised through D-type flip-flops clocked by PCLK.

The Ch1TxClr, Ch2TxClr, Ch3TxClr and Ch4TxClr signals are generated by the Slot 0 Generator module to allow further writes to the AACITXCR1-4 registers. The SI1TxClr, SI2TxClr and SI12TxClr are generated in the Slot 0 Generator module and the synchronised signals are used in the APB interface to indicate that the data from previous writes to the slot transmit registers has been made available to the AACIBITCLK domain. The TxUnderrun1-4 are the transmit underrun flags generated by the Slot 0 Generator. The synchronised TxUnderrunxSync ($x = 1, 2, 3, 4$) signals are used in the Interrupt Generator block to generate the Transmit Underrun Interrupts. The SI12TxEx, SI2TxEx and SI1TxEx signals are generated in the Slot 0 Generator module to indicate that the data from the respective slot transmit registers has been loaded into the transmit shift register. The synchronised versions of these signals are used to assert the SI1TxEmpty, SI2TxEmpty and SI12TxEmpty signals in the APB Interface and Register block. The LstSITxed signal is toggled by the Slot 0 Generator module to indicate that the last slot that had been programmed to be transmitted from one of the channels has been transmitted completely. After synchronisation, the LstSITxedSync signal is used to clear the TxBusy signals from the Tx FIFOs. The Frame Generator toggles the LPMTogl signal on detection of Low Power mode entry. LPMToglSync is used to clear the SI2RxBusy and the RxBusy signals from the Rx FIFOs on Low Power mode entry. RxSlot0 and RxSINoUpd are generated by the Frame Decoder section to indicate the end of slot 0 reception and the end of a valid slot reception respectively. After synchronisation, these signals are used to set the Rx FIFO and Slot register busy signals. RdPtrInc is generated by the Slot 0 Generator section. The synchronised version of this signal is used in the Transmit Control Logic section to generate the read pointer increment signals to the 4 transmit channels.

5.23 Revision Designator block (AaciRevAnd)

This module contains a single AND gate to be used as a place-holder cell to mark the Revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.

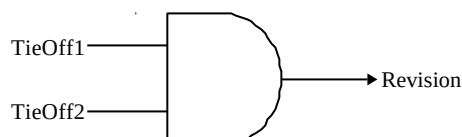


Figure 5.16-5.23-1 RevisionAnd component

This AND gate is instantiated four times at the top level, since the revision number is a four bit quantity. In the present version of the AACI, all the inputs are tied to zero at the top level. Outputs of all AND gates are concatenated and connected to the output read multiplexer of the APB interface logic. This helps software to identify the revision number through an APB read.

5.24 AACI Integration Test Strategy

This section describes the hardware requirements (Integration registers) to support the Integration testing.

The ITEN bit in the AACITCR register is used to force the multiplexers on the non-AMBA inputs and outputs of the AACI to use the test values.

5.24.1 Input Integration Test Harness

Figure 5.24-1 explains the implementation details of the input integration test harness. The ITEN bit is used as the control bit for the multiplexer, which is used in the read path of the AACIDMACLRRX and AACIDMACLRTX intra-chip inputs. If the ITEN control bit is de-asserted, the AACIDMACLRRX and AACIDMACLRTX intra-chip inputs are routed as the internal AACIDMACLRRX and AACIDMACLRTX inputs respectively, else the stored register values is driven on the internal line. The read-only bit AACISDATAIN is connected directly to the primary input pin.

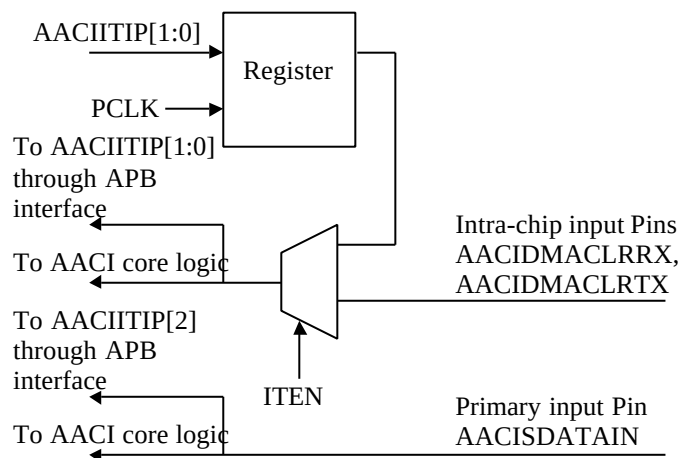


Figure 5.24-1 Input Integration test harness

5.24.2 Output Integration Test Harness

The AACIITOP0[29:0] and AACIITOP1[12:0] registers are used for integration testing of the non-AMBA outputs of the AACI. AACIITOP0[28:0] and AACIITOP1[12:0] are used in the Integration testing of intra-chip outputs and AACIITOP[29] is used in the Integration testing of the primary output AACISDATAOUT. **Figure 5.24-2** explains the implementation details of the output integration test harness in the case of intra-chip outputs. **Figure 5.24-3** explains the implementation details of the output test integration register in the case of primary outputs. The Read

path is different for the intra-chip and the primary output pins. In the case of primary outputs, reads return the values that have been written to the corresponding bit in the AACIITOP0 register. In the case of intra-chip outputs, if ITEN bit is set, AACIITOP0[28:0] and AACIITOP1[12:0] are connected with the actual outputs generated by the AACI core logic, else these pins are connected with the stored register values.

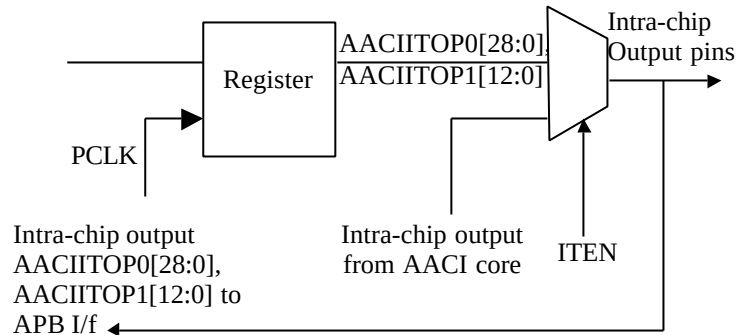


Figure 5.24-2 Output integration test harness - Intra-chip outputs

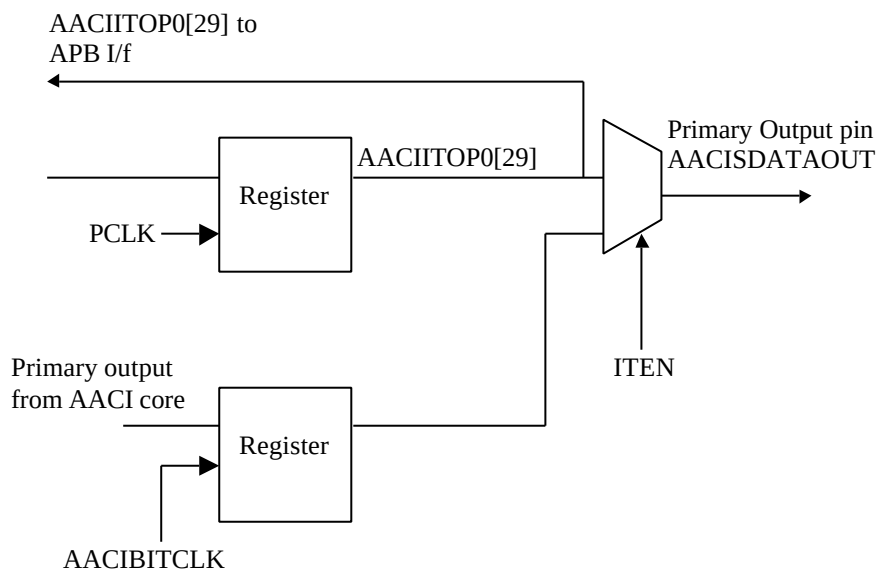


Figure 5.24-3 Output integration test harness - Primary outputs

5.25 Frequency Considerations

The implementation limits the frequency ratios of PCLK to AACIBITCLK that are allowed for proper functioning of the AACI.

For reception, the RxSValid[12:1] is generated by the Frame Decoder section of the AACI in the AACIBITCLK domain. Each bit of the RxSValid[12:1] is asserted when the valid data corresponding to the slot corresponding to that bit has been stored in RxDataIn[19:0]. RxSNoUpd is toggled in the AACIBITCLK domain at the end of every valid slot received on AACISDATAIN, to indicate slot number updation. The Rx FIFOs operate in the PCLK domain. Hence, to transfer the data in RxDataIn[19:0] to the Rx FIFO, the RxSNoUpd signal needs to be double

synchronised from AACIBITCLK domain to PCLK domain to generate a signal, RxSINoUpdSync. In the PCLK domain, after one bit of the RxSValid[12:1] vector is detected to be set when RxSINoUpdSync toggles, the Data from RxDataIn[19:0] is clocked into the Rx FIFO. Hence, a minimum of three PCLKs is required for each slot of data to be transferred from the RxDataIn[19:0] to the Rx FIFO, 2 PCLKs for double synchronisation and one for detecting the toggling of RxSINoUpdSync by XORing it with a one-PCLK delayed version, D1RxSINoUpdSync. In the worst case, when RxSValid[12] is set and data from RxDataIn[19:0] has to be stored in the receive FIFO during slot 0, which is 16 AACIBITCLKs wide, the data for Slot 12 will have to be stored in the Rx FIFO within these 16 AACIBITCLKs. Hence the time for the 16 AACIBITCLKs should be equal to or greater than 3 PCLKs.

In the case of transmission, a RdPtrInc signal is generated by the Slot 0 Generator section of the AACI in the AACIBITCLK domain. This signal is toggled at the start of a slot when the source of the data in that slot is a channel. A vector, FifoSel[4:1] is also generated by the Slot 0 Generator to indicate the channel that is the source of the data. The RdPtrInc signal is double synchronised into the PCLK domain to produce RdPtrIncSync. The toggling of the RdPtrIncSync signal is used as a qualifier to generate the TxFRdPtrIncCo[4:1] vector, which is the signal that is used by the Tx FIFOs to increment their read pointers. New data will be available on the Tx FIFO's read databus after the read pointer has been incremented. Hence the time for 20 AACIBITCLKs should be equal to or greater than 4 PCLKs - 2 for double synchronisation, one for detecting the toggling of RdPtrIncSync and one for the new data on the read databus of the Tx FIFO to settle down and be available before the data for the next slot is transmitted.

Considering the worst case, the PCLK frequency has to be at least 4/20 times (i.e. one-fifth) the AACIBITCLK frequency.

5.26 Software Restrictions

The software should ensure the following:

- The AACI control registers should not be written into during transmission/reception. Software should poll for the TxBusy and RxBusy bits in the AACISR_x ($x = 1, 2, 3, 4$) to be cleared before writing into the AACIRXCR_x ($x = 1, 2, 3, 4$) and AACITXCR_x ($x = 1, 2, 3, 4$) registers.
- The AACITXCR_x ($x = 1, 2, 3, 4$) and AACIRXCR_x ($x = 1, 2, 3, 4$) registers should not be programmed for more than 8 slots per channel at a time.
- Once a value has been written into a AACITXCR_x ($x = 1, 2, 3, 4$) register, the AACI ignores further writes to the same register till the data from the previous write is synchronised to the AACIBITCLK domain.
- Software should not program more than one channel to transmit data for a slot at a time. Similarly, it should not program more than one channel to receive data for a slot at a time.
- The ForcedRESET bit of the AACIRESET register should be kept asserted low for at least 1 us in Low Power Mode in order to restart the AC-Link with a cold reset.
- The ForcedSYNC bit of the AACISYNC register should be kept asserted high for at least 1.3 us in Low Power Mode in order to restart the AC-Link with a warm reset. The ForcedSYNC bit should not be set to 1 when the AACI is in normal mode.
- When the Transmit FIFO of a channel is in Compact Mode, the TSize bits in the corresponding AACITXCR_x ($x = 1, 2, 3, 4$) register should indicate only a 12-bit or 16-bit data width. Similarly, when the Rx FIFO of a channel is in Compact Mode, the RSize bits in the corresponding AACIRXCR_x ($x = 1, 2, 3, 4$) register should indicate only a 12-bit or 16-bit data width.
- All slots programmed to be transmitted from or received by a given channel should have the same sample rate.
- Data should be stored in the Tx FIFO lower slot first. For example, if Slots 4 and 5 are to be stored in a FIFO, Slot 4 should be the first data into the FIFO and Slot 5 the second.
- The FIFOs should not be disabled in Compact mode.

- The Slot Tx/Rx register enables (SlxTxEn and SlxRxEn [$x = 1, 2, 12$] bits in the AACIMAINCR register) should not be disabled while the corresponding Slot Tx/Rx busy bit of that register (SlxTxBusy and SlxRxBusy [$x = 1, 2, 12$] bits in the AACISLFR register) is high.
- The AC-Link can be programmed into Low Power Mode only through the Slot Tx registers and not through the channels.
- All the receive logic and the FIFO transmit logic should be disabled and not BUSY before Low Power Mode is entered. The MainRxBusy bit of the AACIMAINFR register and the TxBusy bits of the AACISR x ($x = 1, 2, 3, 4$) registers should be polled for zero before programming the CODEC into low power mode.
- The LowPowerMode bit in the AACIMAINCR register should be set only after the CODEC has been programmed to go to Low power mode, plus the time it takes for AACIBITCLK and AACISDATAIN to go low (normally the TS2_PDOWN parameter in the CODEC datasheets). If the LowPowerMode bit were set before this, the low to high transitions on the AACISDATAIN line could result in the Wakeup interrupt being asserted. The Wakeup events on the AACISDATAIN line should be monitored once the LowPowerMode bit is set.
- Once a Slot Tx register has been written into and enabled, a new value should not be written into it until the SITxEmpty flag corresponding to that register is asserted.
- Switching from Non-Compact mode to Compact mode should not be done on the fly. The AACI should first be disabled before switching from Non-Compact mode to Compact mode.
- Slot 2 data should be written first and then slot 1 data in the case of CODEC register write accesses. This is to prevent slot 1 data from being transmitted before the associated slot 2 data is available.
- If slots 1 and 2 are programmed in the same channel along with other slots, dummy data will have to be written for slot 2 during CODEC register read accesses. This is to prevent underrun.
- For CODEC register write accesses, if slots 1 and 2 are in separate channels, no slot other than slot 2 should be programmed in the same channel. Note that slot 1 can be programmed with other slots in the same channel.
- SCRA bits in the AACIMAINCR register can be changed only when there is no data for slots 1 and 2 in the slot registers and the channels.
- Once disabled, the AacIfE bit of the AACIMAINCR register should be held low for at least 2 AACIBITCLK-durations + 3 PCLK-durations before being re-enabled.

6 AACI FUNCTIONAL VERIFICATION TEST DETAILS

The AACI Functional Verification testbench is an APB BusTalk testbench. The VHDL and Verilog BusTalk testbench files reside in the **verification/vhdl** and **verification/verilog** directories respectively. The test vectors that are applied to the BusTalk testbenches are in the **verification/bustest** directory.

6.1 AACI VHDL Verification Testbench

The ARM PrimeCell AACI VHDL test world uses a variant of a top level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The testbench is designed to be used with most common VHDL simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The testbench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

Figure 6.1-1 illustrates the Bustalk VHDL testbench for running test vectors.

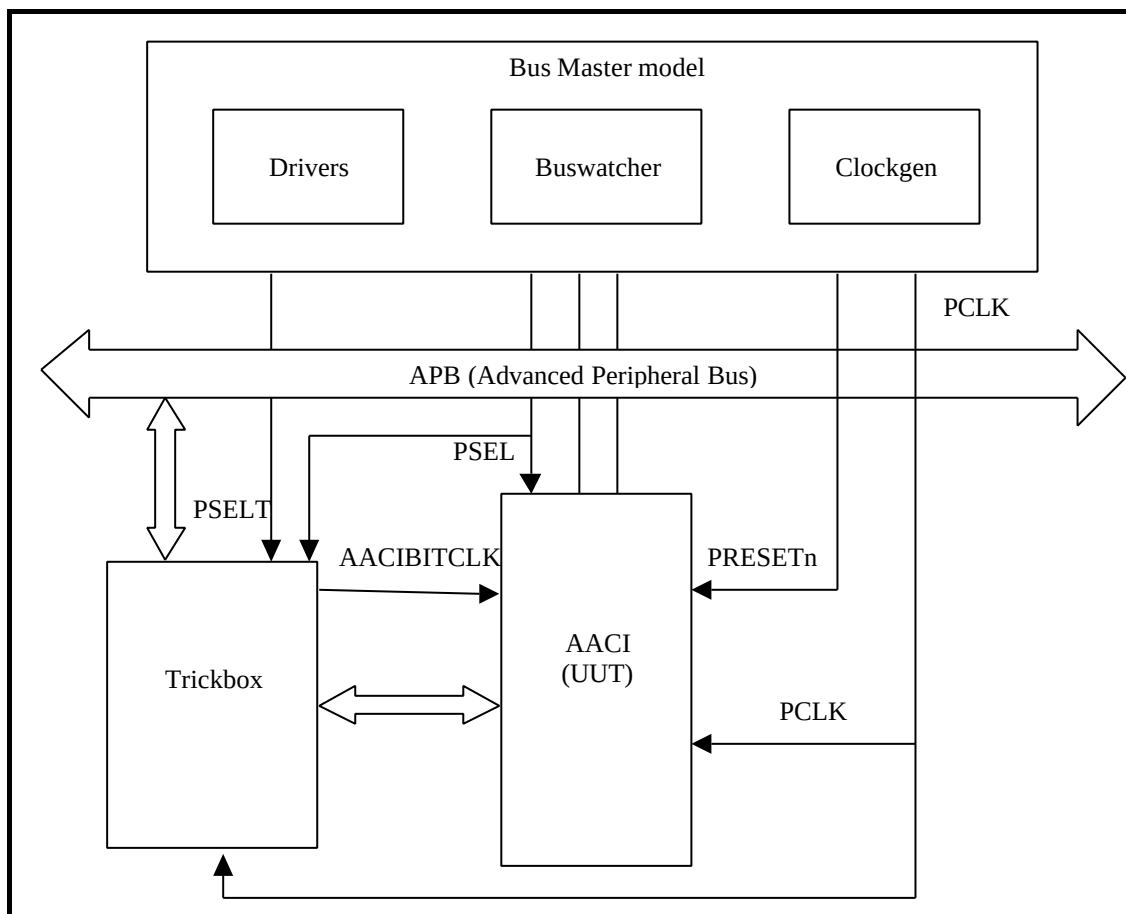


Figure 6.1-1 VHDL testbench for simulating test vectors

Figure 6.1-2 illustrates the hierarchy for the VHDL testbench.

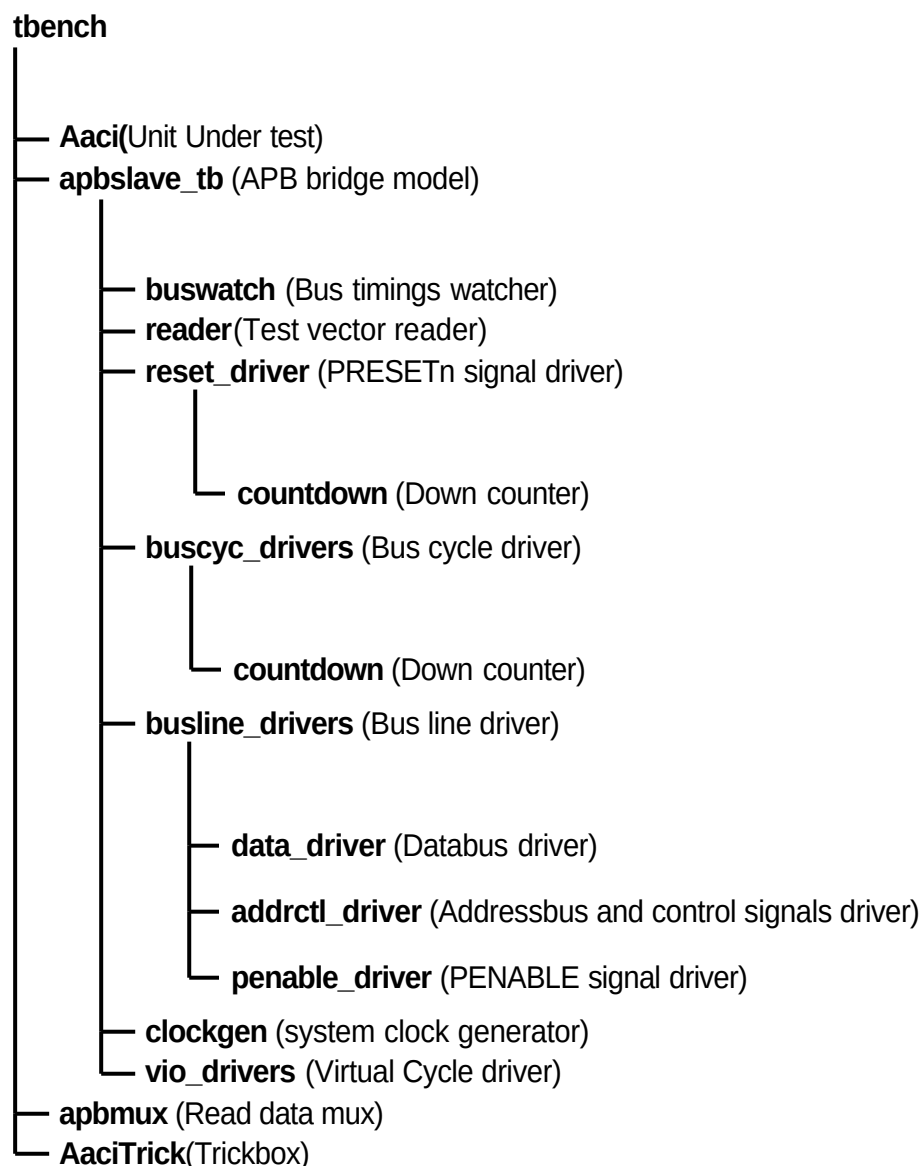


Figure 6.1-2 VHDL Testbench hierarchy

The testbench, **tbench.vhd** is used for functional simulations of the AACI. This module instantiates the AACI (Unit Under Test), the behavioural model of the APB Bridge, the trickbox and the APB Read mux. The AACIBITCLK is generated by the trickbox to facilitate frequency modifications from within the test code.

All the AACIBITCLK domain inputs to the AACI are driven by the trickbox. All the Scan input pins are tied low to keep them from interfering with the test process. All the non-AMBA inputs to the AACI, except the scan data inputs, are driven by the Trickbox.

The default test frequency of PCLK is set at 100MHz. This is the same frequency to which the AACI was constrained during synthesis. PCLK is set to the frequency defined in **tbench.vhd** using the **tcclk** and **tcclk** generic values when instantiating **apbslave_tb.vhd**.

6.1.1 Unit Under Test

The Unit Under Test may be one of the following

- AACI VHDL RTL
- AACI VHDL netlist from VHDL RTL Source

One out of these two versions may be referenced via the corresponding link to the uut directory in ***verification/vhdl***.

6.1.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This module issues commands on the APB bus under program control.

The APB bridge model, `apbslave_tb.vhd`, instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader module reads the test vectors from the `infile.bif` file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the `PRESETn` signal with the correct timings. The `buscyc_drivers` instantiates an APB cycle driver for each cycle type, i.e PSW, PSR, PI, PO, etc. The `busline_drivers` drives the databus, the address and control signals with the correct timings. The buswatch module checks the Read/Write Databus timings. The clockgen generates the system clock, `PCLK`.

The APB bridge model also provides the Select line for the trickbox.

Any access to the address range

0x60000000 to 0x600000FF

will access the TrickBox via the `PSELT` select line.

Accesses to any address outside the above range will select the UUT via the `PSEL` select line.

While the default behaviour of the bridge in the testbench is like a Rev E device, there is an option to force the bridge to issue commands with Rev D timings.

6.1.3 Trickbox

The functionality of the AACI is verified via a trickbox. The trickbox is an APB Slave, which mimics the behaviour of an Audio CODEC. Like the AACI, the trickbox also contains receive and transmit FIFOs. The trickbox has a serial to parallel convertor to line up data received serially from the AACI and it has a parallel to serial convertor to transmit the data stored in its FIFO. The trickbox also flags off any violations in the AC-Link protocol. The trickbox communicates with the APB through the APB-Interface module. All the registers in the trickbox are maintained in a separate register module.

6.1.4 Protocol Checker

The testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the `timings.vhd` file in the ***verification/vhdl/tbench*** directory. The buswatch module, which is in the ***verification/vhdl/buswatcher*** directory checks the Read/Write Databus timings and reports any violations in the timings.

6.1.5 APB Multiplexer

The read data buses from the AACI and the trickbox are muxed together in this module before being connected to the APB Bridge model.

6.1.6 Test Vectors

The test vectors for the verification of the AACI are coded into separate C files depending on the logical region of the AACI, which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

6.1.7 Generics

The AACI testbench provides some configurability options using generics and constants. These generics and constants are configurable through the `tbench.vhd` file.

XonPSEL

XonPSEL is used in the `tbench.vhd` file that resides in *verification/vhdl/tbench*.

This generic is used to eliminate the 'X's that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If **XonPSEL** is set to **0** (default) in the top level of the testbench, these signals will go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in the `data_driver.vhd`, the `reset_driver.vhd` and the `buscyc_drivers.vhd` which reside in *verification/vhdl/bid*.

This generic is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

HaltOnMismatch

This is used in the `data_driver.vhd`, which resides in *verification/vhdl/bid*.

This generic is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to **0** (default – `tbench.vhd`), the error is flagged, but the simulation continues.

SuppressOnReset

SuppressOnReset is used in the `buswatch.vhd`, which resides in *verification/vhdl/buswatcher*.

This generic is used to suppress the checking of the PRDATA timings when the PRESETn signal is asserted. If **SuppressOnReset** is TRUE, the PRDATA set-up and hold timing violations are not flagged when PRESETn is asserted. If **SuppressOnReset** is set to FALSE, PRDATA set-up and hold timing violations are flagged irrespective of PRESETn.

Mesg_enb

Mesg_enb is used in the `buswatch.vhd`, which resides in *verification/vhdl/buswatcher*.

This generic is used to enable or disable the flagging of error messages when PRDATA does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this generic is set to FALSE. If **Mesg_enb** is set to TRUE, the error messages are flagged whenever PRDATA goes to a non-zero value when the UUT is not selected. This generic will have to be set to FALSE in testbenches, which include more than one slave.

Data_check

Data_check is used in the `apbmux.vhd`, which resides in *verification/vhdl/tbench*.

This constant is used to enable or disable the flagging of error messages when the PRDATA driven by a slave has a non-zero value when it is not selected. When **Data_check** has a value **1**, these error messages are flagged.

When it has a value **0**, the error messages are not flagged. This constant is used in testbenches, which include more than one slave.

6.1.8 Running Simulations

The user's guide gives step by step instructions for running simulations using the AACI test vectors on the VHDL source code and the VHDL netlist.

Run time logs for the RTL simulations are:

-verification/vhdl/Aaci_rtl/Aaci_rtl_modelsim.log

Run time logs for the netlist simulations are:

-verification/vhdl/Aaci_scaninsert_vhdl_net_max/Aaci_scaninsert_vhdl_net_max_modelsim.log

The code coverage results are:

-verification/vhdl/Aaci_rtl/vnavigator.summary

6.2 AACI Verilog Verification Testbench

The ARM PrimeCell AACI Verilog test world uses a variant of a top level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The test bench is designed to be used with most common Verilog simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The test bench is "programmable", meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the "generic" property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

Figure 6.2-1 illustrates the BusTalk Verilog testbench for simulating test vectors.

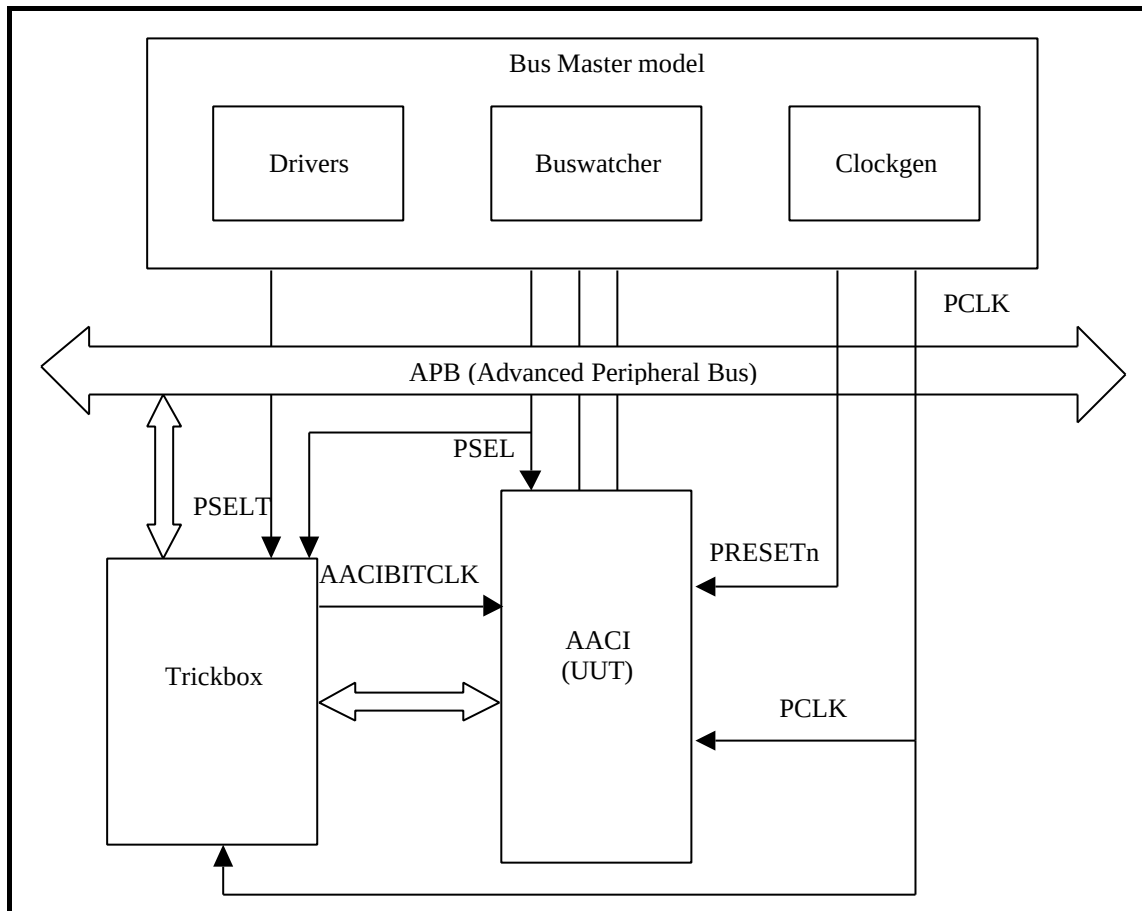


Figure 6.2-1 Verilog Testbench for simulating test vectors

Figure 6.2-2 illustrates the hierarchy for the Verilog testbench.

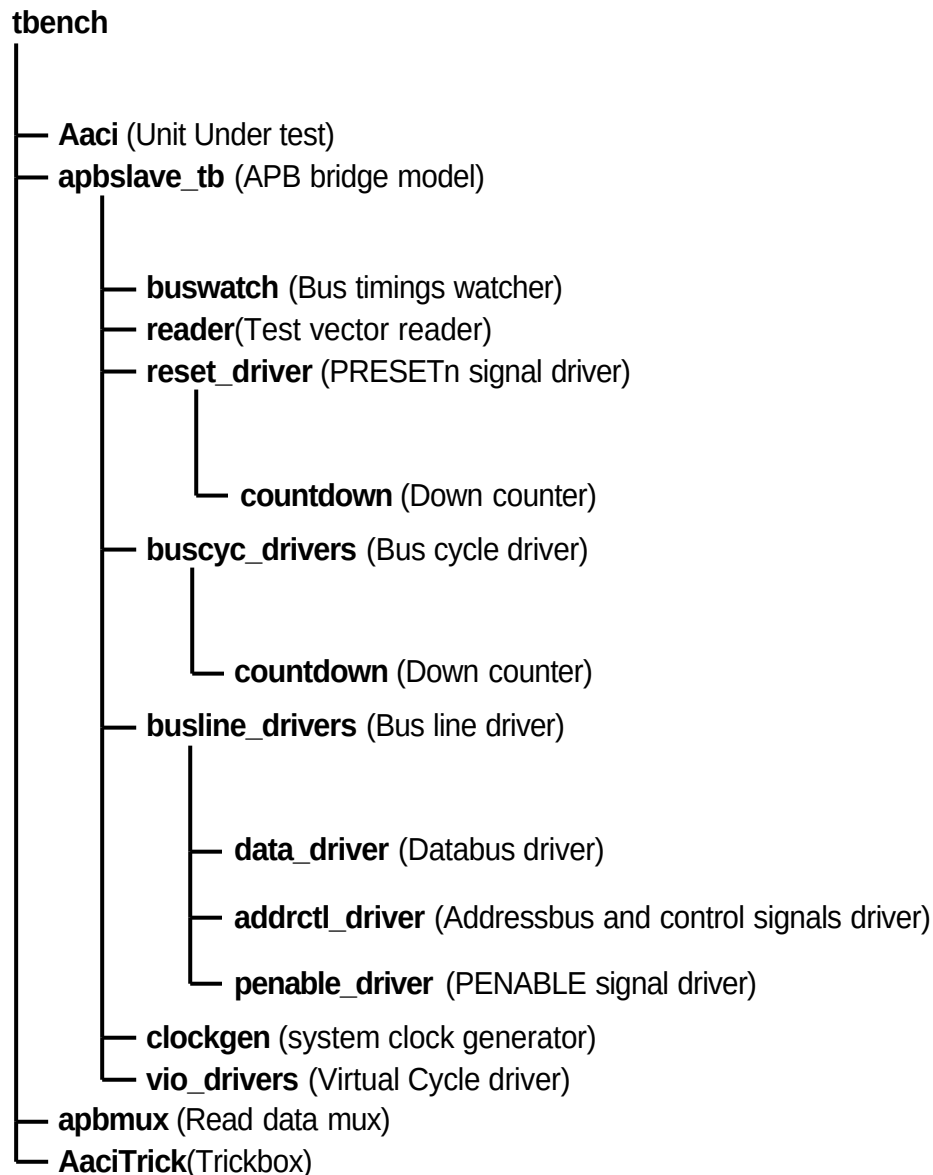


Figure 6.2-2 Verilog Testbench hierarchy

The testbench, **tbench.v** is used for functional simulations of the AACI. This module instantiates the AACI (Unit Under Test), the behavioural model of the APB Bridge, the trickbox and the APB Read mux. The AACIBITCLK is generated by the trickbox to facilitate frequency modifications from within the test code.

All the AACIBITCLK domain inputs to the AACI are driven by the trickbox. All the scan input pins are tied low to keep them from interfering with the test process. All the non-AMBA inputs to the AACI, except the scan data inputs, are driven by the Trickbox.

The default test frequency of PCLK is set at 100MHz. This is the same frequency to which the AACI was constrained during synthesis. PCLK frequency is defined in **timing.v** using the **Tclkh** and **Tckl** 'define values when instantiating **apbslave_tb.v** in **tbench.v**. Note that the 'define values override the local parameters defined in **clockgen.v** in the **verification/verilog/common** directory.

6.2.1 Unit Under Test

The Unit Under Test may be one of the following

- AACI Verilog RTL
- AACI Verilog netlist from Verilog Source
- AACI Verilog netlist from VHDL Source

One out of these three versions may be referenced via the corresponding links to uut directory in **verification/verilog** directory.

6.2.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This block issues commands on the APB bus under program control.

The APB bridge model, `apbslave_tb.v` instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader block reads the test vectors from the `bif.sim` file (from the **verification/bustest/invec** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the `PRESETn` signal with the correct timings. The `buscyc_drivers` instantiates an APB cycle driver for each cycle type, i.e PSW, PSR, PI, PO, etc. The `busline_drivers` drives the databus, the address and control signals with the correct timings. The `buswatch` block checks the Read/Write data bus timings. The `clockgen` generates the system clock, `PCLK`.

The APB bridge model also provides the Select line for the Trickbox.

Any access to the address range

0x60000000 to 0x600000FF

will access the Trickbox via the `PSELT` select line.

Accesses to any address outside the above range will select the UUT via the `PSEL` select line.

6.2.3 Trickbox

The functionality of the AACI is verified via a Trickbox model. The trickbox is an APB Slave, which mimics the behaviour of an Audio CODEC. Like the AACI, the trickbox also contains receive and transmit FIFOs. The trickbox has a serial to parallel convertor to line up data received serially from the AACI and it has a parallel to serial convertor to transmit the data stored in its FIFO. The trickbox has a checker module which flags off any violations in the AC-Link protocol. The trickbox communicates with the APB through the APB-Interface module. All the registers in the trickbox are maintained in a separate register module.

6.2.4 Protocol Checker

The testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the `timings.v` file in the **verification/verilog/tbench** directory. The `buswatch` block, which is in the **verification/verilog/buswatcher** directory checks the Read/Write data bus timings and reports any violations in the `timings`.

6.2.5 APB Multiplexer

The read data buses from the AACI and the Trickbox are multiplexed together in this block before being connected to the APB Bridge model.

6.2.6 Vector Sets

The test vectors for the verification of the AACI are coded into separate C files depending on the logical region of the AACI, which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

6.2.7 Parameters

The AACI testbench provides some configurability options via parameters and defines. All the parameters, except `MESG_ENB` and `SUPPRESSONRESET` are configurable through the `tbench.v` file. `MESG_ENB` and `SUPPRESSONRESET` are configurable through the `buswatch.v` that resides in ***verification/verilog/buswatcher***. The define, `DATA_CHECK` is configurable through the `apbmux.v` that resides in ***verification/verilog/tbench***.

XonPSEL

XonPSEL is used in the `tbench.v` file that resides in ***verification/verilog/tbench***.

This define is used to eliminate the 'X's that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If **XonPSEL** is set to **0** in the top level of the testbench, these signals will go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in the `data_driver.v`, the `reset_driver.v` and the `buscyc_drivers.v` which reside in ***verification/verilog/bid***.

This parameter is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

HaltOnMismatch

This is used in the `data_driver.v`, which resides in ***verification/verilog/bid***.

This parameter is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

SUPPRESSONRESET

SUPPRESSONRESET is used in the `buswatch.v`, which resides in ***verification/verilog/buswatcher***.

This parameter is used to suppress the checking of the PRDATA timings when PRESETn signal is asserted. If **SUPPRESSONRESET** is **1**, the PRDATA set-up and hold timing violations are not flagged when PRESETn is asserted. If **SUPPRESSONRESET** is set to **0**, PRDATA set-up and hold timing violations are flagged irrespective of PRESETn.

MESG_ENB

MESG_ENB is used in the `buswatch.v`, which resides in ***verification/verilog/buswatcher***.

This parameter is used to enable or disable the flagging of error messages when PRDATA does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this parameter is set to **0**. If **MESG_ENB** is set to **1**, the error messages are flagged whenever PRDATA goes to a non-zero value when the UUT is not selected. This generic will have to be set to **0** in testbenches, which include more than one slave.

DATA_CHECK

DATA_CHECK is used in the `apbmux.v`, which resides in ***verification/verilog/tbench***.

This define is used to enable or disable the flagging of error messages when the PRDATA driven by a slave has a non-zero value when it is not selected. When **DATA_CHECK** has a value **1**, these error messages are flagged.

When it has a value **0**, the error messages are not flagged. This define is used in testbenches which include more than one slave.

6.2.8 Running Simulations

The user's guide gives step by step instructions for running simulations using the AACI test vectors on the Verilog source code and the Verilog netlist.

Run time logs for the RTL simulations are:

-verification/verilog/Aaci_rtl/Aaci_rtl_modelsim.log

-verification/verilog/Aaci_rtl/Aaci_rtl_LDV.sim

Run time logs for the netlist simulations are:

-verification/verilog/Aaci_noscan_vhdl_net_min/Aaci_noscan_vhdl_net_min_modelsim.log

-verification/verilog/Aaci_scanready_verilog_net_typ/Aaci_scanready_verilog_net_typ_LDV.log

The code coverage results are:

-verification/verilog/Aaci_rtl/vnavigator.summary

6.3 Trickbox Description

The AACI Trickbox is an APB Slave device used to test the functionality of AACI. It has one APB Slave port and one non-AMBA input port to capture the AC-Link signals, interrupt signals and DMA signals from the AACI.

The Block Schematic of the AACI Trickbox is shown in **Figure 6.3-1**.

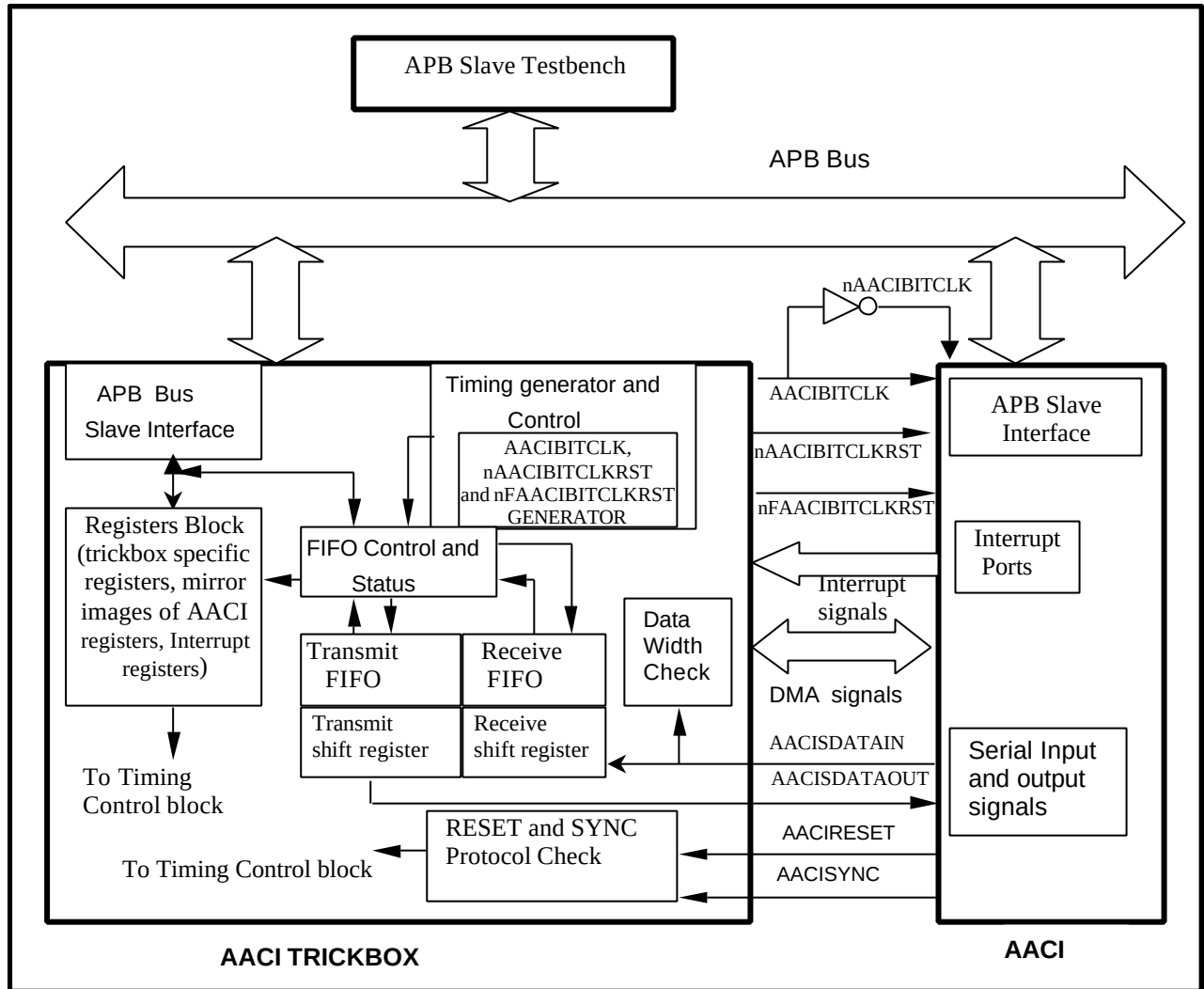


Figure 6.3-1 AACI Testbench and Trickbox block diagram

The AACI Trickbox consists of 8 sub-modules. It performs the following main functions:

- Verification of the AC-Link protocol and timing.
- Setup and hold requirements of the AC link output signals
- Transmission of data to the AACI.
- Reception of data from the AACI.
- Provision to observe the interrupt lines from the AACI.
- Provision to observe the DMA request lines from AACI and to drive DMA request clear lines to AACI.

6.3.1 Trickbox Signal Description

The following tables summarise the AACI Trickbox signal list. **Table 6.3-1** lists the APB Slave interface signals. **Table 6.3-2** lists the AC-Link and other related signals. **Table 6.3-3** lists the interrupt signals from AACI and **Table 6.3-4** lists the DMA signals to/from AACI.

Signal name	Type	Source/Destination	Description
PCLK	IN	APB bus	APB Bus Clock
PRESETn	IN	APB bus	APB bus reset
PSEL	IN	APB bus	AACI Trickbox Select Signal from the APB Slave testbench
PSELCOM	IN	APB bus	AACI Common Select Signal (PSEL of AACI).
PENABLE	IN	APB bus	Transfer enable signal.
PWRITE	IN	APB bus	When HIGH this signal indicates a write transfer and when LOW a read transfer.
PADDR[11:2]	IN	APB bus	APB Address bits used to access the internal trickbox registers.
PWDATA[31:0]	IN	APB bus	APB Write data bus
PRDATA[31:0]	OUT	APB bus	APB Read data bus

Table 6.3-1 APB bus interface signals

Signal name	Type	Source/Destination	Description
AACIRESET	IN	AACI	CODEC Reset signal from the AACI.
AACISYNC	IN	AACI	The serial synchronising signal from AACI.
AACISDATAIN	OUT	AACI	Serial data input line from the AACI.
AACISDATAOUT	IN	AACI	Serial data input line to the AACI.
AACIBITCLK	OUT	AACI	Serial clock for the serial data transmission and reception to/from the AACI.
nAACIBITCLKRST	OUT	AACI	Reset signal for the AACIBITCLK domain of the AACI.
nFAACIBITCLKRST	OUT	AACI	Reset signal for the nAACIBITCLK domain of the AACI.

Table 6.3-2 AC-Link related signals

Signal name	Type	Source/ Destination	Description
AACIINTR	IN	AACI	AACI combined interrupt.
AACIORINTR [1-4]	IN	AACI	Receive Overrun interrupts for channels 1-4.
AACIURINTR [1-4]	IN	AACI	Transmit underrun interrupts for channels 1-4.
AACIRXINTR [1-4]	IN	AACI	Receive FIFO interrupts for channels 1-4.
AACITXINTR [1-4]	IN	AACI	Transmit FIFO interrupts for channels 1–4
AACIRXTOINTR [1-4]	IN	AACI	Receive timeout interrupts for channels 1-4.
AACITXCINTR [1-4]	IN	AACI	Transmit complete interrupts for channels 1-4.
AACIWINTR	IN	AACI	Wakeup Interrupt
AACIGPIOINTR	IN	AACI	GPIO interrupt
AACIS1RXINTR	IN	AACI	Slot1 Receive Interrupt
AACIS2RXINTR	IN	AACI	Slot2 Receive Interrupt
AACIS12RXINTR	IN	AACI	Slot12 Receive Interrupt
AACIS1TXINTR	IN	AACI	Slot1 transmit Interrupt

AACIS2TXINTR	IN	AACI	Slot2 transmit Interrupt
AACIS12TXINTR	IN	AACI	Slot12 transmit Interrupt
AACIRXTOFEINTR [1-4]	IN	AACI	Rx time out FIFO empty interrupts for channels 1-4.

Table 6.3-3 Interrupt signals

Signal name	Type	Source/Destination	Description
AACIDMASREQRX	IN	AACI	DMA single transfer request for receive FIFO1
AACIDMALSREQRX	IN	AACI	DMA last single transfer request for receive FIFO1
AACIDMABREQRX	IN	AACI	DMA burst transfer request for receive FIFO1
AACIDMALBREQRX	IN	AACI	DMA last burst transfer request for receive FIFO1
AACIDMABREQTX	IN	AACI	DMA burst transfer request for transmit FIFO1
AACIDMACLR RX	OUT	AACI	DMA receive request clear
AACIDMACLR TX	OUT	AACI	DMA transmit request clear

Table 6.3-4 DMA input and output signals

6.3.2 Trickbox Register Description

Register Memory Map

Address	Register Name	Type	Description
TrickboxBase	AACITrCntrlReg	Write-only	AACI trickbox Test Control Register.
TrickboxBase	AACITrIntr1Reg	Read-only	AACI trickbox Interrupt Status Register 1.
TrickboxBase + 0x00000004	AACITrBtclkPrd	Write-only	AACI trickbox AACIBITCLK period in ns.
TrickboxBase + 0x00000008	AACITrDMAReg	Read-Write	The DMA ports are to be written to and read from this register.
TrickboxBase + 0x0000000C	AACITrFIFOStat	Read-only	AACI trickbox transmit and receive FIFO status.
TrickboxBase + 0x00000010	AACITrRx FIFO	Read-only	AACI trickbox receive FIFO.
TrickboxBase + 0x00000014	AACITrTx FIFO	Write-only	AACI trickbox transmit FIFO.
TrickboxBase + 0x00000018	AACITrIntr2Reg	Read-only	AACI trickbox Interrupt Status Register 1.
TrickboxBase + 0x0000001C	AACITrClkReg	Read-write	AACI trickbox AACIBITCLK muxing register
AACIBase + 0x0000007C	AACITrRESET	Write-only	AACIRESET (AACIRESET control) register mirror image.
AACIBase + 0x00000080	AACITrSYNC	Write-only	AACISYNC (AACISYNC port control) register mirror image.

Table 6.3-5 AACI Trickbox Memory Map

The AACI Trickbox has 11 registers. Reads from the trickbox base address result in reads from the AACITrIntr1Reg register and return the status of some of the AACI interrupts. Write to this address results in a write to the AACITrCntrlReg register. The AACITrCntrlReg register contains control bits to program the trickbox. The status of the other interrupts can be read from the AACITrIntr2Reg register.

A few registers such as AACITrRESET and AACITrSYNC are mirrored versions of the registers in the AACI. Data written to these registers in the AACI is captured by the trickbox when these registers are written. No extra write to the Trickbox is required for programming these registers.

A detailed description of each of the AACI Trickbox registers is given below.

AACI Trickbox Test Control Register (AACITrCntrlReg)

The Trickbox Control register and the Interrupt Status register share the trickbox base address. Writes to the trickbox base address update the lower 7 bits of the AACITrCntrlReg.

Bits	Name	Description
0	AACITrEn	AACI Trickbox Enable. 0 – Trickbox is disabled 1 – Trickbox is enabled
1	AACITrWidChkEn	When a 1 is written to this location, the trickbox checks the width of the AACISDATAOUT line from the AACI.
2	AACITrTxEn	Serial transmission by the AACI trickbox is enabled.
3	AACITrRxEn	Serial Reception by the AACI trickbox is enabled
4	AACITrBtClkEn	AACIBITCLK enabled onto the output port
5	AACITrBtClkRst	Control bit for nAACIBITCLKRST and nFAACIBITCLKRST. 0 – nAACIBITCLKRST and nFAACIBITCLKRST are pulled low. 1 – nAACIBITCLKRST and nFAACIBITCLKRST are pulled high.
6	AACITrWintGen	Trickbox AACISDATAOUT control in the absence of AACIBITCLK.
31:7	---	Reserved

Table 6.3-6 AACI Trickbox Test Control Register Bit fields

AACI Trickbox Interrupt Status Register (AACITrIntr1Reg)

This is a read only register located at the trickbox base address. It is used to read the status of some of the interrupt

Bits	Name	Description
0	AACIS1RXINTR	Slot1 receive interrupt
1	AACIS1TXINTR	Slot1 transmit interrupt
2	AACIS2RXINTR	Slot2 receive interrupt
3	AACIS2TXINTR	Slot2 transmit interrupt
4	AACIS12RXINTR	Slot12 receive interrupt
5	AACIS12TXINTR	Slot12 transmit interrupt
6	AACIGPIOINTR	GPIO (Slot12 Rx changed) interrupt
7	AACIWINTR	Wake up interrupt
8	AACITXCINTR1	Transmit complete interrupt for FIFO1
9	AACIRXTOINTR1	Receive timeout interrupt for FIFO1
10	AACITXINTR1	Transmit interrupt for FIFO1
11	AACIRXINTR1	Receive interrupt for FIFO1

12	AACIORINTR1	Receive overrun interrupt for FIFO1
13	AACIURINTR1	Transmit under run interrupt for FIFO1
14	AACITXCINTR2	Transmit complete interrupt for FIFO2
15	AACIRXTOINTR2	Receive timeout interrupt for FIFO2
16	AACITXINTR2	Transmit interrupt for FIFO2
17	AACIRXINTR2	Receive interrupt for FIFO2
18	AACIORINTR2	Receive overrun interrupt for FIFO2
19	AACIURINTR2	Transmit under run interrupt for FIFO2
20	AACITXCINTR3	Transmit complete interrupt for FIFO3
21	AACIRXTOINTR3	Receive timeout interrupt for FIFO3
22	AACITXINTR3	Transmit interrupt for FIFO3
23	AACIRXINTR3	Receive interrupt for FIFO3
24	AACIORINTR3	Receive overrun interrupt for FIFO3
25	AACIURINTR3	Transmit under run interrupt for FIFO3
26	AACITXCINTR4	Transmit complete interrupt for FIFO4
27	AACIRXTOINTR4	Receive timeout interrupt for FIFO4
28	AACITXINTR4	Transmit interrupt for FIFO4
29	AACIRXINTR4	Receive interrupt for FIFO4
30	AACIORINTR4	Receive overrun interrupt for FIFO4
31	AACIURINTR4	Transmit under run interrupt for FIFO4

Table 6.3-7 AACI Interrupt status Register bit fields

AACI Trickbox AACIBITCLK period Register (AACITrBtClkPrd)

This is a read / write register. This register controls the frequency of the AACIBITCLK signal driven to the AACI. This indicates the AACIBITCLK period in ns.

Bits	Name	Description
15:0	AACITrBtClkPrd	AACIBITCLK period in ns.
31:16	--	Reserved

Table 6.3-8 AACI Trickbox AACIBITCLK period register

AACI Trickbox DMA Register (AACITrDMAReg)

This is a read / write register. This register is used to monitor the DMA output signals from the AACI and to drive the DMA-related input signals to the AACI.

Bits	Name	Description
0	AACIDMASREQRX	DMA receive request for single transfer
1	AACIDMASLREQRX	DMA receive request for last single transfer
2	AACIDMABREQRX	DMA receive request for burst transfer
3	AACIDMABLREQRX	DMA receive request for last burst transfer
4	AACIDMABREQTX	DMA transmit request for a transfer
5	AACIDMACLRXX	Clear the DMA receive request

6	AACIDMACLR_TX	Clear the DMA transmit request
31:7	-	Reserved

Table 6.3-9 AACI Trickbox DMA Register**AACI Trickbox FIFO status Register (AACITrFIFOStat)**

This is a read-only register. This register indicates the internal status of the AACI trickbox FIFO fill level.

Bits	Name	Description
15:0	RxFFillLevel	Receive FIFO fill level
31:16	TxFFillLevel	Transmit FIFO fill level

Table 6.3-10 AACI Trickbox FIFO Status register**AACI Trickbox Receive FIFO (AACITrRxFIFO)**

This is a read only register. The data coming through AACISDATAOUT of the AACI is read through this FIFO.

Bits	Name	Description
19:0	RxData	20 bit Receive data. Available in left-justified format with respect to the programmed RSize value.
31:20	--	Reserved

Table 6.3-11 AACI Trickbox Receive FIFO**AACI Trickbox Transmit FIFO (AACITrTxFIFO)**

This is a write only register. Data to be sent on the AACISDATAIN line of the AACI is to be written into this FIFO.

Bits	Name	Description
19:0	TxData	20 bit Transmit data. To be written left-justified with respect to the programmed TSize value.
31:20	--	Reserved

Table 6.3-12 AACI Trickbox Transmit FIFO**AACI Trickbox Interrupt Status Register (AACITrIntr2Reg)**

This is a read-only register. This register is used to read the values of the AACIINTR and AACIRXTOFEINTR1-4 interrupt outputs of the AACI

Bits	Name	Description
0	AACIINTR	AACI combined interrupt
1	AACIRXTOFEINTR1	AACI Rx Timeout FIFO empty interrupt for channel 1
2	AACIRXTOFEINTR2	AACI Rx Timeout FIFO empty interrupt for channel 2
3	AACIRXTOFEINTR3	AACI Rx Timeout FIFO empty interrupt for channel 3
4	AACIRXTOFEINTR4	AACI Rx Timeout FIFO empty interrupt for channel 4
31:5	----	Reserved

Table 6.3-13 AACI trickbox Interrupt register2

AACI Trickbox AACIBITCLK Muxing Control Register (AACITrClkReg)

Bits	Name	Description
0	AACITrClkSel	Clock select. 0 – Internal AACIBITCLK routed to AACIBITCLK output port 1 – PCLK routed to AACIBITCLK output port
1	PCLKEn	Trickbox PCLK enable bit 0 – Drive internal PCLK line Low (Default) 1 – Enable internal PCLK line
2	BitClkEn	Trickbox AACIBITCLK enable bit 0 – Drive internal AACIBITCLK line Low (Default) 1 – Enable internal AACIBITCLK line
3	PCLKOn	Read-Only. 0 – Internal PCLK has been gated low. 1 – Internal PCLK is enabled.
4	BITCLKOn	Read-Only. 0 – Internal AACIBITCLK has been gated low. 1 – Internal AACIBITCLK is enabled
31:6	----	Reserved

Table 6.3-14 AACI Clock control register**AACITrRESET**

The AACITrRESET register is a write only register that is used to check the protocol of the AACIRESET port of AACI. This register is a mirrored version of the AACIRESET register of the AACI.

Bits	Name	Type	Function
0	FORCEDRESET	Write	The AACIRESET output port from the AACI is expected to follow the value written to this bit.
31:1	Reserved	--	Reserved

Table 6.3-15 AACI Trickbox RESET register**AACITrSYNC**

The AACITrSYNC register is a write only register that is used to check the protocol of the AACISYNC port of AACI. The FORCEDSYNC bit default value at reset is 0. This register is a mirrored version of the AACISYNC register of the AACI.

Bits	Name	Type	Function
0	FORCEDSYNC	Write	The AACISYNC output port from the AACI is expected to be high when this bit is set.
31:1	---	--	Reserved

Table 6.3-16 AACI SYNC register

6.3.3 Trickbox Functional Description

The AACI Trickbox has 9 sub-modules parts:

- APB Bus Interface
- Register block
- AACIBITCLK clock, and nAACIBITCLKRST signal generator
- AACISYNC and AACIRESET and Data Width Protocol Check
- AACI Trickbox Receive FIFO and control logic
- AACI Trickbox Transmit FIFO and control logic
- AACI Trickbox Main control logic
- Synchronising block for signals from AACIBITCLK to PCLK
- Synchronising block for signals from PCLK to AACIBITCLK

A brief description of each block of the trickbox is given in the following sections:

APB Bus Interface

Reading and writing of data from the internal registers and the Tx and Rx FIFO is done via the APB interface.

The trickbox has 2 PSELS - one is the PSELCOM, which is the same as the PSEL to the AACI and the other is the trickbox-specific PSEL.

The PSEL is used for writing into the trickbox-specific registers and the trickbox transmit and receive FIFOs. The PSELCOM (PSEL of the AACI) is used for writing into registers that are mirror images of the AACI registers (AACIRESET, AACISYNC).

Register block

All registers except the FIFOs are implemented in this module. The register contents are routed to the other sections of the trickbox.

AACIBITCLK, nAACIBITCLKRST and nFAACIBITCLKRST signal generator

The clock generator section drives the AACIBITCLK signal to the AACI if the AaciTrBtClkEn bit in the control register is set. This module also generates the nAACIBITCLKRST and nFAACIBITCLKRST signals to the AACI.

AACISYNC, AACIRESET and Data Width Protocol Check

- The AACIRESET port is monitored to verify that it follows the FORCEDRESET bit in the AACITrRESET register.
- The AACISYNC port is monitored to verify that it is high when the FORCEDSYNC bit in the AACITrSYNC register is set.
- AACISDATAOUT data width is checked in this block. The data pattern written to the AACI transmit FIFO is 1010... The AACITrWidChkEn bit in the control register of the trickbox has to be set. In normal mode this bit has to be cleared. The data width is checked for half the period of AACIBITCLK with some allowable tolerance.

AACI Trickbox Transmit FIFO and FIFO control

This block contains the Transmit FIFO registers and the associated control logic. The Transmit FIFO is implemented as a circular buffer and has programable depth (number of entries) upto maximum of 32K. The FIFO depth is programmed in the AaciTrPackage file by changing the FIFODEPTH parameter. The POINTERWIDTH should also be accordingly defined.

Based on the values of the Read and Write pointers, the fill level of the Transmit FIFO is calculated. The fill level is readable through the Transmit FIFO status register. The number of valid bits for the fill level in this register are

dependant on the programmed FIFO depth. For example if the FIFO depth is programmed for depth = 64 then the maximum fill level can be 64, requiring 7 bits of information for the filllevel. For this lower 7 bits out of 16 bits assigned in the status register would be used to check the filllevel.

Reads from the transmit FIFO are done through the RdPtrInc signal from the Main Control module and writes are done through the TxFIFOWr signal from the APB interface.

AACI Trickbox Receive FIFO and FIFO control

This block contains the Receive FIFO registers and the associated control logic. The Receive FIFO is implemented as a circular buffer and the depth(number of entries) of the FIFO is programmable upto a maximum depth of 32K. The FIFO depth is programmed in the AaciTrPackage file by changing the FIFODEPTH parameter. The POINTERWIDTH should also be accordingly defined.

Based on the values of the read and write pointers, the fill level of the receive FIFO is calculated. The fill level is readable through the Receive FIFO status register. The number of valid bits for the filllevel out of 16 bits in the status register is dependant on the FIFO depth programmed. For example if the FIFO depth is 64 entries then the fill level can be maximum of 64, requiring 7 bits of information. For this the lower 7 bits out of assigned 16 bits in the status register will be used.

Reads from the receive FIFO are done through the RxFRdPtrInc signal from the APB interface and writes into the receive FIFO are done through the RxFWr signal from the Main Control module.

AACI Trickbox Main control

This block contains the main transmission and reception control state machine. It also checks for the AACISYNC protocol during normal transmission and reception. In normal mode the width of the AACISYNC pulse is checked for 16 AACIBITCLK periods and the LOW period of AACISYNC is checked for 240 AACIBITCLK periods.

This state machine keeps track of the slot information on the AC-Link interface depending on the AACISYNC signal driven by AACI. For every slot, the data from the transmit FIFO is loaded in to the transmit shift register to drive it onto the AACISDATAIN line of the AACI. The bit stream on the AACISDATAOUT line from the AACI is shifted into the receive shift register and at the end of every slot the data in the shift register is copied to the temporary buffer and then the contents of the buffer are transferred into the Receive FIFO.

PCLK to AACIBITCLK Synchronisers

The signals crossing over from the PCLK domain to the AACIBITCLK domain are double-synchronised to AACIBITCLK in this block.

AACIBITCLK to PCLK Synchronisers

The signals crossing over from the AACIBITCLK domain to the PCLK domain are double-synchronised to PCLK in this block.

6.4 Functional Tests

The following sections describe the functional test cases to validate the AACI.

6.4.1 Register Tests

The function, RegisterTest() is used to test the AACI registers.

Reset Value Test

PRESETn is asserted and then de-asserted. All the PCLK domain registers are checked for their reset values after the reset. The registers should reflect their reset values as defined in the specifications.

The Register Reset values are listed below :

AACISR1-4 - 0x0B
 AACIRESET - 0x01
 AACISLFR - 0xA80

All other registers should reset to zero.

nAACIBITCLKRST and nFAACIBITCLKRST are asserted and then de-asserted. All the registers are then read back to verify the reset values.

AACIBITCLK from the trickbox is then enabled and then nAACIBITCLKRST and nFAACIBITCLKRST are asserted and then de-asserted. The registers are then read back to verify the reset values.

Read/Write-ability of the functional registers

Read-only registers are checked by writing a value other than their reset values and reading back to check whether they return the reset values.

Read-write registers are checked by writing data patterns of 0x00000000, 0xAAAAAAAA, 0x55555555 and 0xFFFFFFFF and reading back to verify that the data that was written is returned.

All the hardware masked registers are tested in the register test mode by setting the REGTEST bit in the AACITCR register of AACI, for read write test.

The registers with hardware masking are also checked for their hardware masking in the normal mode. In this case the REGTEST bit in the AACITCR register is clear. And 2 successive datas are written to the register on successive APB transactions. Reads from these registers should return the values written during the first write in normal mode. And in the test mode it should return the value written during last write.

6.4.2 AACISYNC protocol test

The generation of the AACISYNC pulse by setting the ForcedSYNC bit of the AACISYNC register is tested on calling the ForcedSync() function.

The AACISYNC register in the AACI is mirrored in the trickbox. When this register is written to in the AACI, the corresponding register in the trickbox is also updated simultaneously. The value on the AACISYNC port is monitored by the trickbox to verify that the AACISYNC port follows the FORCEDSYNC bit in the AACISYNC register.

The AaciTrBtClkEn bit in the trickbox control register is cleared so as to shutdown AACIBITCLK to AACI. Then the FORCEDSYNC bit in the AACISYNC register is written with a value of 1. It is kept set for few PCLK cycles. The protocol checkers in the trickbox verify that the AACISYNC port follows the FORCEDSYNC bit value. Then, the FORCEDSYNC bit is made zero and kept low for a few PCLK cycles. Again, the trickbox verifies that the value on the AACISYNC port follows the FORCEDSYNC bit.

6.4.3 AACIRESET protocol test

The ForcedReset() function tests the generation of the AACIRESET signal by the AACI.

The AACIRESET port of the AACI is forced to '0' and to '1' by writing to the ForcedRESET bit of the AACIRESET register. The trickbox will continuously monitor the AACIRESET port to verify that it follows the value written in the ForcedRESET bit of the AACIRESET register. If the AACIRESET port does not follow this bit then the trickbox flags an error message.

6.4.4 Warm Reset protocol Check

The WarmResetTest() function is used to verify that the AACI performs error-free transmission/reception after the CODEC is brought out of low power mode by warm reset.

The AACI is programmed for normal transmission and reception. Also the trickbox is programmed to enable transmission/reception and to enable AACIBITCLK. During normal transmission and reception, the Trickbox and

all channels in the AACI are monitored to write data into the Transmit FIFOs and to read data from the Receive FIFOs at the required times.

After two frames of data transmission and reception the AACI is programmed through AACISL1TX and AACISL2TX registers to force the CODEC into low power mode. The AaciTrBtClkEn bit in the trickbox is made zero deterministically at the end of transmission of Slot 2 data. This disables the AACIBITCLK to simulate entry into the power down mode. Then, after a few PCLK cycles of idle time, the AACISYNC output port from the AACI is driven high by writing 1 to the ForcedSYNC bit in AACISYNC register. After a few PCLK cycles of idle time, this bit is cleared. Thus, a Warm Reset has been induced through the AACI.

The AaciTrBtClkEn bit in the trickbox is set in a deterministic manner so as to restart AACIBITCLK. It is then verified that the AACISYNC output from the AACI is asserted after less than 14 AACIBITCLKs. Also, it is verified that the Slot 2 data received when the power-down mode was entered is not written into the Rx FIFO or into the Rx holding registers. The AACI and the Trickbox are programmed for two frames of data transmission / reception to verify error-free operation after the Warm Reset. This test is repeated for different time intervals between the end of Slot 2 and the shutdown of AACIBITCLK by changing the number of idle cycles after the slot2 completion.

6.4.5 Cold Reset protocol check in power down mode

The ColdResetTest() function tests cold reset protocol in the power down mode.

The test sequence is as follows. Normal transmission and reception of data is allowed to occur for 2 frames. The AACI is programmed through AACISL1TX and AACISL2TX registers to force the CODEC into low power mode. Then the AACIBITCLK in the trickbox is disabled so as to simulate low power mode. The AACIRESET port is driven to low value for few cycles through the ForcedRESET bit in the AACIRESET register. AACIBITCLK from the trickbox is then enabled and two frames of transmission / reception are allowed. It is verified that the transmission / reception is error-free.

6.4.6 Interrupt Tests

Overrun Interrupt Test

The Receive Overrun Interrupts are tested using the AACIORINTRTests() function.

The Rx FIFO of the AACI is enabled and the RxOIE bit is enabled in the AACIIE Register. The Tx FIFO in the Trickbox is filled with data for two frames. The AACIRXCR register is programmed for 8 valid slots. The trickbox is allowed to transmit the first frame. The Rx FIFO in the AACI is expected to be filled with 8 words of data. The Rx Overrun interrupt should not be asserted. This is confirmed by reading the AACITrIntr1Reg register in the trickbox and the AACIISR, AACISR and AACIALLINTS registers in the AACI.

The second frame from the trickbox is then allowed to be transmitted. The data written into the Tx FIFO of the trickbox is such that the Slot0 of the received frame indicates valid data for one slot. When the Rx FIFO is written with the 9th data word, the Rx Overrun interrupt should be asserted. This is confirmed by reading the AACITrIntr1Reg register in the trickbox and the AACISR, AACIISR, AACIALLINTS in AACI. The second frame is allowed to complete.

Writing a 1 into the RxOEC bit of the AACIINTCLR register of the AACI should clear the Interrupt. This is confirmed by reading the AACITrIntr1Reg register and the AACISR, AACIISR, AACIALLINTS register bit.

This test is repeated after clearing the RxOIE bit of AACIIE to ensure that this interrupt does not get set when the interrupt is masked. After the interrupting condition has been created, the Interrupt enable bit is set and it is then verified that the interrupt is asserted. This test is done for all the channels.

This test is also repeated with the channels programmed for Character mode reception.

Rx Interrupt Test

The function AACIRXINTRTests() is used to verify the Receive Interrupt generation.

The Rx FIFO of the AACI is enabled and the RxIE bit in the AACIIE Register is enabled. The Tx FIFO in the Trickbox is filled with data for one full frame within which data for only four slots are valid. The AACIRXCR register is programmed to receive four valid slots. Then, the trickbox is allowed to transmit data until the Rx FIFO in the AACI is filled with 4 valid slots. After the Rx FIFO in the AACI is written with the fourth data, the Rx Interrupt should be asserted. This is verified by reading the AACITrIntr1Reg register and the AACIISR and AACIALLINTS registers.

Reading one Data from Rx FIFO of AACI should clear the Interrupt. This is confirmed by reading the AACITrIntr1Reg of the trickbox and the AACIISR, AACIALLINTS registers.

This test is repeated by clearing the RxIE bit in the AACIIE register to verify that this interrupt is not set when masked. This test is repeated for various combinations of valid slot positions and number of valid slots, typically more than four slots. This test is repeated by disabling the Rx FIFO (i.e. in Character mode). But this time, one write into the Rx FIFO will raise the interrupt. This test is repeated in the holding-buffer mode, after clearing the RxIE bit in the AACIIE register to ensure that this interrupt is not set when masked. After the interrupting condition has been created, it is also confirmed that setting the Interrupt enable bit will raise the interrupt. This Test is done for all the channels.

This test is also repeated with the channels programmed for Character mode reception.

Tx Interrupt Test

The Transmit Interrupt functionality is verified on the AACITXINTRTests() function call.

The Tx FIFO of the AACI is enabled and TxIE bit in the AACIIE Register is set. Initially, the Tx FIFO is empty and hence the Tx Interrupt should be asserted. This is confirmed by reading the AACITrIntr1Reg, AACIISR and AACIALLINTS registers. Then the Tx FIFO of the AACI is loaded with four data and the Tx interrupt is checked. The interrupt is expected to remain asserted.

One more data is written to the Tx FIFO of the AACI. This should clear the Interrupt. This is verified by reading the AACITrIntr1Reg of the trickbox and the AACIISR register.

This test is repeated after clearing the TxIE bit in the AACIIE register to ensure that the interrupt is not set when masked. After the interrupting condition is created, the Interrupt enable bit is set and it is verified that the interrupt is asserted. This test is also repeated after disabling the Tx FIFO. This test is repeated for all the channels. This test is also repeated with the channels programmed for Character mode reception.

RxTimeout Interrupt Test

The Receive Timeout Interrupt is verified using the AACIRXTOINTRTests() function.

The Rx FIFO of the AACI is enabled and RxTIE bit in the AACIIE register is set to enable the Receive Timeout interrupt. The TOC bit field of AACIRXCR is written with 4 (timeout after 4 frames). The AACIRXCR1 register is programmed for reception of slots 3, 5, 7, and 12. The Tx FIFO in the trickbox is filled with valid data for 4 slots and with invalid data for the other slots in the first frame. Then the trickbox is allowed to transmit data. After the completion of the first frame, four data words are expected to be present in the Rx FIFO of the AACI. This is verified by polling for the RxHF bit in the AACIISR register. The trickbox is then allowed to transfer four invalid frames to the AACI that do not result in any further data being written to the Rx FIFO of the AACI. When the AACI receives Slot12 of the fourth invalid frame after the last received valid frame, it is expected to raise the RxTimeout Interrupt. Then AACITrIntr1Reg is read to verify that the timeout interrupt has been raised. The AACIISR and AACIALLINTS registers are also checked.

Reading all data from the Rx FIFO of the AACI should clear the RxTimeout Interrupt and also the RXTIME interrupt should not be raised for all non DMA channels (channels 2, 3 and 4 and channel 1 with DMAEnable bit in AACIMAINCR = 0) even if the FIFO is now empty. But for the DMA channel (channel 1 with DMAEnable bit in AACIMAINCR = 1) the RXTIME interrupt will now be set as the FIFO has become empty and timeout condition is still there. This is verified by reading the AACITrIntr1Reg register of the trickbox and the AACIISR, AACIALLINTS registers.

This test is repeated after clearing the RxTIE bit in the AACIIE register to ensure that this interrupt is not set when masked even if the raw interrupt status is set. After the interrupting condition is created, it is also verified that setting the Interrupt enable bit results in the assertion of this interrupt.

Tx Complete Interrupt Test

The function AACITXCINTRTests() is used for testing the Transmit Complete Interrupt.

The Tx FIFO of the AACI is enabled and the TxClE bit in the AACIIE Register is set to enable the Transmit Complete interrupt. The AACITXCR1 register is programmed for transmitting four valid slots. The Tx FIFO 1 of the AACI is filled with 4 data. The AACI is allowed to transmit one full frame. The TxFE and TxBusy bits of the AACISR register are polled to detect the completion of the frame. Then the AACITrIntr1Reg is read to verify that the Transmit Complete interrupt is asserted. Also, the AACIISR and AACIALLINTS register are read to verify the assertion of the interrupt. Data is written to the Tx FIFO to clear this interrupt.

This test is repeated after clearing the TxClE bit in the AACIIE register to ensure that this interrupt is not set when masked. This test is repeated for all the channels. This test is also repeated with the channels programmed for Character mode transmission.

Tx Underrun Interrupt Test

The Transmit Underrun Interrupt generation is verified on calling the AACIURINTRTests() function.

The Tx FIFO of the AACI is enabled and the TxUIE bit in the AACIIE Register is enabled. The AACITXCR1 register is programmed for transmitting four valid slots. The Tx FIFO of the AACI is filled with 7 data – 4 for the first frame and 3 for the second. Data is written into the transmit FIFO of the trickbox such that the SRC bits in Slot1 request four valid slots in the next frame. During the transmission of the second frame the FIFO underrun interrupt request for that channel is expected to be asserted by the AACI because enough data is not available in the Tx FIFO. The assertion of this interrupt is verified by reading the AACITrIntr1Reg from the trickbox and the AACISR, AACIISR and AACIALLINTS registers of the AACI.

Writing a 1 into the TxUEC bit in the AACIINTCLR register clears this interrupt. This is verified by again reading the AACITrIntr1Reg from the trickbox and the AACISR, AACIALLINTS and AACIISR registers of the AACI.

This test is repeated after clearing the TxUIE bit in the AACIIE register to verify that this interrupt is not asserted when masked. After the interrupting condition is created, it is also verified that setting the Interrupt enable bit results in the assertion of this interrupt. This test is repeated for all the channels and for different combinations of valid slots. This test is also repeated with the channels programmed for Character mode reception.

RXTOFE Interrupt Test

The RxTOFE Interrupt functionality is tested on the AACIRXTOFEINTRTests() function call.

RxTOIE bit in the AACIIE register is set to enable the RxTOFE interrupt. The TOC bit field of AACIRXCR is written with value 4 (timeout after 4 frames). The AACIRXCR register is programmed for reception of slots 5, 7, 9, and 10. The Tx FIFO in the trickbox is filled with invalid data for all the slots in the first frame. Then the trickbox is allowed to transmit data. After the completion of the first frame, the Rx FIFO is expected to be empty. This is verified by polling for the RxHF bit in the AACISR register. The trickbox is then allowed to transfer four valid frames with invalid data to the AACI that do not result in any further data being written to the Rx FIFO of the AACI. When the AACI receives Slot12 of the fourth invalid frame after the last received valid frame, it is expected to raise the RXTOFE interrupt. Then the AACITrIntr2Reg register is read to verify that the AACIRXTOFEINTR has been raised. The AACIISR and AACIALLINTS registers are also checked.

This test is repeated after clearing the RxTOIE bit in the AACIIE register to ensure that this interrupt is not set when masked even if the raw interrupt status is set. After the interrupting condition is created, it is also verified that setting the Interrupt enable bit results in the assertion of this interrupt.

Wakeup Interrupt Test

The function AACIWINTRTest() is used to verify the functionality of the Wakeup Interrupt.

The WakeupIE bit in the AACISLIEN register of the AACI is set and the WISE bit in the same register is cleared. The AACITrBtClkEn bit in the Trickbox is disabled to simulate entry into power down mode. Initially, when the LPM bit is cleared, the WintGen bit in the Trickbox is set. It is expected that the Wakeup Interrupt will not be asserted. Then, the LPM bit in the AACMAINCR register is set. Also, the WintGen bit in the Trickbox is set. Then the AACITrIntr1Reg register is read to verify that the Wakeup Interrupt is asserted. Also the RWIS bit in the AACISLFR register and the WIS bit in the AACISLISTAT register are read to verify that these are not set.

Then the WintGen bit in the Trickbox is cleared. This is expected to clear the Wakeup Interrupt. This is confirmed by reading the relevant interrupt status registers. The RWIS bit in the AACISLFR register and the WIS bit in the AACISLISTAT register are read to verify that these are still in the cleared condition.

Then, the WISE bit is set and the interrupting condition is created again. Then the AACITrIntr1Reg register is read to verify that the Wakeup Interrupt is asserted. Now in this condition the Wakeup interrupt in the AACITrIntr1Reg is checked for being set and also the RWIS bit in the AACISLFR register and the WIS bit in the AACISLISTAT register are read to verify that these are set to indicate the occurrence of the interrupt.

Then the WintGen bit in the trickbox control register is then made zero. The wake interrupt and related flags are to be still set as the interrupt is the latched version. Then a value of '1' is written to the WISC bit of the AACIINTCLR Register to clear the Wake up interrupt. The AACITrIntr1Reg, AACISLFR and AACISLISTAT registers are read to verify that the interrupt is cleared. This test is repeated after disabling the WakeupIE bit in the AACISLIEN register to verify that the interrupt is not asserted when masked.

GPIO Interrupt Test

The GPIO Interrupt functionality is verified by calling the function AACIGPIOINTRTest().

The GpioIE bit in the AACISLIEN register is set. The AACISL12RX register is enabled via the AACIMAINCR register. The Tx FIFO in the Trickbox is loaded with data for two frames. The data written is such that the bit 0 of the slot 12 data for the first frame is zero and bit-0 of slot 12 for the next frame is 1. The AACITrFIFOStat register is polled to verify that the first frame is transmitted. The AACITrIntr1Reg is read to verify that the AACIGPIOINTR interrupt is not raised. The AACISLFR register and the AACISLISTAT and the AACIALLINTS registers are also checked.

Then the second frame is allowed to be transmitted. After the transmission of the second frame, the AACIGPIOINTR interrupt is expected to be asserted. This is verified by reading the AACITrIntrReg register. The AACISLFR register, the AACISLISTAT register and the AACIALLINTS registers are also checked to verify that the AACIGPIOINTR interrupt is asserted.

Data is read from the AACISL12RX register and it is expected that the interrupt be cleared. This is verified by first reading data from the AACISL12RX register and then reading the AACITrFIFOStat, AACISLFR, AACISLISTAT and AACIALLINTS registers for confirming that the interrupt is cleared.

This test is repeated with another frame being allowed to be received by the AACI before the data already stored in the AACISL12RX register is read out. When the second data received is about to be written to the AACISL12RX register, a read is deterministically initiated to read out the first received data. In this case, the interrupt is expected to remain set to indicate the presence of new data.

The test is repeated by clearing the GpioIE bit in the AACISLIEN Register to verify that the interrupt is not asserted when masked. After the interrupting condition is created, it is also verified that setting the Interrupt enable bit results in the assertion of this interrupt.

AACISnRXINTR Test [n = 1,2 or 12]

The AACISnRXINTRTests() (n = 1, 2 or 12) functions are used to verify the functionality of the Slot Rx interrupts.

The SlotnRxIE [n = 1,2 or 12] bit is enabled in the AACISLIEN Register. The AACI is programmed such that none of the AACIRXCR1-4 registers have the slot n valid bits [n = 1,2 or 12] set. The AACISLnRX [n = 1,2 or 12] register is enabled by writing to the AACIMAINCR register. The Tx FIFO in the Trickbox is loaded with data for one frame within which slot3, slot 6, slot8, and slot n, [where n is the slot number (either of 1,2 and 12) for which this test is been called] are valid slots. After the reception of the new slot data, the AACITrIntr1Reg is tested to

verify that the AACISnRXINTR [n = 1,2 or 12] interrupt is raised. Also, the AACISLFR, AACISLISTAT and AACIALLINTS registers are checked to verify the interrupt status. Reading the data in the AACISLnRX [n = 1,2 or 12] register clears this interrupt. This is confirmed by reading the Interrupt status registers.

This test is repeated for 2 frames of reception. Data received for the slot n [n = 1,2 or 12] register in the first frame is not read until the next data for that register in the next frame is about to be written.

When the second data received is about to be written to the AACISLnRX [n = 1,2 or 12] register, a read is deterministically initiated to read out the first received data. In this case, the interrupt is expected to remain set to indicate the presence of new data. This test is repeated after disabling the SlotnRxIE [n = 1,2 or 12] bit in the AACISLIEN register. The interrupt is not expected to get set when it is masked. Also, it is confirmed that setting interrupt enable bit results in the assertion of this interrupt.

AACISnTXINTR Test [n = 1, 2, or 12]

The AACISnTXINTRTests() (n = 1, 2 or 12) functions are used to verify the functionality of the Slot Tx interrupts.

The SlotnTxIE [n = 1,2 or 12] bit is enabled in the AACISLIEN register. The AACI is programmed such that none of the AACITXCR1-4 registers have the slot n [n = 1,2 or 12] valid bits set. The AACISLnTX register is enabled by writing to corresponding enable bit in the AACIMAINCR register. The AACITXCR1 is programmed for 3 valid slots. Then, the AACITrIntr1Reg is read to verify that the AACISnTXINTR [n = 1,2 or 12] interrupt is raised. Also the AACISLFR, AACISLISTAT and AACIALLINTS registers are checked. Three data are written into the Tx FIFO of the AACI and it is verified that the interrupt is not cleared. Then the SlnTxEn [n = 1, 2, 12] bit in the AACIMAINCR register is cleared thus disabling the transmission from that register. This should not clear the AACISnTXINTR [n = 1, 2, 12] interrupt. This is confirmed by reading the flag registers and interrupt status registers. Again the bit SlnTxEn [n = 1, 2, 12] is set to enable that slot register for transmission. The SlnTxBusy [n = 1, 2, 12] bit in the AACISLFR register is read to confirm that it is not set as the AACISLnTX register is still not written with any data and is empty.

Then the data is written into the AACISLnTX [n = 1, 2 or 12] register and the AACISnTXINTR [n = 1, 2, or 12] is checked for being cleared. And the SlnTxBusy [n = 1, 2, 12] bit in the AACISLFR register are read to confirm that it is set as the AACISLnTX register is written with some data and is non-empty.

Then data transmission is enabled. Data from the AACISLnTX [n = 1,2 or 12] register is expected to be immediately copied into the transmit shift register of the AACI, thereby rendering the AACISLnTX [n = 1,2 or 12] register empty. The AACITrIntr1Reg is read to verify that the AACISnTXINTR [n = 1,2 or 12] interrupt is raised. This interrupt is cleared by a single write to the AACISLnTX [n = 1,2 or 12] register.

This test is repeated by disabling the SlotnTxIE [n = 1, 2 or 12] bit in the AACISLIEN register. The interrupt is not expected to get set when it is masked. Also, it is confirmed that setting the Interrupt enable bit again results in the assertion of this interrupt.

6.4.7 DMA Tests

All the tests which verify the DMA functionality of the Tx and Rx FIFOs of Channel 1 are called in the DMATest() function.

AACIDMASREQRX Test

The DMASREQRXWTTTest() function call is used to test the AACIDMASREQRX generation in the presence of RxTimeout. The DMASREQRXTest() function is used to test the AACIDMASREQRX generation in the absence of RxTimeout.

The AACIDMASREQRX is tested with following combinations of parameters

- With Timeout and Without Timeout Condition.
- FIFO mode and Character Mode.
- Compact Mode and Non Compact Mode.

The AacIfE bit and DMAEnable bit of the AACIMAINCR register is set. The AACIRXCR1 register is programmed for only one / two valid slots reception per frame, depending on whether FIFO is in Non Compact Mode or Compact Mode respectively. To check the AACIDMASREQRX behaviour with Timeout, the TOC field in the AACIRXCR1 register is set to four invalid frames. Before the start of data transmission, it is expected that the Rx FIFO is empty, the AACIDMASREQRX bit of the AACITrDMAReg is checked for a '0'. The AACI is allowed to receive 4 data frames from the trickbox, each containing one valid slot. After every frame, the AACIDMASREQRX bit is tested for a '1'. If AACIDMASREQRX behaviour is checked with the Timeout condition, then the Trickbox is allowed to transmit four invalid frames. When the AACI receives slot 12 of the fourth invalid frame, it is expected to raise the RxTimeout Interrupt. The AACISR Register is read to verify that the RxTimeout flag has been set.

After the reception of all frames, transmission and reception is disabled. Data is read out one / two words at a time from the Rx FIFO, depending on whether FIFO is in Non Compact Mode or Compact Mode respectively. After each read the AACIDMASREQRX bit is checked for a '1' or '0' according to the Fill level of the FIFO and the AACIDMACLRRX signal from the trickbox is asserted. This is expected to clear the AACIDMASREQRX signal. The AACITrDMAReg register is read to verify this. After the deassertion of AACIDMACLRRX signal, the AACIDMASREQRX remains set or cleared depending on the Rx FIFO Fill level, timeout condition and the FIFO operating mode. The AACIDMASREQRX is expected to be set depending on the following conditions resulting due to the different combinations of with/without timeout condition and FIFO / Character mode and Compact / Non-Compact mode.

With Timeout Condition

- FIFO Mode

If FIFO is in Compact Mode, the AACIDMASREQRX will never be asserted.

If FIFO is in Non Compact Mode, the AACIDMASREQRX will be asserted if the Rx FIFO fill level is 2 or 3.

- Character Mode

Only Non Compact mode is supported if the FIFO is in character mode. Therefore, the AACIDMASREQRX will never be asserted in character mode with timeout condition.

Without Timeout Condition

- FIFO Mode

If FIFO is in Compact Mode, the AACIDMASREQRX will be asserted if the Rx FIFO fill level is 2

If FIFO is in Non Compact Mode, the AACIDMASREQRX will be asserted if the Rx FIFO fill level is 1, 2 or 3.

- Character Mode

Only Non Compact mode is supported if the FIFO is in character mode. The AACIDMASREQRX will always be asserted whenever the FIFO contains data in character mode.

Some exceptions to the above conditions occur due to mutual exclusivity of the AACIDMASREQRX and AACIDMALSREQRX. These exceptions are tested as follows:

With RxTimeout Condition and Rx FIFO in Character mode:

The AacIfE bit and DMAEnable bit is set in AACIMAINCR. The AACIRXCR1 register is programmed for only one valid slot reception per frame and the TOC field of AACIRXCR1 is set for four invalid frame. The RxFEEn bit is cleared to program the Rx FIFO in Character mode. Before the start of data transmission, it is expected that the Rx FIFO is empty and the AACIDMASREQRX bit of the AACITrDMAReg is checked for a '0'. The AACI is allowed to receive one frame from the trickbox, containing one valid slot. After the valid frame reception, the trickbox is allowed to transmit four invalid frames. After the valid frame reception, the AACIDMASREQRX bit is verified for a '1'. When the AACI receives slot 12 of the fourth invalid frame, it is expected to raise the RxTimeout interrupt. Then AACISR Register is read to verify that the RxTimeout flag has been set. After the reception of all frames, transmission and reception is disabled. The AACIDMASREQRX and AACIDMALSREQRX bits of the AACITrDMAReg are read to verify that AACIDMASREQRX is set and AACIDMALSREQRX is not set. In this case,

AACIDMASREQRX is set before the timeout condition occurs. AACIDMALSREQRX is not expected to be set here even though there is only one data in the FIFO when timeout occurs because AACIDMASREQRX and AACIDMALSREQRX are mutually exclusive.

With Rx Timeout Condition and Rx FIFO in Compact Mode (FIFO mode):

Theoretically, if FIFO is in compact mode (FIFO mode) with Rx Fill level = 2 and RxTimeout is set, it is expected that the AACIDMALSREQRX should be set and AACIDMASREQRX should be cleared. But the setting and clearing of the AACIDMALSREQRX and AACIDMASREQRX will depend on the amount of data Rx FIFO received and data reading. The following test cases are used to verify the above functionality.

- The AacIfE bit and DMAEnable bit of AACIMAINCR is set. The AACIRXCR1 register is programmed for Compact mode with only two valid slots reception per frame and TOC field is set to four invalid frame. The AACI is allowed to receive one valid frame and four invalid frames from the trickbox. When the AACI receives Slot12 of the fourth invalid frame, it is expected to raise the RxTimeout Interrupt. The AACISR Register is read to verify that the RxTimeout flag has been set. After the reception of all frames, transmission and reception is disabled. The AACIDMASREQRX and AACIDMALSREQRX bits of the AACITrDMAReg are read to verify that AACIDMASREQRX is set and AACIDMALSREQRX is not set. In this case, AACIDMASREQRX is set and not AACIDMALSREQRX even though the FIFO contains two data and RxTimeout is asserted. This is because AACIDMASREQRX gets set once two data are written into the FIFO, before the timeout condition occurs and AACIDMALSREQRX does not get set because of mutual exclusivity.
- The AACIFE bit and DMAENABLE bit of AACIMAINCR is set. The AACIRXCR1 register is programmed for compact mode with only two valid slots reception per frame and TOC field is set to four invalid frames. The AACI is allowed to receive two valid frames. After the transmission of two valid frames, the trickbox is allowed to transmit four invalid frames from the trickbox. When the AACI receives Slot12 of the fourth invalid frame, it is expected to raise the RxTimeout Interrupt. Then AACISR Register is read to verify that the RxTimeout flag has been set. After the reception of all frames, transmission and reception is disabled. It is expected that the AACIDMASREQRX and AACIDMABREQRX are set and AACIDMALSREQRX is cleared. Data is read out, one word at a time from the Rx FIFO in the AACI. After the first data read, the AACIDMACLRRX is asserted from the trickbox. The assertion of AACIDMACLRRX will clear the AACIDMASREQRX and AACIDMABREQRX. After deassertion of the AACIDMACLRRX, as RxTimeout is set and Rx FIFO is in Compact mode with fill level = 2, the AACIDMALSREQRX will be set and AACIDMASREQRX will be cleared.

AACIDMABREQRX Test

The DMABREQRXWTTTest() function call is used to test the AACIDMABREQRX generation in the presence of RxTimeout. The DMABREQRXTest() function is used to test the AACIDMABREQRX generation in the absence of RxTimeout.

The AACIDMABREQRX is tested with the following combinations of parameters

- With Timeout and Without Timeout Condition.
- Compact mode and Non Compact Mode.

Without Timeout Condition:

The AacIfE and DMAEnable bits are set in the AACIMAINCR register. The AACIRXCR1 register is programmed for only one valid slot / two valid slots reception per frame, depending on whether the FIFO is in Non Compact mode or Compact mode respectively. When the Rx FIFO is empty, the AACIDMABREQRX bit of the AACITrDMAReg is checked for a '0'. The AACI is allowed to receive from the trickbox two / four data frames, depending on whether the FIFO is in Compact Mode or Non Compact mode respectively. After every frame, the AACIDMABREQRX bit of the AACITrDMAReg register is read for a '0' or '1' depending on Rx FIFO fill level and FIFO mode. The AACIDMABREQRX will be asserted if Rx FIFO fill level is ≥ 4 when RxTimeout is not asserted.

The AACIDMABREQRX is tested for a '1' when Rx FIFO fill level reaches four. Also, it is verified that transferring another four data into the Rx FIFO does not clear this DMA request. After the reception of all valid frames, transmission and reception is disabled. Data is read out one/two words at a time, from the Rx FIFO in the AACI depending on whether the FIFO is in Compact Mode or Non Compact Mode, and the AACIDMABREQRX is checked for '1' or '0' depending on the Rx FIFO Fill level. After each data read, the AACIDMACLRRX is asserted

from the trickbox. The assertion of AACIDMACLRRX will clear the AACIDMABREQRX. After the deassertion of AACIDMACLRRX, the AACIDMABREQRX is checked for a '1' or '0' depending on the Rx FIFO fill level.

With Timeout Condition:

The AacIfE and DMAEnable bit is set in AACIMAINCR register. The AACIRXCR1 register is programmed for only one valid slot / two valid slot reception per frame, depending on whether the FIFO is in Non Compact mode or Compact mode respectively. The TOC bit field of AACIRXCR1 is written with 4 (timeout after 4 frames). Before start of transmission, it is expected that the Rx FIFO is empty, the AACIDMABREQRX bit of the AACITrDMAReg is checked for a '0'. The AACI is allowed to receive from the trickbox two / four data frames, depending on whether FIFO is in Compact Mode or Non Compact mode respectively. After every frame, the AACIDMABREQRX register is tested for a '0' or '1' depending on the Rx FIFO fill level and FIFO mode. The AACIDMABREQRX is tested for a '1' when the Rx FIFO fill level reaches four. Also, it is verified that transferring another four data into the Rx FIFO does not clear this DMA request. Now the trickbox is allowed to transmit four invalid frames. After the reception of all frames, transmission and reception is disabled. When the AACI receives Slot12 of the fourth invalid frame, it is expected to raise the RxTimeout Interrupt. Then the AACISR register is read to verify that the RxTimeout interrupt has been raised. Data is read out one/ two words at a time from the Rx FIFO in the AACI depending on whether the FIFO is in Compact Mode or Non Compact Mode and the AACIDMABREQRX is checked for a '1' or '0' depending on the Rx FIFO Fill level. After each data read, the AACIDMACLRRX is asserted from trickbox. The assertion of AACIDMACLRRX will clear the AACIDMABREQRX. After the deassertion of AACIDMACLRRX, the AACIDMABREQRX is checked for a '1' or '0' depending on the Rx FIFO fill level. The AACIDMABREQRX will be asserted if Rx FIFO fill level is > 4 when RxTimeout is asserted.

Some exceptions to the above conditions occur due to mutual exclusivity of the AACIDMABREQRX and AACIDMALBREQRX. These exceptions are tested as follows:

With Rx Timeout Condition and Rx FIFO in Compact Mode (FIFO mode):

Theoretically, if the Rx FIFO fill level is four and RxTimeout is set, it is expected that the AACIDMALBREQRX should be set and AACIDMABREQRX should be cleared. But the AACIDMALBREQRX and AACIDMABREQRX depending on the amount of data it had received and data reading. The following test cases are used to verify the above functionality.

- The AacIfE and DMAEnable bits are set in the AACIMAINCR register. The AACIRXCR1 register is programmed for only one valid slot / two valid slots reception per frame, depending on whether the FIFO is in Non Compact mode or Compact mode respectively. The TOC bit field of AACIRXCR1 is written with 4 (timeout after 4 frames). The AACI is allowed to receive from the trickbox two / four data frames, depending on whether the FIFO is in Compact Mode or Non Compact mode respectively. After the transmission of all valid frames, the trickbox is allowed to transmit the four invalid frames. The AACIDMABREQRX is tested for a '1' when the Rx FIFO fill level reaches four. Also, it is verified that transferring another four data into the Rx FIFO does not clear this DMA request. After the reception of all frames, transmission and reception is disabled. The AACIDMABREQRX and AACIDMALBREQRX bits of the AACITrDMAReg register are read for a '1' and '0' respectively. When the AACI receives Slot12 of the fourth invalid frame, it is expected to raise the RxTimeout Interrupt. Since the FIFO already contains 4 data before the RxTimeout condition occurs, AACIDMABREQRX is set and AACIDMALBREQRX does not get set when the timeout condition occurs, due to mutual exclusivity.
- The AacIfE and DMAEnable bit is set in AACIMAINCR register. The AACIRXCR1 register is programmed for only one valid slot / two valid slots reception per frame, depending on whether the FIFO is in Non Compact mode or Compact mode respectively. The TOC bit field of AACIRXCR1 is written with 4 (timeout after 4 frames). The AACI is allowed to receive from the trickbox four / eight data frames, depending on whether the FIFO is in Compact Mode or Non Compact mode respectively. After the transmission of all valid frames, the trickbox is allowed to transmit four invalid frame. The AACIDMABREQRX is tested for a '1' when Rx FIFO fill level reaches four. After the reception of all frames, transmission and reception is disabled. When the AACI receives Slot12 of the fourth invalid frame, it is expected to raise the RxTimeout Interrupt. Data is read out one/two words at a time from the Rx FIFO in the AACI, depending on whether the FIFO is in Non Compact Mode or Compact Mode respectively. After two / four frame reads, Rx FIFO fill level drops to four. Now AACIDMACLRRX is asserted from the trickbox which will clear the AACIDMABREQRX. After deassertion of

the AACIDMACLRRX signal, AACIDMALBREQRX will be set and AACIDMABREQRX will remain cleared as Rx FIFO fill level is four and RxTimeout condition is true.

AACIDMALSREQRX Test

The DMALSREQRXTest() function is used to test the generation of AACIDMALSREQRX in FIFO mode. The DMALSChar() function is used to test the generation of AACIDMALSREQRX when the Rx FIFO is in Character mode.

The AACIDMALSREQRX is tested with following combinations of parameters

- FIFO mode and Character Mode.
- Compact Mode and Non Compact Mode.

FIFO Mode:

The AacIfE and DMAEnable bits are set in the AACIMAINCR register. The TOC bit field of AACIRXCR1 is written with 4 (timeout after 4 frames). The AACIRXCR1 register is programmed for reception of 4 valid slots, say slot 3, 5, 7, and 8. The Tx FIFO in the trickbox is filled with valid data for 4 slots and with invalid data for the other slots in the first frame. Then the trickbox is allowed to transmit data. Subsequent frames transmitted from the Trickbox are invalid frames that do not result in any further data being written to the Rx FIFO of the AACI. When the AACI receives Slot12 of the fourth invalid frame, it is expected to raise the RxTimeout Interrupt. Then the AACISR Register is read to verify that the RxTimeout flag has been set.

The AACIDMALSREQRX bit of the AACITrDMAReg register is checked for a '0'. Then one / two data words, depending on whether the FIFO is in Compact mode or Non Compact mode respectively, are read from the Rx FIFO in the AACI. The AACIDMACLRRX is asserted and deasserted. The assertion of the AACIDMACLRRX is expected to clear the AACIDMASREQRX. After deassertion of the AACIDMACLRRX signal, since the Rx FIFO fill level is one / two, depending on whether FIFO is in Compact mode or Non Compact mode respectively, and RxTimeout is set, the AACIDMALSREQRX is expected to be set. The AACIDMALSREQRX bit of the AACITrDMAReg register is checked for a '1'. After the last word is read out of the Rx FIFO, the AACIDMALSREQRX bit of the AACITrDMAReg register is again checked for a '1'. The AACIDMACLRRX signal is asserted via the trickbox to clear this DMA request.

Character Mode:

The test sequence to verify AACIDMALSREQRX generation when the FIFO is in Character mode with RxTimeout condition is as follows:

The AacIfE bit is set in AACIMAINCR (DMAEnable = 0 , ie DMA disable). The AACIRXCR1 register is programmed for only one valid slot reception per frame and the TOC field is set for four invalid frames. The AACI is allowed to receive one valid frame from the trickbox. Subsequent frames transmitted from the Trickbox are invalid frames that do not result in any further data being written to the Rx FIFO of the AACI. When the AACI receives Slot12 of the fourth invalid frame, it is expected to raise the RxTimeout Interrupt. Then the AACISR Register is read to verify that the RxTimeout flag has been set. After the reception of all frames, transmission and reception is disabled. The AACIDMASREQRX and AACIDMALSREQRX bits of the AACITrDMAReg are checked for '0's. Now the DMAEnable bit is set in the AACIMAINCR register. Since the Rx Fill level is one and RxTimeout is set, now the AACIDMALSREQRX will be set. The data in the Rx FIFO is read and verified for correct reception.

AACIDMALBREQRX Test

The function, DMALBREQRXTest(), is used to test the AACIDMALBREQRX functionality.

The AACIDMALBREQRX is tested in Compact mode and Non Compact mode.

The AacIfE and DMAEnable bits are set in the AACIMAINCR register. The TOC bit field of AACIRXCR1 is written with 4 (timeout after 4 frames). The AACIRXCR1 register is programmed for reception of slots 3, 4, 5, 6, 7, 8, 10

and 12. The Tx FIFO in the trickbox is filled with valid data for 8 slots and with invalid data for the other slots in the first frame. Then the trickbox is allowed to transmit data. Subsequent frames transmitted from the trickbox are invalid frames that do not result in any further data being written to the Rx FIFO of the AACI. When the AACI receives Slot12 of the fourth invalid frame, it is expected to raise the RxTimeout Interrupt. Then RxTimeout bit in the AACISR Register is read to verify that the RxTimeout interrupt has been raised. After the reception of all frames, transmission and reception is disabled.

The AACIDMALBREQRX bit in the AACITrDMAReg register is tested for a '0'. Then one / two data words, depending on whether the Rx FIFO is in Compact mode or Non Compact mode respectively, are read from the Rx FIFO of the AACI. After each read, the AACIDMALBREQRX bit in the AACITrDMAReg register is tested for a '0'. Also after each read the AACIDMACLRRX signal is asserted and deasserted. After deassertion of the AACIDMACLRRX signal, the AACIDMALBREQRX bit in the AACITrDMAReg register is tested for a '0'. After reading the fourth data, the AACIDMALBREQRX is checked for a '0'. Now the AACIDMACLRRX signal is asserted and deasserted. After deassertion of the AACIDMACLRRX signal, as RxTimeout is set and the Rx FIFO fill level is four, it is expected that the AACIDMALBREQRX is set. This is verified by reading the AACITrDMAReg register. The remaining four data in the Rx FIFO is read and verified for correct reception.

AACIDMABREQTX Test

The AACIDMABREQTX generation by the AACI is verified by calling the function DMABREQTXTest().

The AacIfE and DMAEnable bit are set in the AACIMAINCR register. The AACITXCR1 register of the AACI is programmed for only one / two valid slots transmission per frame depending on whether the FIFO is in Non Compact mode or Compact Mode respectively. When the Tx FIFO in the AACI is empty, the AACIDMABREQTX bit in the AACITrDMAReg register is tested for a '1'. Then, the Tx FIFO in the AACI is filled with eight data. After each data write to the Tx FIFO, the AACIDMABREQTX line is checked for a '1'. Then, the AACIDMACLRTX signal is asserted through the trickbox. The AACIDMABREQTX line is then checked for a '0' and the AACIDMACLRTX is deasserted. Data transfer is enabled. After each of the first one / three frames transmission, depending on whether the FIFO is in Compact mode or in Non Compact mode respectively, AACIDMABREQTX is tested for a '0'. After the next frame is transmitted, the Tx FIFO in the AACI has four data, so the AACIDMABREQTX bit of the AACITrDMAReg is tested for a '1'. After transmission of each of the next two / four frames depending on whether the FIFO is in Compact Mode or Non Compact Mode, AACIDMABREQTX is expected to remain asserted. When all frames are transmitted, transmission and reception is disabled. The AACIDMACLRTX signal is asserted and deasserted. After deassertion of the AACIDMACLRTX, the AACIDMABREQTX is expected to get set. The AACITrDMAReg in the trickbox is read to verify this.

DMA Disable Test

The function, DMADisableTest() is used to test the effect of clearing the DMAenable bit on DMA request generation.

The DMADisableTest is tested in Compact Mode and Non Compact Mode.

The AacIfE bit of the AACIMAINCR register is set (DMAEnable = 0). The AACIRXCR1 register is programmed for only one/two valid slots reception per frame, depending on whether FIFO is in Compact Mode or Non Compact Mode. The AACI is allowed to receive 4 data frames from the trickbox. After every frame, the AACIDMASREQRX and AACIDMABREQRX bits are checked for a '0'. After the reception of all frames, transmission and reception is disabled. Data is read out one/two word at a time from the Rx FIFO in the AACI, depending on whether the FIFO is in Compact Mode or Non Compact Mode. After each read the AACIDMASREQRX, AACIDMALSREQRX, AACIDMABREQRX and AACIDMALBREQRX bits are checked for a '0'.

AACIFE Disable DMA Test

The function, DMAACIFEDisable() is used to test the effect of clearing the DMAEnable bit on DMA request generation.

The AacIfE and DMAEnable bits are set in the AACIMAINCR register. The AACIRXCR1 register is programmed for only one valid slot / two valid slots reception per frame, depending on whether the FIFO is in Non Compact mode or Compact mode respectively. When Rx FIFO is empty, the AACIDMASREQRX, AACIDMABREQRX, AACIDMALSREQRX, AACIDMALBREQRX bits are checked for a '0' and AACIDMABREQTX is checked for a '1'.

The AACI is allowed to receive from the trickbox four / eight data frames, depending on whether the FIFO is in Compact Mode or Non Compact mode respectively. After every frame, all the DMA request bits of the AACITrDMAReg register are checked for their expected values depending on the Rx FIFO fill level.

After the reception of all valid frames, transmission and reception is disabled. The AacIfE bit is cleared in the AACIMAINCR register and all the DMA request bits in AACITrDMAReg are checked for '0's. Once again the AacIfE bit is set in the AACIMAINCR register. All the DMA receive signals i.e. AACIDMASREQRX, AACIDMABREQRX, AACIDMALSREQRX, AACIDMALBREQRX bits are checked for a '0' and AACIDMABREQTX is checked for a '1'.

6.4.8 Data Setup/Hold Test

The function DataWidthCheck() is called to check the Setup and Hold times of the data output on the AACISDATAOUT port with respect to the falling edge of AACIBITCLK. The AACITrWidChkEn bit in the Trickbox Control register is set. The AACI is allowed to transmit data words of 0x55555/0xAAAAA such that the AACISDATAOUT line from the AACI toggles with every bit period (except for the slot 0 and slot 2 in the frame). The Trickbox determines whether the Transmit data line meets setup and hold with respect to the falling edge of AACIBITCLK. Any violations are flagged as errors.

The test sequence is as follows

- All the channels are programmed for a TSize of 20.
- All the channel FIFOs are written with data 0xAAAAA at all the 8 locations.
- The registers AACISL1TX and AACISL2TX are written with the data 0xAAAAA.
- Trickbox Tx FIFO is filled with the data for 5 valid frames so that it will set the CODEC ready bit on AACISDATAIN line at the start of the every frame at the tag slot and detecting this, the AACI will send out data on AACISDATAOUT line for 4 frames.
- Then the channels are enabled for transmission.
- The AACISL1TX and AACISL2TX registers are enabled for transmission by setting the respective enable bits in the AACIMAINCR register.
- The trickbox is enabled for reception with AacITrWidChkEn bit in the control register of the trickbox set to enable the checking of the width of every pulse on the AACISDATAOUT and its set up and hold with respect to the falling edge of AACIBITCLK. Here, in the normal transmission, the trickbox checks the data width of the every pulse on the AACISDATAOUT line for all the slots except the tag slot and the slot 2 of the frame. It also checks for setup and hold violations, if any, for AACISDATAOUT with respect to falling edge of the AACIBITCLK and flags error messages if any violation is there.
- The normal transmission of the data takes place. After 4 frames of transmission from AACISDATAOUT line the transmission and reception is disabled and the 4 frames of data from the AACISDATAOUT port of the AACI (which was received in the trickbox receive FIFO) are read from the trickbox receive FIFO.

6.4.9 FIFO Status Flag Test

FIFO Status Flag Test

The FIFO status flags are tested on calling the function FIFOStatusTest().

This test verifies the generation of the following status flags of each channel from the AACISR registers:

TxBusy, RxBusy, TxFF, RxFF, TxHE, RxHF, TxFE and RxFE.

The AacIfE bit of the AACIMAINCR register is set. The Trickbox is loaded with data for one frame within which 8 slots are valid. Then TxBusy, RxBusy, TxFF, RxFF, TxHE, RxHF bits of the AACISR register are checked for '0's, and the TxFE and RxFE bits are checked for '1's. The AACITXCR and AACIRXCR registers are programmed for 8 valid slots. The AACI Tx FIFO is loaded with 8 data and the TxFF bit is checked for a '1' and the TxHE bit is checked for '0'. Data transmission and reception is started and immediately the RxBusy and TxBusy bits are expected to be set. After one slot is transferred, the RxFE and TxFF bits are expected to be cleared. When data

for the fifth slot is removed from the Tx FIFO, the RxHF bit is expected to be set and the TxHE bit is expected to be cleared. Then, after all slots are transmitted, the RxFF bit is expected to be set and TxBusy and RxBusy are expected to be cleared. Data is read back from the Rx FIFO of the Trickbox and the Rx FIFO of the AACI to check for error-free transmission and reception.

This test is repeated for all 4 channels.

AACI Disable FIFO Flag Test

The effect of clearing the AacIfE bit on the FIFO status flags is tested by the function FlagTest().

The AacIfE bit of the AACIMAINCR register is cleared. The Trickbox is loaded with data for one frame within which 8 slots are valid. Then TxFF, RxFF, TxHE, RxHF bits in the AACISR register are checked for '0's, and the TxFE and RxFE bits are checked for '1's. The AACITXCR and AACIRXCR registers are programmed for 8 valid slots. Data transmission from the Trickbox is enabled. After transmission has started, it is verified that the FIFO flags status will remain in default state due to the AacIfE bit being disabled. Now the AacIfE bit is enabled and the AACI Tx FIFO is loaded with 8 data. After the reception of a frame, transmission and reception is disabled. The data is read from Rx FIFO of the AACI and compared with expected data to verify the error-free reception.

6.4.10 FIFO pointer cross-over tests

These are boundary checks on the Transmit FIFOs and Receive FIFOs. When the FIFO pointers are both at 7 and when the FIFO is FULL, if a read and write occur simultaneously to the FIFO, the read and write should both be allowed to have effect.

The FIFOTEST bits in the AACITCR register (AACITCR[2:1]) are used for testing the FIFOs. These bits determine which FIFO (Rx or Tx) the reads and writes to a AACIDR register will target.

1 : FIFOTEST = 00 Normal mode.

In this mode, the read will return the data from the read port of the Rx FIFO and the write will write into the write port of the Tx FIFO

2 : FIFOTEST = 01 [test mode]

Reads return Read Port of Tx FIFO and Writes will write data into the Write Port of the Rx FIFO.

3 : FIFOTEST = 10 [test mode] for Rx FIFO test

Reads will return data from the Read port of the Rx FIFO and additionally, read access automatically generates a Write Access to the Write port of the Rx FIFO.

4 : FIFOTEST = 11 [test mode] for Tx FIFO test

Reads will return data from the Read port of the Tx FIFO and additionally, read access automatically generates a Write Access to the Write port of the Tx FIFO.

The test sequence is as follows

For Tx FIFO

1. Fill the FIFO with 7 entries in normal mode.
2. Read the FIFO for 7 entries in test mode 01.
3. Fill the FIFO with 8 entries in normal mode.
4. Read the FIFO last data in test mode 11.

(this simulates the simultaneous read and write when FIFO is full and read and write pointers are at value 7)

5. Read first 8 datas in mode 01, which were written at step 3.

6. Confirm last data = 0x55555. Read this also in test mode 01.

(the last data is introduced by hardware in the test mode 11 by read in the step 4)

For Rx FIFO

1. Fill the FIFO with 7 entries in test mode 01.

2. Read the FIFO for 7 entries in normal mode.

3. Fill the FIFO with 8 entries in test mode 01.

4. Read the FIFO last data in test mode 10.

(this simulates the simultaneous read and write when FIFO is full and read and write pointers are at value 7)

5. Read first 8 datas in mode 01, which were written at step 3.

6. Confirm last data = 0x55555. Read this also in test mode 01.

(the last data is introduced by hardware in the test mode 10 by read in the step 4)

These tests are also done for the FIFO in the CHARACTER mode where the FIFO size is considered as the single entry FIFO.

6.4.11 Data Tests

The transmission / reception logic of the AACI is verified under various operating modes using the Data Tests. These tests are split into 25 major categories with each category containing several sub-tests. These 25 categories are derived from combinations of the following 4 fields:

<Data Direction> - <Data Src/Dest> - <FIFO mode> - <SRC mode>

Data Direction	- Tx Only / Rx Only / Tx & Rx
Data Src/Dest	- CH / SLReg / CH & SLReg
FIFO mode	- FIFO / Char
SRC	- SRC / No SRC

AACI Simultaneous Transmit-Receive Tests

The variables TxSize[n] (n = 1, 2, 3, 4) and CmpctMod are initialised such that the elements in the TxSize array indicate TSize of 12, 16, 18 and 20 respectively and CmpctMod indicates non-compact mode. The variables RxSize[n] (n = 1, 2, 3, 4) and CmpctMod are initialised such that the elements in the RxSize array indicate RSize of 12, 16, 18 and 20 respectively and CmpctMod indicates non-compact mode. Then TxRx_FsPCLK_Call() function is called to set the values of the control parameters required for transmission and reception. The TxRx_FsPCLK_Call() function calls the InitCntlWrd() function to initialise all the ControlWord structure elements to their default values and reassigns the value of the TxSize and RxSize array to new values derived from the previous value of the same array. All the ControlWord structure elements which contain values which are different from the default values are assigned to the values required for a particular test after the InitCntlWrd() function call. Then the function TxRx_FsPCLK_Test() is called with a parameter indicating the number of frames to be transmitted and received. The function TxRx_FsPCLK_Test() starts the transmission and reception.

Tx & Rx - CH - FIFO mode - No SRC:

The AACI is programmed for both transmission and reception in the FIFO mode. No SRC bits are set in the incoming frames. Five tests are done under this category.

and all the slot valid bits set. Therefore, AACI receives all the slots (Slot 2 and 12) programmed in the AACIRXCR3 and AACIRXCR4 registers.

Finally, the Rx FIFOs of the trickbox and the AACI are read to verify the last frame transmission and reception.

This basic sequence of the data test is repeated twice with the following variations.

- The number of slots enabled per transmitted frame.
- The number of slots enabled for reception per channel
- The slot numbers enabled in different channels are varied.
- Slot0 to be transmitted by the trickbox for each frame
- TSize and RSize

Tx & Rx - CH - Char mode - SRC:

The AACI is programmed for transmission and reception in the Character mode. Multiple SRC bits are set in the incoming frames. Three tests are done under this category.

A Sample test sequence is described here:

The following AACITXCR registers are programmed for transmitting the mentioned slots.

AACITXCR1	Slot 3 with TSize = 16
AACITXCR2	Slot 1 with TSize = 18
AACITXCR3	Slot 6 with TSize = 20
AACITXCR4	Slot 4 with TSize = 12

The following AACITXCR registers are programmed for receiving the mentioned slots.

AACIRXCR1	Slot 3 with RSize = 16
AACIRXCR2	Slots 9 with RSize = 18
AACIRXCR3	Slot 12 with RSize = 20
AACIRXCR4	Slot 1 with RSize = 12

The following SRC bits are set in the Slot1 of the frames transmitted by the trickbox:

Frame1	Slots 4 and 6
Frame2	Slots 3 and 4
Frame3	Slots 3 and 6
Frame4	Slot 4

The Tx FIFOs of the AACI are filled with the data (Slots 1, 3, 4 and 6) for the first frame. The Tx FIFO of the trickbox is filled with the data for two frames. Then AACI is allowed to start the transmission and reception. The first frame transmitted should be invalid since the CODEC ready bit was set only in the first incoming frame. The Slot0 transmitted by the trickbox has the CODEC ready and all the slot valid bits set. Hence, AACI receives all the slots (Slot 1, 3, 4 and 6) programmed in the AACIRXCR[n] (n = 1 to 4) registers. The received slots are stored in the respective Rx FIFOs of the AACI.

During the second frame transmission, the following steps are performed. This frame contains the slots 1 and 3. The other slots 4 and 6 are inhibited by the SRC bits in the previous frame. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Tx FIFO of the Channel 1 and Channel 2 are filled with new data for the next

frame. The Slot0 transmitted by the trickbox has the CODEC ready bit and all the slot valid bits set. Therefore, AACI receives all the slots (Slot 1, 3, 4 and 6) programmed in the AACIRXCR[n] (n = 1 to 4) registers. The received slots are stored in the respective Rx FIFOs of the AACI.

During the third frame transmission, the following steps are performed. This frame contains the slots 1 and 6. The other slots 3 and 4 are inhibited by the SRC bits in the previous frame. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Tx FIFO of the Channel 3 is filled with a new data for the next frame. The Slot0 transmitted by the trickbox has the CODEC ready bit and all slot valid bits set. Therefore, the AACI receives all the slots (Slot 1, 3, 4 and 6) programmed in the AACIRXCR[n] (n = 1 to 4) registers. The received slots are stored in the respective Rx FIFOs of the AACI.

During the fourth frame transmission, the following steps are performed. This frame contains only slot 4. The other slots 3 and 6 are inhibited by the SRC bits in the previous frame and there is no data for slot 1 in the Channel 1. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Tx FIFO of the Channel 4 is filled with a new data (slot4) for the next frame. The Slot0 transmitted by the trickbox has the CODEC ready bit set, and all the slot valid bits set. Therefore, AACI receives all the slots (Slot 1, 3, 4 and 6) programmed in the AACIRXCR[n] (n = 1 to 4) registers. The received slots are stored in the respective Rx FIFOs of the AACI.

During the fifth frame transmission, the following steps are performed. This frame contains the slots 3 and 6. The slot 4 is inhibited by the SRC bits in the previous frame and there is no data for slot 1 in the Channel 1. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Slot0 transmitted by the trickbox has the CODEC ready bit set but no slot valid bits are set. Therefore, no data is stored in the Rx FIFOs of the AACI.

During the sixth frame transmission, the following steps are performed. This frame contains only the slot 4, there is no data for other slots in all other channels. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Slot0 transmitted by the trickbox has the CODEC ready bit set but no slot valid bits are set. Therefore, no data are stored in the Rx FIFOs of the AACI.

Finally, the Rx FIFO of the trickbox is read to verify the last frame transmission by the AACI.

This basic sequence is repeated for another three sets of data tests with the following variations.

- The number of slots enabled per transmitted frame.
- The number of slots enabled for reception per channel
- The slot numbers enabled in different channels are varied.
- SRC bit-pattern to be transmitted by the trickbox for each frame
- Slot0 to be transmitted by the trickbox for each frame
- TSize and RSize

Tx & Rx - SLReg - No SRC

The AACI is programmed for both transmission and reception through the AACISLnTX (n = 1, 2 and 12) and AACISLnRX (n = 1, 2 and 12) registers. No SRC bits are set in the incoming frames. Two data tests are done under this category.

A Sample test sequence is described here:

All the AACISLnTX (n = 1, 2 and 12) and all the AACISLnRx (n = 1, 2 and 12) registers are enabled through the AACIMAINCR register.

All the Tx Slot registers in the AACI are filled with data for second frame. The Tx FIFO of the trickbox is filled with the data for two frames. Then AACI is allowed to start the transmission and the reception. The first frame transmitted should be invalid since the CODEC ready bit was set only in the first incoming frame. The Slot0 transmitted by the trickbox has the CODEC ready and all the slot valid bits set. The AACI receives the data for

slots 1, 2 and 12 and stores it in the corresponding AACISLnRX ($n = 1, 2$ and 12) registers. At the end of the first frame, the data stored in the registers are read back to verify the correct reception.

During the second frame transmission, the following steps are performed. This frame should contain the slots 1, 2 and 12 transmitted from the Tx slot registers of the AACI. The Rx FIFO of the trickbox is read back to verify the previous frame transmitted by the AACI. The Tx Slot registers are filled with new data for the next frame. The AACI receives the data for slots 1, 2 and 12 and stores it in the corresponding AACISLnRX ($n = 1, 2$ and 12) registers. At the end of the second frame, the data stored in the Rx registers are read back to verify the correct reception.

During the third frame transmission, the following steps are performed. This frame should contain the slots 1, 2 and 12 transmitted from the Tx slot registers of the AACI. The Rx FIFO of the trickbox is read back to verify the previous frame transmitted by the AACI. The Tx Slot registers are filled with new data for the next frame. The AACI receives the data for slots 1, 2 and 12 and stores it in the corresponding AACISLnRX ($n = 1, 2$ and 12) registers. At the end of the third frame, the data stored in the registers are read back to verify the correct reception.

During the fourth frame transmission, the following steps are performed. This frame should contain the slots 1, 2 and 12 transmitted from the Tx slot registers of the AACI. The Rx FIFO of the trickbox is read back to verify the previous frame transmitted by the AACI. The AACI receives the data for slots 1, 2 and 12 and stores it in the corresponding AACISLnRX ($n = 1, 2$ and 12) registers. At the end of the fourth frame, the data stored in the registers are read back to verify the correct reception.

Finally, the Rx FIFO of the trickbox is read to verify the last frame transmitted by the AACI.

This basic sequence is repeated for another data test with the following variations.

- The AACISLnTX ($n = 1, 2, 12$) registers which are to be loaded with new data for each frame.
- The AACISLnTX ($n = 1, 2, 12$) registers which are to be enabled for each frame.
- The AACISLnRX ($n = 1, 2, 12$) registers which are to be enabled for each frame.
- The number of slots enabled per received frame.
- Slot0 to be transmitted by the trickbox for each frame.

Tx & Rx - SLReg - SRC:

The AACI is programmed for both transmission and reception through the AACISLnTX ($n = 1, 2$ and 12) and AACISLnRX ($n = 1, 2$ and 12) registers. Multiple SRC bits are set in the incoming frames. Two data tests are done under this category.

A Sample test sequence is described here:

All the AACISLnTX ($n = 1, 2$ and 12) and all the AACISLnRX ($n = 1, 2$ and 12) registers are enabled through the AACIMAINCR register.

The following SRC bits are set in the Slot1 transmitting by the trickbox

Frame1	slots 3, 4, 5, 6, 7, 8, 9, 10, 11 and 12
Frame2	No SRC bit set
Frame3	slots 3, 4, 5, 6, 7, 8, 9, 10, 11 and 12
Frame4	No SRC bit set

All the Tx Slot registers in the AACI are filled with data for the first frame. The Tx FIFO of the trickbox is filled with the data for two frames. Then AACI is allowed to start the transmission and the reception. The first frame transmitted should be invalid since the CODEC ready bit was set only in the first incoming frame. The Slot0 transmitted by the trickbox has the CODEC ready and slots 1, 2, 3, 4, 10, 12 valid bits set. The AACI receives the

data for slots 1, 2 and 12 and stores it in the corresponding AACISLnRX (n = 1, 2 and 12) registers. At the end of the first frame, the data stored in the Rx registers are read back to verify the correct reception.

During the second frame transmission, the following steps are performed. This frame should contain the slots 1 and 2. The other slot 12 is inhibited by the SRC bits in the previous frame. The Rx FIFO of the trickbox is read to verify the previous (invalid) frame transmitted by the AACI. The Tx Slot registers AACISL1TX and AACISL2TX are filled with the new data for the next frame. The Slot0 transmitted by the trickbox has the CODEC ready and slots 1, 2, 3, 4, 10, 12 valid bits set. Hence, the AACI receives the data for slots 1, 2 and 12 and stores it in the corresponding AACISLnRX (n = 1, 2 and 12) registers. At the end of the second frame, the data stored in the Rx registers are read to verify the correct reception.

During the third frame transmission, the following steps are performed. This frame should contain the slots 1, 2 and 12 transmitted from the Tx slot registers of the AACI. The Rx FIFO of the trickbox is read back to verify the previous frame transmitted by the AACI. The Tx Slot registers AACISL1TX, AACISL2TX and AACISL12TX are filled with new data for the next frame. The Slot0 transmitted by the trickbox has the CODEC ready and the valid bits of slots 1, 2, 3, 4, 10, 12 set. Hence, the AACI receives the data for slots 1, 2 and 12 and stores it in the corresponding AACISLnRX (n = 1, 2 and 12) registers. At the end of the third frame, the data stored in the Rx registers are read to verify the correct reception.

During the fourth frame transmission, the following steps are performed. This frame should contain the slots 1 and 2. The other slot 12 is inhibited by the SRC bits in the previous frame. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Tx Slot registers AACISL1TX and AACISL2TX are filled with the new data for the next frame. The Slot0 transmitted by the trickbox has the CODEC ready and slot valid bits of slots 1, 2, 3, 4, 10, 12 set. Hence, the AACI receives the data for slots 1, 2 and 12 and stores it in the corresponding AACISLnRX (n = 1, 2 and 12) registers. At the end of the fourth frame, the data stored in the Rx registers are read to verify the correct reception.

During the fifth frame transmission, the following steps are performed. This frame should contain the slots 1, 2 and 12 transmitted from the Tx slot registers of the AACI. The Rx FIFO of the trickbox is read back to verify the previous frame transmitted by the AACI. The Slot0 transmitted by the trickbox has the CODEC ready and the slot valid bits of slots 1, 2, 3, 4, 10, 12 set. Hence, the AACI receives the data for slots 1, 2 and 12 and stores it in the corresponding AACISLnRX (n = 1, 2 and 12) registers. At the end of the fifth frame, the data stored in the Rx registers are read to verify the correct reception.

Finally, the Rx FIFO of the trickbox is read to verify the last frame transmitted by the AACI.

This basic sequence is repeated for another data test with the following variations.

- The AACISLnTX (n = 1, 2, 12) registers which are to be loaded with new data for each frame.
- The AACISLnTX (n = 1, 2, 12) registers which are to be enabled for each frame
- The AACISLnRX (n = 1, 2, 12) registers which are to be enabled for each frame
- SRC bit-pattern to be transmitted by the trickbox for each frame
- The number of slots enabled per received frame.
- Slot0 to be transmitted by the trickbox for each frame

Tx & Rx - CH & SLReq - FIFO mode - No SRC:

The AACI is programmed for both transmission and reception in the FIFO mode. AACISLnTX (n = 1, 2 and 12) are also enabled for transmission. AACISLnRX (n = 1, 2 and 12) are enabled for reception. No SRC bits are set in the incoming frames. Eleven data tests are done under this category.

A sample test sequence is described:

The following AACITXCR registers are programmed for transmitting the mentioned slots.

AACITXCR3 Slots 5, 6, 7, 8, 9, 10 and 12 with TSize = 18

AACITXCR4 Slots 3, 4 and 11 with TSize = 20

AACIRXCR3 Slots 3, 4, 5, 6, 7 and 11 with RSize = 18

The Tx FIFOs of the AACI and the Tx slot register AACISL1TX and AACISL12TX are filled with the data for the first frame. The Tx FIFO of the trickbox is filled with data for two frames and Slot 0 data is written to indicate CODEC ready. The first frame transmitted should be invalid since the CODEC ready bit was set only in the first incoming frame. The Slot0 transmitted by the trickbox has the CODEC ready and the valid bits of slots 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 set. The AACI should receive slots 2, 8, 9, 10 and 12 in the Channel 2 and slots 3, 4, 5, 6, 7 and 11 in the Channel 3. It is expected that data for slot 12 should also get written into the Rx slot registers. At the end of the first frame, the data stored in the Rx registers are read to verify the correct reception.

During the third frame transmission, the following steps are performed. This frame should contain slots 3, 4, 5, 6, 7, 8, 9, 10, 11 and 12 from the channels and slot 1 from the AACISL1TX register. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Tx Slot register AACISL1TX is filled with new data. The Slot0 transmitted by the trickbox has the CODEC ready and the valid bits of slots 3, 4, 5, 6, 7 and 11 set. Hence the AACI should receive the slots 3, 4, 5, 6, 7 and 11 only, and the other slots 1, 2, 8, 9, 10 and 12 are rejected by the AACI even though AACIRXCR2 is programmed to receive these slots. It is expected that data for slot 1 and 12 should not be written into the Rx slot registers.

During the fourth frame transmission, the following steps are performed. This frame should contain slot 1 from the Tx slot register AACISL1TX and slot 2 from the Tx slot register AACISL12TX. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Slot0 transmitted by the trickbox has the CODEC ready and the valid bits of slots 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 set. The AACI should receive slots 1, 2, 8, 9, 10 and 12 in the Channel 2 and slots 3, 4, 5, 6, 7 and 11 in the Channel 3. It is expected that data for slot 1 is getting written into the Rx Slot register AACISL1RX and data for slot12 is getting written into the Rx slot register AACISL12RX. At the end of the fourth frame, the data stored in the Rx registers are read to verify the correct reception

Finally, the Rx FIFO of the trickbox is read to verify the last frame transmitted by the AACI.

This basic sequence is repeated for another ten data tests with the following variations.

- The number of slots enabled for transmission per channel
- The number of slots enabled per Rx channel
- The number of slots enabled per transmitted frame.
- The number of slots enabled per received frame.
- The slot numbers enabled in different channels are varied.
- TSize and RSize

expected that data for slot 1 and 12 should also get written into the Rx slot registers. At the end of the first frame, the data stored in the Rx slot registers are read to verify the correct reception.

During the second frame transmission, the following steps are performed. This frame should contain slots 3, 4 and 11 from the channels and slot 1 from the AACISL1TX register. Other slots are inhibited by the SRC bits in the previous frame. The data corresponding to the previous frame are read from the Rx FIFO of the Channel 1 and the Channel 2. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Tx FIFO of the Channel 2 is filled with data for slots 3, 4 and 11 for the next frame. The Tx Slot register AACISL1TX is filled with new data. The Slot0 transmitted by the trickbox has the CODEC ready and all the slot valid bits set. The AACI should receive all slots programmed in AACIRXCR1 register in the Channel 1 and all the slots programmed in AACIRXCR2 register in the Channel 2. It is expected that data for slot 1 and 12 should also get written into the Rx slot registers. At the end of the second frame, the data stored in the Rx slot registers are read to verify the correct reception.

During the third frame transmission, the following steps are performed. This frame should contain slots 5, 6, 7, 8, 9, 10 and 12 from the channels and slot 1 from the AACISL1TX register. Other slots are inhibited by the SRC bits in the previous frame. The data corresponding to the previous frame are read from the Rx FIFO of the Channel 1 and the Channel 2. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Tx FIFO of the Channel 1 is filled with data for slots 5, 6, 7, 8, 9, 10 and 12 for the next frame. The Slot0 transmitted by the trickbox has the CODEC ready and all the slot valid bits set. The AACI should receive all slots programmed in AACIRXCR1 register in the Channel 1 and all the slots programmed in AACIRXCR2 register in the Channel 2. It is expected that data for slot 1 and 12 should also get written into the Rx slot registers. At the end of the third frame, the data stored in the Rx slot registers are read to verify the correct reception.

During the fourth frame transmission, the following steps are performed. This frame should contain slots 3, 4 and 11 from the channels. Other slots are inhibited by the SRC bits in the previous frame. The data corresponding to the previous frame are read from the Rx FIFO of the Channel 1 and the Channel 2. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Tx FIFO of the Channel 2 is filled with data for slots 3, 4 and 11 for the next frame. The Slot0 transmitted by the trickbox has the CODEC ready and all the slot valid bits set. The AACI should receive all slots programmed in AACIRXCR1 register in the Channel 1 and all the slots programmed in AACIRXCR2 register in the Channel 2. It is expected that data for slot 1 and 12 should also get written into the Rx slot registers. At the end of the fourth frame, the data stored in the Rx slot registers are read to verify the correct reception.

During the fifth frame transmission, the following steps are performed. This frame should contain slots 5, 6, 7, 8, 9, 10 and 12 from the channels. Other slots are inhibited by the SRC bits in the previous frame. The data corresponding to the previous frame are read from the Rx FIFO of the Channel 1 and the Channel 2. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Slot0 transmitted by the trickbox has the CODEC ready set and all the slot valid bits are not set. Hence, the AACI should not receive any data, although AACIRXCR registers are programmed to receive the given slots.

During the sixth frame transmission, the following steps are performed. This frame should contain slots 3, 4 and 11 from the channels and slot 12 from the Tx slot register AACISL12TX. There are no other slots due to lack of the data in the Channel 1 and the Tx slot register AACISL1TX. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Slot0 transmitted by the trickbox has the CODEC ready set and all the slot valid bits are not set. Hence the AACI should not receive any data, even though AACIRXCR registers are programmed to receive the given slots

Finally, the Rx FIFO of the trickbox is read to verify the last frame transmitted by the AACI.

This basic sequence is repeated for another five data tests with the following variations.

- The number of slots enabled for transmission per channel
- The number of slots enabled per Rx channel
- The number of slots enabled per transmitted frame.
- The number of slots enabled per received frame.
- The slot numbers enabled in different channels are varied.

the previous frame are read from the Rx FIFO of Channel 1 and Channel 2. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Tx FIFO of the Channel 2 is filled with the data for slot 1 and the Tx slot register AACISL1TX is filled with a new data corresponding to the next frame. The Slot0 transmitted by the trickbox has the CODEC ready and all the slot valid bits set. The AACI should receive slot2 in the Channel 1 and slot12 in the Channel 2. It is expected that data for slot 1 and 12 also should get written into the Rx slot registers. At the end of the third frame, the data stored in the Rx registers are read to verify the correct reception

During the fourth frame transmission, the following steps are performed. This frame should contain slot 2 from the Channel 2 and slot 1 from the Tx slot registers. It is expected that this frame should not contain the data for slot 6 and 12 because there is no data in the Channel 1 and the Tx slot register AACISL12TX. The data corresponding to the previous frame are read from the Rx FIFO of the Channel 1 and the Channel 2. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Slot0 transmitted by the trickbox has the CODEC ready and all the slot valid bits set. The AACI should receive slot 2 in the Channel 1 and slot 12 in the Channel 2. It is expected that data for slot 1 and 12 should also get written into the Rx slot registers. At the end of the fourth frame, the data stored in the Rx registers are read to verify the correct reception.

Finally, the Rx FIFO of the trickbox is read to verify the last frame transmitted by the AACI.

This basic sequence is repeated for another two data tests with the following variations.

- The number of slots enabled per transmitted frame.
- The number of slots enabled per received frame.
- The slot numbers enabled in different channels are varied.
- TSize and RSize
- The AACISLnTX (n = 1, 2, 12) registers which are to be loaded with new data for each frame
- The AACISLnRX (n = 1, 2, 12) registers which are to be enabled for each frame.
- The AACISLnTX (n = 1, 2, 12) registers which are to be enabled for each frame
- Slot0 to be transmitted by the trickbox for each frame
- Whether data for Slots 1, 2 and 12 are present in Channels AND Slot registers (as against data being present for these slots ONLY in Slot registers or ONLY in channels)

Tx & Rx - CH & SLReg - Char mode - SRC:

The AACI is programmed for both transmission and reception in the character mode. AACISLnTX (n = 1, 2 and 12) are also enabled for transmission. AACISLnRX (n = 1, 2 and 12) are enabled for reception. Multiple SRC bits are set in the incoming frames. Six data tests are done under this category.

A sample test sequence is described:

The following AACITXCR registers are programmed for transmitting the mentioned slots.

AACITXCR1	Slots 6 with TSize = 20
AACITXCR2	Slots 4 with TSize = 12
AACITXCR3	Slots 3 with TSize = 16
AACITXCR4	Slots 1 with TSize = 18

The following AACITXCR registers are programmed for receiving the mentioned slots.

AACIRXCR1	Slots 1 with RSize = 20
AACIRXCR2	Slots 3 with RSize = 12
AACIRXCR3	Slots 9 with RSize = 16

AACIRXCR4 Slots 12 with RSize = 18

The Tx slot register AACISL1TX, AACISL2TX and AACISL12TX are enabled for transmission through the AACIMAINCR register. The Rx Slot register AACISL1RX, AACISL12RX and AACISL12RX are enabled for reception through the AACIMAINCR register.

The following SRC bits are set in the Slot1 of the frames transmitted by the trickbox:

Frame1	Slots 5 and 6
Frame2	Slots 3 and 4
Frame3	Slots 3 and 6
Frame4	Slot 4

The Tx FIFOs of the AACI and the Tx slot registers AACISL1TX, AACISL2TX and AACISL12TX are filled with the data for the first frame. The Tx FIFO of the trickbox is filled with data for two frames. The first frame transmitted should be invalid since the CODEC ready bit was set only in the first incoming frame. The Slot0 transmitted by the trickbox has the CODEC ready and all the slot valid bits set. The AACI should receive slots 1, 3, 9 and 12 in the Channel 1, 2, 3 and 4 respectively. It is expected that data for slot 1, 2 and 12 should also get written into the Rx slot registers AACISL1RX, AACISL2RX and AACISL12RX respectively. At the end of the first frame, the data stored in the Rx slot registers are read to verify the correct reception.

During the second frame transmission, the following steps are performed. This frame should contain slots 1 and 3 from the channels and slot 2 and 12 from the Rx slot register AACISL1TX and AACISL12TX respectively. Other slots (5 and 6) are inhibited by the SRC bits in the previous frame. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Tx FIFO of the Channel 3 and the Channel 4 are filled with the data for slots 3 and 1 respectively for the next frame. The Slot0 transmitted by the trickbox has the CODEC ready and all the slot valid bits set. The AACI should receive slots 1, 3, 9 and 12 in the Channel 1, 2, 3 and 4 respectively. It is expected that data for slot 1, 2 and 12 should also get written into the Rx slot registers AACISL1RX, AACISL2RX and AACISL12RX respectively. At the end of the second frame, the data stored in the Rx slot registers are read to verify the correct reception.

During the third frame transmission, the following steps are performed. This frame should contain slots 1 and 6 from the channels, Other slots (3 and 6) are inhibited by the SRC bits in the previous frame. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Tx FIFO of the Channel 1 is filled with the data for slot 6 for the next frame. The Slot0 transmitted by the trickbox has the CODEC ready and all the slot valid bits set. The AACI should receive slots 1, 3, 9 and 12 in the Channel 1, 2, 3 and 4 respectively. It is expected that data for slot 1, 2 and 12 should also get written into the Rx slot registers AACISL1RX, AACISL2RX and AACISL12RX respectively. At the end of the third frame, the data stored in the Rx slot registers are read to verify the correct reception.

During the fourth frame transmission, the following steps are performed. This frame should contain slot 4 from the Channel 2, and the slot 1 from the Tx slot register AACISL1TX. The SRC bits in the previous frame inhibit the slots 3 and 6 and there is no data for the slot 1 in the Channel 4. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Tx FIFO of the Channel 2 is filled with the data for the slot 4 for the next frame. The Slot0 transmitted by the trickbox has the CODEC ready and all the slot valid bits set. The AACI should receive slots 1, 3, 9 and 12 in the Channel 1, 2, 3 and 4 respectively. It is expected that data for slot 1, 2 and 12 should also get written into the Rx slot registers AACISL1RX, AACISL2RX and AACISL12RX respectively. At the end of the fourth frame, the data stored in the Rx slot registers are read to verify the correct reception.

During the fifth frame transmission, the following steps are performed. This frame should contain slot 3 and 6 from the Channels. The SRC bits in the previous frame inhibit the slots 4 and there is no data for the slot 1 in the Channel 4. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Slot0 transmitted by the trickbox

has the CODEC ready bit set and no slot valid bits are set. Hence, the AACI should not receive any data, although AACIRXCR registers are programmed to receive the given slots.

During the sixth frame transmission, the following steps are performed. This frame should contain only slot 4 from the Channel 2. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The Rx FIFO of the trickbox is read to verify the previous frame transmitted by the AACI. The Slot0 transmitted by the trickbox has the CODEC ready bit set and no slot valid bits are set. Hence, the AACI should not receive any data although AACIRXCR registers are programmed to receive the given slots.

Finally, the Rx FIFO of the trickbox is read to verify the last frame transmitted by the AACI.

- This basic sequence is repeated for another five data test with the following variations.
- The number of slots enabled per transmitted frame.
- The number of slots enabled per received frame.
- The slot numbers enabled in different channels are varied.
- TSize and RSize.
- The AACISLnTX (n = 1, 2, 12) registers which are to be loaded with new data for each frame.
- The AACISLnRX (n = 1, 2, 12) registers which are to be enabled for each frame.
- The AACISLnTX (n = 1, 2, 12) registers which are to be enabled for each frame.
- SRC bit-pattern to be transmitted by the trickbox for each frame.
- Slot0 to be transmitted by the trickbox for each frame.
- Whether data for Slots 1, 2 and 12 are present in Channels AND Slot registers (as against data being present for these slots ONLY in Slot registers or ONLY in channels).

Additionally the following functionality is also tested in all of the above AACI Simultaneous Transmit-Receive Tests that have the slots 1 and 2 valid.

The slot 1 data is written such that it indicates a CODEC register write (bit 19 of slot 1 is cleared) access for all the frame transmitted by the AACI except the third frame in the data test sequence. It is verified that the least significant four bits of the data for slot 2 transmitted by the AACI are stuffed with zeros by the AACI. The data for slot 1 in the third frame is written such that it indicates a CODEC register read access (bit 19 of slot 1 is set). It is verified that this time the AACI transmits invalid data for the slot 2.

AACI Transmit-only Tests

The variables TxSize[n] (n = 1, 2, 3, 4) and CmpctMod are initialised such that the elements in the TxSize array indicate TSize of 12, 16, 18 and 20 respectively and CmpctMod indicates non-compact mode. Then the Tx_Call() function is called to set the values of the control parameters required for transmission. The Tx_Call() function calls the InitCntlWrd() function to initialise all the ControlWord structure elements to their default values and reassign the value of the TxSize array to new values derived from the previous value of the same array. All the ControlWord structure elements which contain values that differ from the default values are assigned the values required for a particular test after the InitCntlWrd() function call. Then the function TxRx_FsPCLK_Test() is called with a parameter indicating the number of frames to be transmitted. The function TxRx_FsPCLK_Test() starts the transmission from the AACI.

The test sequence for all the following combinations are similar to the test sequences in the AACI Simultaneous Transmit-Receive Tests, but without setting the receive enables of the channels and the slot registers.

Tx Only - CH - FIFO mode - No SRC

The AACI is programmed for transmission-only in the FIFO mode and No SRC bits are set in the incoming frames. Ten tests are done under this category. The following parameters are varied for each test.

- The number of slots enabled for transmission per channel.

- The number of slots enabled per transmitted frame.
- The slot numbers enabled in different channels are varied.
- TSize

These test cases are repeated again with an even number of slots enabled per channel with the TxCompactMode bit set in the AACITXCRn (n = 1 to 4) register.

Tx Only - CH - FIFO mode - SRC

The AACI is programmed for transmission-only in the FIFO mode. Multiple SRC bits are set in the incoming frames. Two tests are done under this category. The following parameters are varied for each test.

- The number of slots enabled for transmission per channel.
- The number of slots enabled per transmitted frame.
- The slot numbers enabled in different channels are varied.
- SRC bit-pattern to be transmitted by the trickbox for each frame.
- The slot numbers enabled in different channels are varied.
- TSize

These test cases are repeated again with an even number of slots enabled per channel with the TxCompactMode bit set in the AACITXCRn (n = 1 to 4) register.

Tx Only - CH - Char mode - No SRC:

The AACI is programmed for transmission-only in the Character mode. No SRC bits are set in the incoming frames. Three tests are done under this category. The following parameters are varied for each test.

- The number of slots enabled for transmission per channel
- The number of slots enabled per transmitted frame.
- The slot numbers enabled in different channels are varied
- TSize

Tx Only - CH - Char mode - SRC:

The AACI is programmed for transmission-only in the Character mode. Multiple SRC bits are set in the incoming frames. Three tests are done under this category. The following parameters are varied for each test.

- The number of slots enabled for transmission per channel
- The number of slots enabled per transmitted frame.
- The slot numbers enabled in different channels are varied
- SRC bit-pattern to be transmitted by the trickbox for each frame.
- TSize

Tx Only - SLReg - No SRC:

The AACI is programmed for transmission-only through the AACISLnTX (n = 1, 2 and 12) registers. No SRC bits are set in the incoming frames. Two data tests are done under this category. The following parameters are varied for each test.

- The AACISLnTX (n = 1, 2, 12) registers which are to be loaded with new data for each frame
- The AACISLnTX (n = 1, 2, 12) register which are to be enabled for each frame

Tx Only - SLReg - SRC:

The AACI is programmed for transmission-only through the AACISLnTX (n = 1, 2 and 12) registers. Multiple SRC bits are set in the incoming frames. Two data tests are done under this category. The following parameters are varied for each test.

- The AACISLnTX (n = 1, 2, 12) registers are to be loaded with new data for each frame
- The AACISLnTX (n = 1, 2, 12) registers are to be enabled for each frame
- SRC bit-pattern to be transmitted by the trickbox for each frame

Tx Only - CH & SLReg - FIFO mode - No SRC:

The AACI is programmed for transmission-only in the FIFO mode. AACISLnTX (n = 1, 2 and 12) are also enabled for transmission. No SRC bits are set in the incoming frames. Five data tests are done under this category. The following parameters are varied for each test.

- The number of slots enabled for transmission per channel
- The number of slots enabled per transmitted frame.
- The slot numbers enabled in different channels are varied
- TSize
- The AACISLnTX (n = 1, 2, 12) registers which are to be loaded with new data for each frame
- Whether data for Slots 1, 2 and 12 are present in Channels AND Slot registers (as against data being present for these slots ONLY in Slot registers or ONLY in channels)

These test cases are repeated again with an even number of slots enabled per channel with the TxCompactMode bit set in the AACITXCRn (n = 1 to 4) register.

Tx Only - CH & SLReg - FIFO mode - SRC:

The AACI is programmed for transmission-only in the FIFO mode. AACISLnTX (n = 1, 2 and 12) are also enabled for transmission. Multiple SRC bits are set in the incoming frames. Four data tests are done under this category. The following parameters are varied for each test.

- The number of slots enabled for transmission per channel
- The number of slots enabled per transmitted frame
- SRC bit-pattern to be transmitted by the trickbox for each frame
- The slot numbers enabled in different channels are varied
- TSize
- The AACISLnTX (n = 1, 2, 12) registers which are to be loaded with new data for each frame
- Whether data for Slots 1, 2 and 12 are present in Channels AND Slot registers (as against data being present for these slots ONLY in Slot registers or ONLY in channels)

These test cases are repeated again with an even number of slots enabled per channel with the TxCompactMode bit set in the AACITXCRn (n = 1 to 4) register.

Tx Only - CH & SLReg - Char mode - No SRC:

The AACI is programmed for transmission-only in the Character mode. AACISLnTX (n = 1, 2 and 12) registers are also enabled for transmission. No SRC bits are set in the incoming frames. Three data tests are done under this category. The following parameters are varied for each test.

- The number of slots enabled for transmission per channel
- The number of slots enabled per transmitted frame
- The slot numbers enabled in different channels are varied
- TSize

- The AACISLnTX (n = 1, 2, 12) registers which are to be loaded with new data for each frame
- Whether data for Slots 1, 2 and 12 are present in Channels AND Slot registers (as against data being present for these slots ONLY in Slot registers or ONLY in channels)

Tx Only - CH & SLReg - Char mode - SRC:

The AACI is programmed for transmission-only in the Character mode. AACISLnTX (n = 1, 2 and 12) registers are also enabled for transmission. Multiple SRC bits are set in the incoming frames. Three data tests are done under this category. The following parameters are varied for each test.

- The number of slots enabled for transmission per channel
- The number of slots enabled per transmitted frame
- The slot numbers enabled in different channels are varied
- SRC bit-pattern to be transmitted by the trickbox for each frame
- TSize
- The AACISLnTX (n = 1, 2, 12) registers which are to be loaded with new data for each frame
- Whether data for Slots 1, 2 and 12 are present in Channels AND Slot registers (as against data being present for these slots ONLY in Slot registers or ONLY in channels)

Additionally the following functionality is also tested in all of the above AACI Transmit-Only Tests that have the slots 1 and 2 valid.

The slot 1 data is written such that it indicates a CODEC register write (bit 19 of slot 1 is cleared) access for all the frame transmitted by the AACI except the third frame in the data test sequence. It is verified that the least significant four bits of the data for slot 2 transmitted by the AACI are stuffed with zeros by the AACI. The data for slot 1 in the third frame is written such that it indicates a CODEC register read access (bit 19 of slot 1 is set). It is verified that this time the AACI transmits invalid data for the slot 2.

AACI Receive-only Tests

The variables RxSize[n] (n = 1, 2, 3, 4) and CmpctMod are initialised such that the elements in the RxSize array indicate RSize of 12, 16, 18 and 20 respectively and CmpctMod indicates non-compact mode. Then the Rx_Call() function is called to set the values of the control parameters required for reception. The Rx_Call() function calls the InitCntlWrd() function to initialise all the ControlWord structure elements to their default values and reassign the value of the RxSize array to new values derived from the previous value of the same array. All the ControlWord structure elements which contain values that differ from the default values are assigned the values required for a particular test after the InitCntlWrd() function call. Then the function TxRx_FsPCLK_Test() is called with a parameter indicating the number of frames to be received. The function TxRx_FsPCLK_Test() starts the reception in the AACI.

The test sequence for all the following combinations are similar to the test sequences in the AACI Simultaneous Transmit-Receive Tests, but without setting the transmit enables of the channels and the slot registers.

Rx Only - CH - FIFO mode:

AACI is programmed for reception-only in the FIFO mode. Three data tests are done under this category. The following parameters are varied for each test.

- The number of slots enabled for reception per channel
- The number of slots enabled per received frame.
- The slot numbers enabled in different channels are varied
- Slot0 to be transmitted by the trickbox for each frame
- RSize

These test cases are repeated again with an even number of slots enabled per channel with the RxCompactMode bit set in the AACIRXCRn (n = 1 to 4) register.

Rx Only - CH - Char mode:

The AACI is programmed for reception-only in Character mode. Three data tests are done under this category. The following parameters are varied for each test under this category.

- The number of slots enabled for reception per channel
- The slot numbers enabled in different channels are varied
- Slot0 to be transmitted by the trickbox for each frame
- RSize

In addition, the AACI is programmed with the AACIRXCRn (n = 1 to 4) enabled for more than one slot in Character mode. This is to verify that the AACI prevents overwriting of data.

Rx Only - SLReg:

The AACI is programmed for reception-only through the AACISLnRX (n = 1, 2 and 12) registers. Three data tests are done under this category. The following parameters are varied for each test under this category.

- The AACISLnRX (n = 1, 2, 12) registers which are to be enabled for each frame
- The number of slots enabled per received frame.
- Slot0 to be transmitted by the trickbox for each frame

Rx Only - CH & SLReg - FIFO mode:

The AACI is programmed for reception-only in the FIFO mode. AACISLnRX (n = 1, 2 and 12) are also enabled for reception. Three data tests are done under this category. The following parameters are varied for each test under this category.

- The number of slots enabled per Rx channel
- The AACISLnRX (n = 1, 2, 12) registers which are to be enabled for each frame
- The slot numbers enabled in different channels are varied
- The number of slots enabled per received frame.
- Slot0 to be transmitted by the trickbox for each frame
- RSize

These test cases are repeated again with an even number of slots enabled per channel with the RxCompactMode bit set in the AACIRXCRn (n = 1 to 4) register.

Rx Only - CH & SLReg - Char mode

The AACI is programmed for reception-only in Character mode. AACISLnRX (n = 1, 2 and 12) are also enabled for reception. Six data tests are done under this category. The following parameters are varied for each test under this category.

- The AACISLnTX (n = 1, 2, 12) registers which are to be loaded with new data for each frame
- The AACISLnTX (n = 1, 2, 12) registers which are to be enabled for each frame
- The slot numbers enabled in different channels are varied
- The number of slots enabled per received frame.
- Slot0 to be transmitted by the trickbox for each frame
- RSize

In addition, the AACI is programmed with the AACIRXCRn (n = 1 to 4) enabled for more than one slot in Character mode. This is to verify that the AACI prevents overwriting of data.

These test cases are repeated again with an even number of slots enabled per channel with the RxCompactMode bit set in the AACIRXCRn (n = 1 to 4) register.

6.4.12 Loop back Test

The variables TxSize[n] (n = 1, 2, 3, 4) and CmpctMod are initialised such that the elements in the TxSize array indicate TSize of 12, 16, 18 and 20 respectively and CmpctMod indicates non-compact mode. The variables RxSize[n] (n = 1, 2, 3, 4) and CmpctMod are initialised such that the elements in the RxSize array indicate RSize of 12, 16, 18 and 20 respectively and CmpctMod indicates non-compact mode. Then TxRx_FsPCLK_Call() function is called to set the values of the control parameters required for transmission and reception. The TxRx_FsPCLK_Call() function calls the InitCntlWrd() function to initialise all the ControlWord structure elements to their default values and reassigns the value of the TxSize and RxSize array to new values derived from the previous value of the same array. All the ControlWord structure elements which contain values which are different from the default values are assigned to the values required for a particular test after the InitCntlWrd() function call. Then the function LoopBack_Test() is called with a parameter indicating the number of frames to be transmitted and received. The function LoopBack_Test() starts the transmission and reception in loop back mode.

The AACI is programmed in the Loop back mode. The Channels are enabled in the FIFO mode. AACISLnTX (n = 1, 2 and 12) registers are enabled for transmission and AACISLnRX (n = 1, 2 and 12) registers are enabled for reception. No SRC bits are set in the incoming frames. Four data tests are done under this category.

A sample test sequence is described:

The following AACITXCR registers are programmed for transmitting the mentioned slots.

AACITXCR1	Slots 8 and 10 with TSize = 20
AACITXCR2	Slots 4 and 11 with TSize = 20
AACITXCR3	Slots 3 and 7 with TSize = 20
AACITXCR4	Slots 6 and 12 with TSize = 20

The following AACITXCR registers are programmed for receiving the mentioned slots.

AACIRXCR1	Slots 6, 9 and 12 with RSize = 20
AACIRXCR2	Slots 3, 7 and 10 with RSize = 20
AACIRXCR3	Slots 1, 2, and 4 with RSize = 20
AACIRXCR4	Slots 5, 8 and 11 with RSize = 20

The Rx Slot register AACISL1RX, AACISL2RX and AACISL12RX are enabled for reception through the AACIMAINCR register.

The Tx FIFOs of the AACI are filled with the data for the first frame. Enabling the LoopBack bit in the AACIMAINCR, register starts the transmission and reception in the AACI. The first frame transmitted should be invalid. In Loop back mode, the output port AACISDATAOUT is connected to the input port AACISDATAIN of the AACI. It is expected that AACI should not receive any data in the first frame, because the slot 0 transmitted by the AACI has no slot valid bits set.

During the second frame transmission, the following steps are performed. This frame should contain slots 3, 4, 6, 7, 8, 10, 11 and 12 from the channels. The Tx FIFOs of the AACI are filled with the data for slots 3, 4, 6, 7, 8, 10, 11 and 12 corresponding to the next frame. The slot 0 transmitted by the AACI has CODEC ready and the valid bits of slots 3, 4, 6, 7, 8, 10, 11 and 12 are set. Hence, it is expected that the AACI should receive only slots 6 and 12 in the Channel 1 and slots 3, 7 and 10 in the Channel 2 and slot 4 in the Channel 3 and slots 8 and 11 in the Channel 4. The slots 1, 2, 5 and 9 are rejected by the AACI although these slots are programmed in the

AACIRXCR registers since the slot valid bits of these slots are not set in the received slot 0. Also, It is expected that data for slot 12 should get written into the Rx slot registers. At the end of the second frame, the data stored in the Rx registers are read to verify the correct reception.

During the third frame transmission, the following steps are performed. This frame should contain slots 3, 4, 6, 7, 8, 10, 11 and 12 from the channels. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The Tx FIFOs of the AACI are filled with the data for slots 3, 4, 6, 7, 8, 10, 11 and 12 corresponding to the next frame. The slot 0 transmitted by the AACI has CODEC ready and the valid bits of slots 3, 4, 6, 7, 8, 10, 11 and 12 set. Hence, it is expected that the AACI should receive only slots 6 and 12 in the Channel 1 and slots 3, 7 and 10 in the Channel 2 and slot 4 in the Channel 3 and slots 8 and 11 in the Channel 4. The slots 1, 2, 5 and 9 are rejected by the AACI, although these slots are programmed in the AACIRXCR registers. In addition, it is expected that data for slot 12 should get written into the Rx slot registers. At the end of the third frame, the data stored in the Rx registers are read to verify the correct reception.

During the fourth frame transmission, the following steps are performed. This frame should contain slots 3, 4, 6, 7, 8, 10, 11 and 12 from the channels. The data corresponding to the previous frame are read from the Rx FIFOs of the AACI. The slot 0 transmitted by the AACI has CODEC ready and the valid bits of slots 3, 4, 6, 7, 8, 10, 11 and 12 set. Hence, it is expected that the AACI should receive only slots 6 and 12 in the Channel 1 and slots 3, 7 and 10 in the Channel 2 and slot 4 in the Channel 3 and slots 8 and 11 in the Channel 4. The slots 1, 2, 5 and 9 are rejected by the AACI, although these slots are programmed in the AACIRXCR registers. In addition, it is expected that data for slot 12 should be being written into the Rx slot registers. At the end of the fourth frame, the data stored in the Rx registers are read to verify the correct reception.

Finally, the Rx FIFOs of the AACI is read to verify the last frame transmitted by the AACI.

This basic sequence is repeated for another three Loop back tests with the following variations.

- The number of slots enabled for transmission per channel
- The number of slots enabled for reception per channel
- The number of slots enabled per transmitted frame
- The number of slots enabled per received frame
- The slot numbers enabled in different channels are varied
- TSize and RSize

The above test cases are repeated again with an even number of slots enabled per channel with the TxCompactMode bit set in the AACITXCRn (n = 1 to 4) and the RxCompactMode AACIRXCRn (n = 1 to 4) registers.

6.5 Verification Test Vector Generation

A **make sourcefilename** (without the .c extension) in the **verification/bustest** directory will create **.bif** formatted and **.sim** formatted vectors in the **verification/bustest/invec** directory. This is the default setting. The makefile in **verification/bustest** directory can be referred for the make options.

make all can be used to create a test vector file including all the tests.

7 AACI INTEGRATION TEST DETAILS

The AACI integration test vector suite allows the integrator to verify that the AACI has been connected into the target system correctly. These vectors test 3 classes of signals:

- a) AMBA signals (AACI signals connected directly to the APB)
- b) Intra-Chip signals (AACI signals connected to the DMA and Interrupt Controllers)
- c) Primary I/O signals (AACI signals connected to the chip pads)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref. 6].

The AACI Integration testbench is based on an AHB TICTalk testbench. The VHDL and Verilog TICTalk testbenches are used to verify that the AACI has been connected into the target system correctly. The VHDL and Verilog TICTalk testbench files reside in the ***integration/vhdl*** and ***integration/verilog*** directories respectively. The test vectors that are applied to the TICTalk testbenches are in the ***integration/bustest*** directory. The testvectors for integration testing are written in BusTalk and then converted into ***.tifl.sim*** format to be applied to the TICTalk integration testbench.

7.1 AACI VHDL Integration testbench

The AMBA TICTalk VHDL testbench used for integration testing of the AACI is located in the ***integration/vhdl/tbench*** directory. The Integration test vectors are located in the ***integration/bustest*** directory.

The test vectors used with this testbench can be applied to the AACI when it is part of an AMBA system design.

Figure 7.1-1 illustrates the testbench to apply TICTalk test vectors.

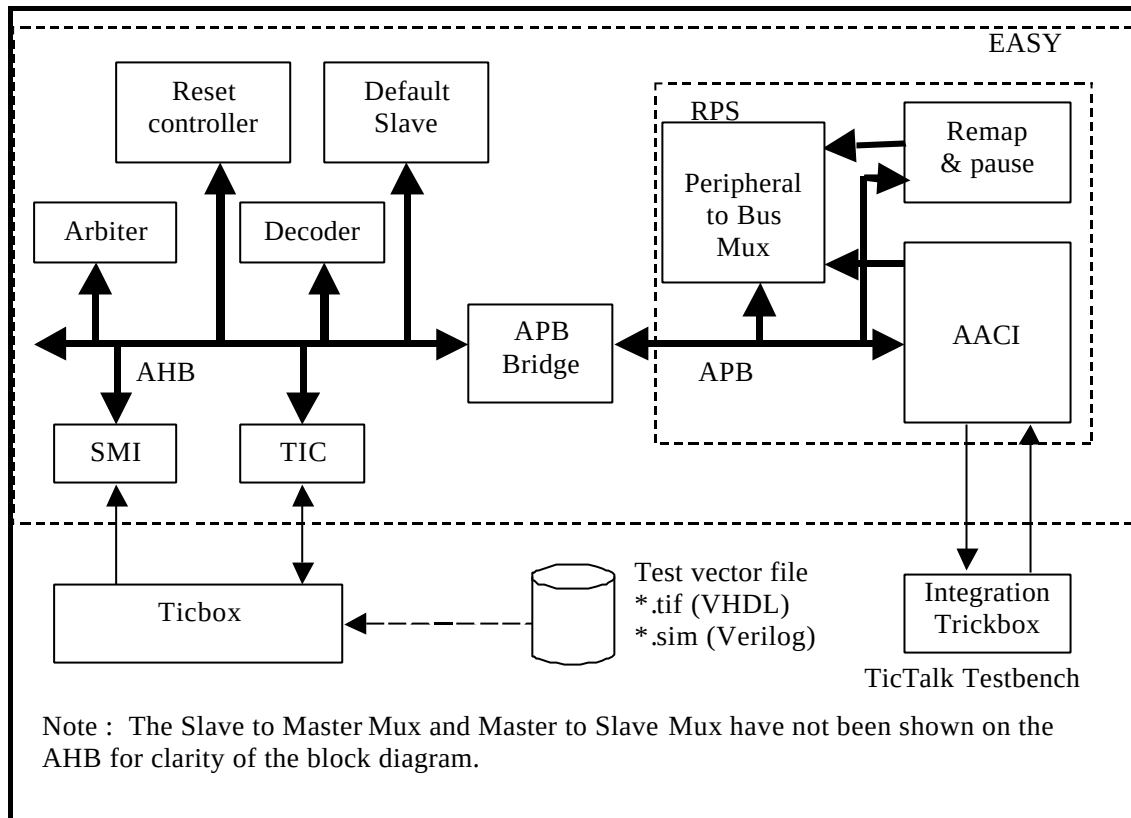


Figure 7.1-1 Testbench for simulating TICTalk test vectors

The AACI VHDL Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 7.1-2 illustrates the hierarchy for the TICTalk testbench.

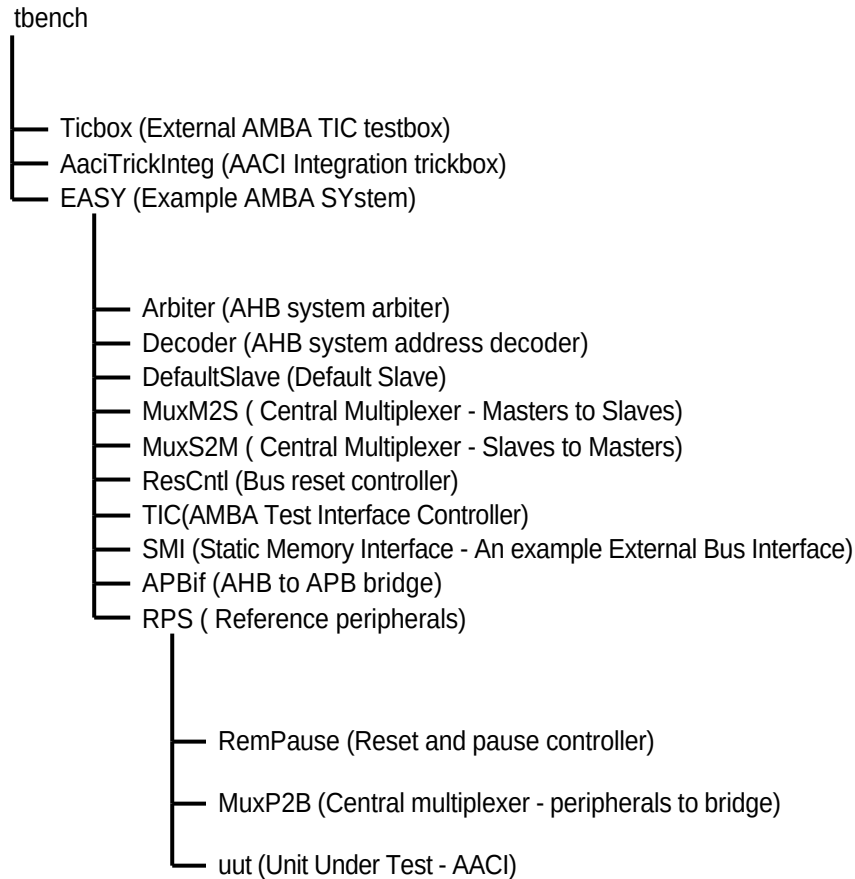


Figure 7.1-2 Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.vhd**, the top-level of the testbench, which is used for running the integration tests of the AACI. This module instantiates the model of the EASY microcontroller, the integration trickbox and the ticbox. The AACI is instantiated in the **rps.vhd**, which is a part of the EASY system. The AACIBITCLK is driven by the integration trickbox. All the Scan input pins are tied low to keep them from interfering with the test process.

The integration trickbox is used to give a loopback facility to test the primary I/O pins of the AACI, and also to generate the AACIBITCLK. For further details on the integration trickbox, refer to **Section 7.3**.

7.1.1 Unit Under Test

The Unit Under Test may be one of the following

- AACI VHDL RTL
- AACI VHDL netlist from VHDL RTL Source

One out of these two versions may be referenced via the corresponding link to the uut directory in **integration/vhdl**.

7.1.2 Test Vectors

The test vectors for the integration testing of the AACI are coded into separate C files for testing the AACI registers and for Integration testing. Make options are provided to run the tests together or individually.

7.1.3 Running Simulations

The **README** file in the *integration/vhdl/tbench* directory gives step by step instructions for running simulations using the AACI Integration test vectors on the VHDL source code.

Toggle coverage numbers (for block-level I/Os) when the integration test vectors are run on the VHDL RTL and netlist forms of the AACI are available in the *integration/vhdl/Aaci_rtl* and *integration/vhdl/Aaci_scaninsert_vhdl_net_max* directories.

Aaci_rtl_modelsim.untog

Aaci_scaninsert_vhdl_vhdl_net_max_modelsim.untog

A sample toggle report (*.untog) file is shown below:

Toggle Report	Node	-> 0	->1
	Signal 1	x	y
	Signal 2	a	b
Total Node Count	= C		
Toggled Node Count	= T		
Untoggled Node Count	C-T = U		
Toggle Coverage	T/C = P %		

The column "Node" in the .untog file lists the individual nodes that are not toggled when the test vectors are run on the VHDL in RTL or netlist forms, whichever the case may be. The column "->0" indicates the number of times that particular node has toggled to 0 and the column "->1" indicates the number of times it has toggled to 1. In the above example, the Node "Signal 1" toggles to 0 "x" times and it toggles to 1 "y" times. The "Total Node Count", indicates the total number of I/O nodes in the block and the "Toggled Node Count" indicates the number of these nodes that are toggled when the test vectors are run. The number "Untoggled Node Count" indicates the number of I/O nodes in the block that are not toggled when the test vectors are run and the "Toggle Coverage" gives the percentage of the total number of nodes in the block that are toggled when the test vectors are run.

Run time logs for all the combinations are available in the following files in the *integration/vhdl/Aaci_rtl* and *integration/vhdl/Aaci_scaninsert_vhdl_net_max* directories.

Aaci_rtl_modelsim.log

Aaci_scaninsert_vhdl_vhdl_net_max_modelsim.log

7.2 AACI Verilog Integration testbench

The AMBA TICTalk Verilog testbench used for integration testing of the AACI is located in the *integration/verilog/tbench* directory. The Integration test vectors are located in the *integration/bustest/invec* directory.

The test vectors used with this testbench can be applied to the AACI when it is part of an AMBA system design.

Figure 7.2-1 illustrates the testbench to apply TICTalk test vectors.

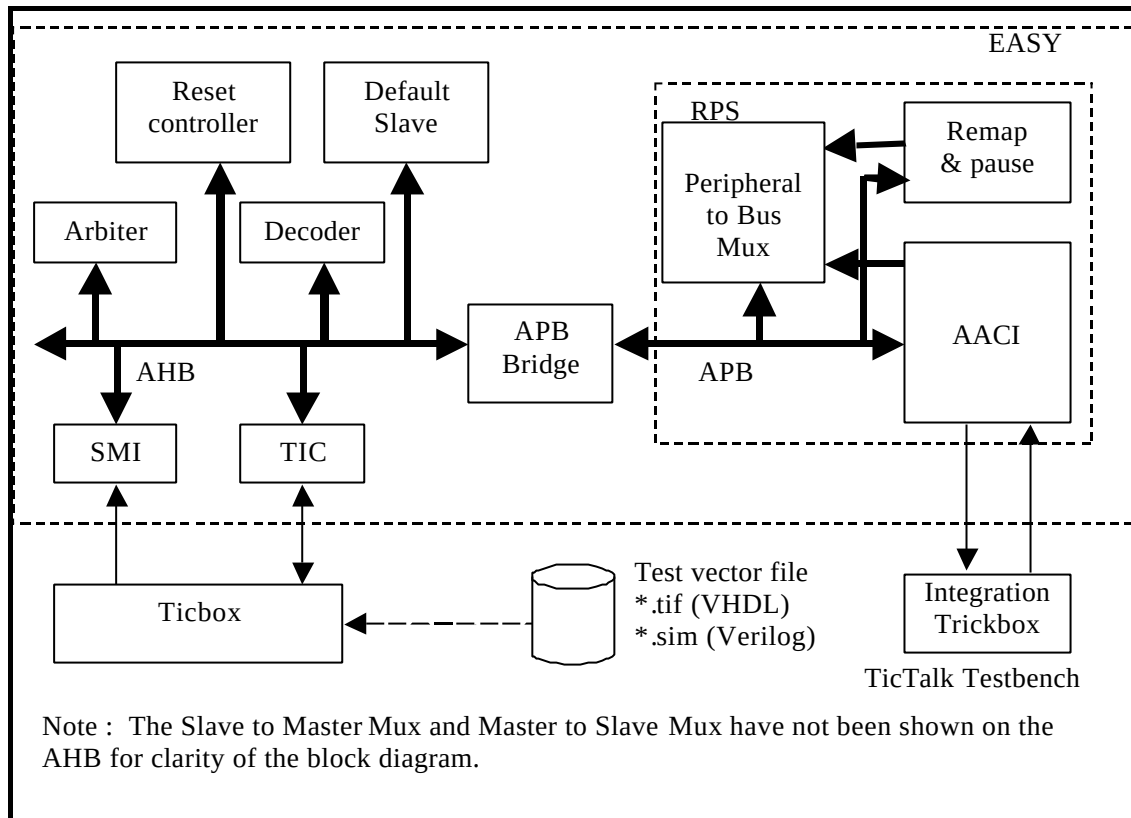


Figure 7.2-1 Testbench for simulating TICTalk test vectors

The AACI Verilog Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 7.2-2 illustrates the hierarchy for the TICTalk testbench.

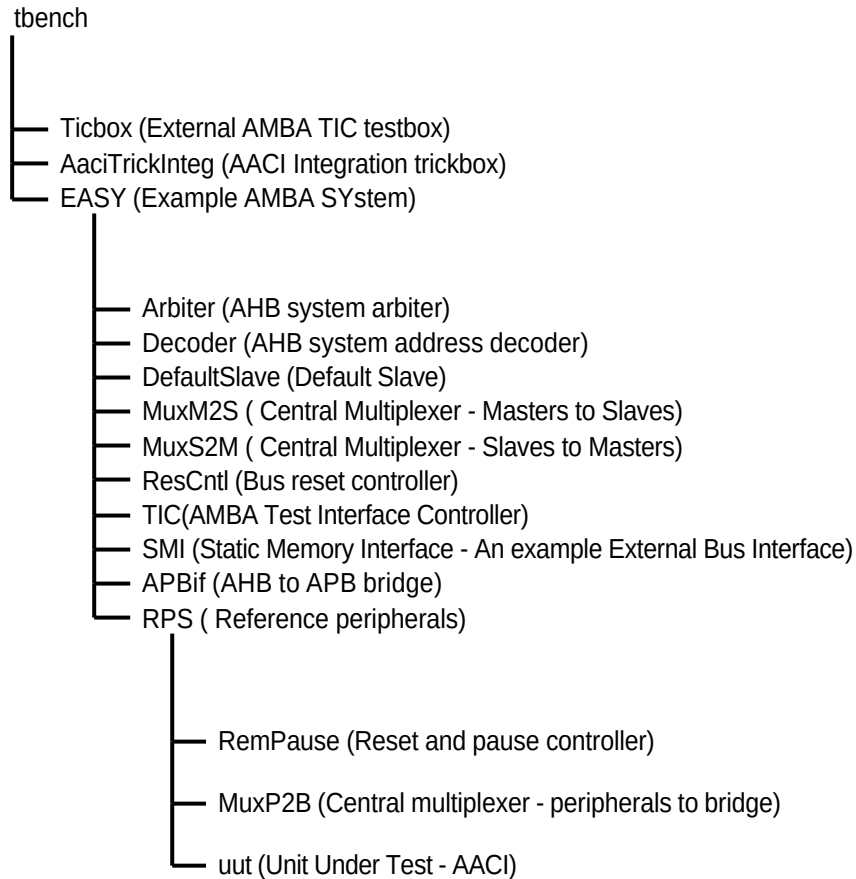


Figure 7.2-2 Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.v**, the top-level of the testbench, which is used for running the integration tests of the AACI. This module instantiates the model of the EASY microcontroller, the integration trickbox and the ticbox. The AACI is instantiated in the **rps.v**, which is a part of the EASY system. The AACIBITCLK is driven by the integration trickbox. All the Scan input pins are tied low to keep them from interfering with the test process.

The integration trickbox is used to give a loopback facility to test the primary I/O pins of the AACI, and also to generate the AACIBITCLK. For further details on the integration trickbox, refer to **Section 7.3**.

7.2.1 Unit Under Test

The Unit Under Test may be one of the following

- AACI VERILOG RTL
- AACI VERILOG netlist from VHDL RTL Source
- AACI VERILOG netlist from VERILOG RTL Source

One out of these two versions may be referenced via the corresponding link to the uut directory in **integration/verilog**.

7.2.2 Test Vectors

The test vectors for the integration testing of the AACI are coded into separate C files for testing the AACI registers and for Integration testing. Make options are provided to run the tests together or individually.

7.2.3 Running Simulations

The **README** file in the *integration/verilog/tbench* directory gives step by step instructions for running simulations using the AACI Integration test vectors on the VERILOG source code.

Toggle coverage numbers (for block-level I/Os) when the integration test vectors are run on the VHDL RTL and netlist forms of the AACI are available in the *integration/verilog/Aaci_rtl* and *integration/verilog/Aaci_noscan_vhdl_net_min* directories.

Aaci_rtl_modelsim.untog

Aaci_noscan_vhdl_verilog_net_min_modelsim.untog

A sample toggle report (*.untog) file is shown below:

Toggle Report	Node	-> 0	->1
	Signal 1	x	y
	Signal 2	a	b

Total Node Count	= C
Toggled Node Count	= T
Untoggled Node Count	C-T = U

Toggle Coverage	T/C = P %
-----------------	-----------

The column "Node" in the .untog file lists the individual nodes that are not toggled when the test vectors are run on the Verilog in RTL or netlist forms, whichever the case may be. The column "->0" indicates the number of times that particular node has toggled to 0 and the column "->1" indicates the number of times it has toggled to 1. In the above example, the Node "Signal 1" toggles to 0 "x" times and it toggles to 1 "y" times. The "Total Node Count", indicates the total number of I/O nodes in the block and the "Toggled Node Count" indicates the number of these nodes that are toggled when the test vectors are run. The number "Untoggled Node Count" indicates the number of I/O nodes in the block that are not toggled when the test vectors are run and the "Toggle Coverage" gives the percentage of the total number of nodes in the block that are toggled when the test vectors are run.

Run time logs for all the combinations are available in the following files in the *integration/verilog/Aaci_rtl*, *integration/verilog/Aaci_noscan_vhdl_net_min* and *integration/verilog/Aaci_scanready_verilog_net_typ* directories.

Aaci_rtl_modelsim.log

Aaci_rtl_LDV.log

Aaci_noscan_vhdl_verilog_net_min_modelsim.log

Aaci_scanready_verilog_verilog_net_typ_LDV.log

7.3 Integration Trickbox

Integration testing of primary outputs and primary inputs are carried out with the help of the Integration Vector Trickbox.

7.3.1 Integration Trickbox Signal Description

Name	Type	Source / Destination	Description
AACISDATAOUT	Input	From AACI	AACI Serial data output
AACIRESET	Input	From AACI	AACI RESET port
AACISYNC	Input	From AACI	AACI SYNC port
AACISDATAIN	Output	To AACI	AACI Serial data input
AACIBITCLK	Output	To AACI	AACI Serial clock

Table 7.3-1 AACI Integration Trickbox Signal Description

7.3.2 Integration Trickbox Functional Description

The primary outputs of AACI are routed back to its primary inputs by ORing all the primary outputs. A separate clock generator is used to generate the AACIBITCLK to drive the serial clock of the AACI, AACIBITCLK.

Figure 7.3-1 shows the block diagram of the AACI Integration Vector Trickbox.

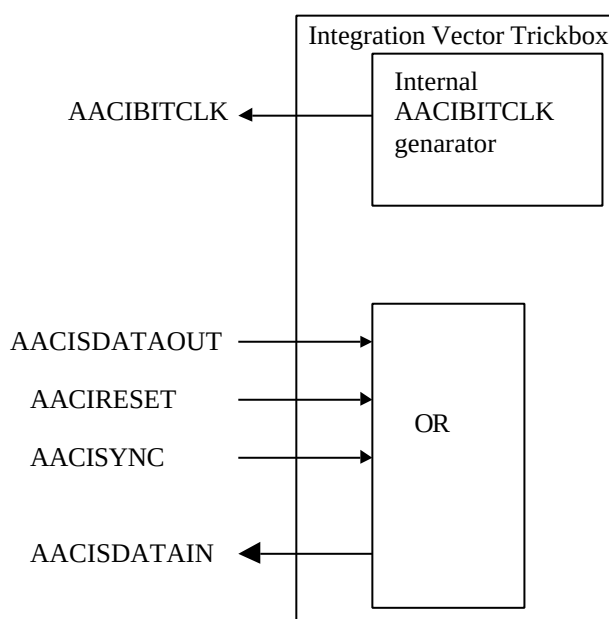


Figure 7.3-1 AACI Integration Vector Trickbox

7.4 Integration Tests

When the AACI is used in the AMBA system, it must be ensured that all the I/O pins are correctly connected. Integration vectors allow the user to verify that the AACI has been wired into the system correctly.

This test is performed with `IntegrationTest()` and `Register test()` and `AddrRangeTest()` function calls.

The `RegisterTest()` and `AddrRangeTest()` function call is used to increase the toggle coverage of the address lines. In the `RegisterTest()` function the reset inputs (`PRESETn` and `nAACIBITCLKRST`) are asserted. When the reset inputs are deasserted all the registers in the AACI are read back to check for the default values. In the `AddrRangeTest()` the address range of the FIFOs in the AACI is tested.

7.4.1 Integration Testing of primary inputs and outputs

Signals : `AACISDATAOUT`, `AACISYNC`, `AACIRESET` and `AACISDATAIN` , `AACIBITCLK`

The connectivity of primary outputs and inputs are checked by looping the primary outputs back into primary inputs through the Integration test trickbox. '1's and '0's are driven on the primary outputs through the corresponding bit in the `AACIITOP0`, `AACISYNC`, `AACIRESET` registers and read via the primary input bit in the `AACIITIP` register.

The Integration trickbox ORs all the primary outputs of the AACI (`AACISYNC`, `AACIRESET` and `AACISDATAOUT`) and feeds back to the primary input (`AACISDATAIN`) of the AACI.

Here, the test code ensures that only one primary output is high at any any instant when reading back from the primary input (which is ORing of the all the primary outputs of the AACI).

Clock inputs signals:

The connectivity of the `AACIBITCLK` input signal is tested indirectly by initiating data transfers in the loopback mode.

7.4.2 Integration Testing of intra-chip outputs

Signals : `AACIRXINTRn`, `AACITXINTRn`, `AACIORINTRn`, `AACIURINTRn`, `AACIRXTOINTRn`,

`AACIRXTOFEINTRn`, `AACIORINTRn`, (n = 1 to 4)

`AACIWINTR`, `AACIGPIOINTR`, `AACISmTXINTR`, `AACISmRXINTR` (m = 1, 2, 12)

`AACIDMABREQRX`, `AACIDMASREQRX`, `AACIDMALBREQRX`, `AACIDMALSREQRX`,

`AACIDMABREQTX`

When Integration tests are run with the AACI in a stand-alone test setup :

- Write a '1' to the `ITEN` bit in the Control register. This selects the test path from the corresponding bit the `AACIITIP` register bits to the intra-chip output signals.
- Write a '1' and then a '0' to the corresponding bit in the `AACIITOP` register and read the same register bits to verify that the value written is read out.

When Integration tests are run in an integrated system, '1's and '0's written to the `AACIITOP` register bits to toggle the signal connections between the AACI and the DMA Controller / Interrupt Controller's internal register to verify that the value written into the `AACIITOP` register bits are to be read out through the DMA Controller / Interrupt Controller's `ITIP` registers.

So the user needs to replace the read commands from the `AACIITOP` register by the suitable command to read from the `ITIP` registers of the DMA controller/Interrupt controller.(i.e. Destination peripheral of the signal)

7.4.3 Integration Testing of intra-chip inputs

Signals : AACIDMACLRRX, AACIDMACLRTX.

When Integration tests are run with the AACI in a stand-alone test setup:

- Write a '1' to the ITEN bit in the Control register. This selects the test path from the corresponding bit in the AACIITIP register to the internal AACIDMACLR signal.
- Write a '1'(Set the AACIDMACLR input) and then a '0'(Clear the AACIDMACLR input) to the corresponding bit in the AACIITIP register and read the same register bit to ensure that the value written is read out.

When Integration tests are run in an integrated system '1' and '0' are actually written into internal registers (ITIP registers of DMA controller i.e Source peripheral of the signal) of DMA controller and the connection is verified by reading out from the corresponding bit in the AACIITIP register. User should ensure that the ITEN bit in the AACITCR register is made '0'.

7.5 Integration Test Vector Generation

A **make sourcefilename** (without the .c extension) in the **integration/bustests** directory will create **.tif** formatted and **.sim** formatted vectors in the **integration/bustest/invec** directory. This is the default setting. The makefile in **integration/bustest** directory can be referred for the make options.

make all can be used to create a test vector file including all the tests.